MSX turboR Technical Hand Book

English translation from the Japanese original

Contents

1.3 Making Full Use of PCM to the Limit 21
1.3.1 Basics - How to Use with BASIC 21
2.1 What is a Slot? 27
2.1 What are Slots? 29
2.2. Challenge of Slot Switching 32
2.2.1. About Slot Switching 32
2.2.2. How to Specify Slot Numbers 33
2.2.6 MSX2+ Hardware Specifications 36
3.1.6 Kanji Text and Kanji Graphics 43
Section 5 - MSX-MUSIC 78
5.1.3 The Pitch is not Restricted to Equal Temperament 78
5.2 FM Sound Source Control 88
5.2.3 Compiling with MSX-C 89
5.3 FM Sound Source Data Structure 90
5.3.2 Specifying Percussion Data 91
5.3 FM Sound Data Construction 91

”MSX Magazine Editorial Department Supervision MSX turbo R Technical Handbook ASCII Publishing Bureau”

The English title on the book is ”MSX turbo R Technical Hand Book”.

(Blank page in the original.)

MSX turbo R Technical Hand Book

ASCII Publishing Bureau

(Blank page in the original.)

MSX turbo R Technical Hand Book

At the bottom of the page:

ASCII Publishing Bureau

Preface

The following content details the internal information necessary for maximizing the usage of the MSX personal computer, which has become powerful enough to rival high-performance CPUs and large amounts of memory.

1. Converting internal memory to 16-bit to achieve more than 10 times the processing capacity of conventional MSX compared to the R800 CPU, and how to manage its heat.

While the diagrams excluding the first page and inner part were roughly sketched, this publication was compiled using ASCII-made, Japanese TeX. We thank the author of msdos.sty, Mr. ishii@cts.dnp.co.jp, and the publishing technique division. Thank you also to everyone for drawing the main illustrations roughly.

To keep the content concise this time, there are no notes on memory mappers related to "Japanese MSX-DOS2." This will be explained in the "Japanese MSX-DOS2 Technical Handbook (provisional title)," which is scheduled to be published soon.

Introduction

Welcome to the world of MSX turbo R. This book details the internal information necessary for maximizing the use of the MSX personal computer, which has become exceptionally powerful due to its high-speed CPU and large-capacity memory.

1. Transformation to a 16-bit architecture, allowing the high-speed CPU, R800, to deliver processing speeds more than 10 times faster compared to previous MSX systems, and drawing out graphics.
2. Details and techniques for using PCM sound source and FM sound source, which are standard features in the MSX turbo R.
3. The structure and handling of the SLOT mechanism necessary for using MSX.
4. The structure of Kanji BASIC required for developing Japanese software.
5. Methods for drawing out techniques for screen display using VDP.

The MSX turbo R greatly changes the architecture of conventional machines, implementing high-performance capabilities with a transformed 16-bit CPU, making it a first-time personal computer.

In other machines, when transitioning from 8-bit to 16-bit, the architecture had to be completely changed, resulting in the loss of software and know-how developed on the 8-bit machines. We considered how to utilize and enhance the know-how born from the experience of users and the resources of hardware and software exclusively developed for MSX, and arrived at the method of realizing these new and higher-level MSX models by stacking the CPU on top of the traditional MSX Z80, along with the high-performance R800 newly developed for compatibility.

Thus, the MSX turbo R, equipped with both the new R800 and the conventional Z80, was

developed to realize a fully compatible enhanced MSX, so users could continue to accumulate software assets built up over time without interruption.

The MSX turbo R is designed to meet the expectations and needs of MSX users by providing superior performance on top of the conventional MSX.

Just by running it on the MSX turbo R as is, you can achieve several times the performance improvement. Also, the knowledge necessary to develop software can be utilized as it is, but by utilizing the young know-how that we introduce in this book, you can pull out even more of the machine's performance and realize a system that boasts overwhelming cost performance.

Systems Business Division 1, Product Integration Department, Integration Department Manager, Ryozo Yamashita

Note:

1. For MSX commercial software, running it on the R800 may result in slower speeds and unstable operations, as there are cases where the Z80 is automatically used for processing and the intended speed may not be achieved.

Table of Contents

1. MSX turbo R 15

1.1 MSX turbo R Hardware 16
1.1.1 These are the features of MSX turbo R! 16
1.1.2 System Configuration of MSX turbo R 16
1.1.3 Switching to elegant CPUs, ... 18
1.1.4 MSX turbo R ROM structure 18
1.1.5 System timer for speed adjustment 19
1.1.6 MSX turbo R I/O ports 20
1.1.7 DRAM mode for speed generation 22
1.1.8 These are the features of R800! 22
1.1.9 All about the R800 23
1.2 Utilizing MSX turbo R: 27
1.2.1 Programming for the speed of R800 27
1.2.2 Precautions and issues when using R800 27
1.2.3 Explanation of the added BIOS functions 28
1.2.4 About the changes and the removed BIOS settings 30
1.2.5 Precautions in application development 32
1.2.6 Example programs for switching CPUs 33

1.3 Utilizing PCM to the very limit ↪ 37
1.3.1 Fundamentals..... How to use PCM in ↪ BASIC..... 37
1.3.2 BASIC commands related to PCM ↪ 38
1.3.3 Let the PCM ring with BEEP sound! ↪ 39
1.3.4 Advanced.....PCM in machine ↪ language..... 41

2. SLOT ..... 47 2.1 What is a slot 48

Contents

2.1.1 How is the CPU connected to memory? .....48 2.1.2 Exploring the inside of the 8-bit CPU Z80 .....48 2.1.3 Various characteristics of memory .....50 2.1.4 What's the slot on the MSX like? .....50 2.1.5 The secret of MSX's expandability was in the slot ...52 2.1.6 How the MSX2+ slot has changed .....53 2.1.7 Let's expand the slot .....55 2.2 Challenge to Slot Switching .....57 2.2.1 To switch slots .....57 2.2.2 Method to specify slot numbers .....58 2.2.3 Functions of BIOS that operate slots .....58 2.2.4 How to know the slot configuration .....60 2.2.5 Exploring the system's firmware .....61 2.2.6 MSX2+ hardware specifications .....64 2.2.7 Device table to prevent conflicts .....65 2.3 MSX Turbo R Slot Mechanism .....67 2.3.1 Finally, the slot mechanism has come this far ...67

3. Kanji BASIC .....71 3.1 Analyzing Kanji BASIC .....72 3.1.1 Hardware required for Kanji BASIC .....72 3.1.2 What is the software compatible with MSX-JE? ....72 3.1.3 Understanding the operating principle of Kanji drivers ..73 3.1.4 JE compatible ROM & software .....75 3.1.5 Various screen modes usable with Kanji BASIC .....76 3.1.6 Kanji text and Kanji graphics .....77 3.1.7 Proper usage of the Kanji driver 78

4. V9958 VDP .....81 4.1 V9958 Register List .....83 4.2 New features of V9958 .....85 4.2.1 Horizontal scroll .....85 4.2.2 Wait .....87 4.2.3 Command .....87 4.2.4 Display in YJK mode .....87 4.3 Ban function of V9958 89

4.4 V9958 Hardware Specifications (Modified Parts) ..... 90 4.5 V9958 and MSX2+ ..... 91 4.5.1 All 12 Types of Screen Modes ..... 91 4.5.2 Controlling the VDP Registers ..... 92 4.5.3 V9958 Registers ..... 94 4.5.4 Horizontal Scrolling by VDP ..... 95 4.5.5 Don't Use Haphazard Techniques Even If They Exist! ..... 98 4.6 Understanding the YJK Method ..... 99 4.6.1 TV Broadcasting and the YJK Method ..... 99 4.6.2 RGB Method's YJK Data Structure ..... 99 4.6.3 Color Display Program ..... 101 4.6.4 Color Calculation Operations You Need to Know About ..... 103 4.6.5 Softening Colors ..... 105 4.6.6 How are SCREEN 10 and 11 Different? ..... 105 4.6.7 Using Fill Patterns in SCREEN 11 ..... 106 4.6.8 The Background for Displaying Characters in SCREEN 12 ..... 108 4.6.9 VDP Registers in the YJK Method ..... 108 4.7 Investigating Interlaced Sync

..... 110 4.7.1 What is the Mechanism for Displaying Interlaced Sync Pictures? .... 110 4.7.2 TV Broadcasting with Interlaced Sync Mode ..... 110 4.7.3 Interlaced Sync Screen on MSX2 ..... 112 4.7.4 Exploring the Cause of Interlaced Sync ..... 113 4.7.5 Demonstrating a Practical Use of Interlaced Sync ..... 116 4.7.6 Practical Program: Let's Do It! ..... 116 4.7.7 VDP Registers Used in Interlaced Sync ..... 117 4.7.8 Writing a Sample Program in BASIC ..... 118 4.7.9 Operational Mechanism of Assembly Parts ..... 121 4.7.10 Machine Language Cheese for Interlaced Sync 128

5. MSX-MUSIC .....129 5.1 What is FM Sound Source? ..... 130 5.1.1 History of Electronic Music to FM Sound Source ..... 130 5.1.2 Analyze Musical Tones ..... 132 5.1.3 What is Operator Averaging? ..... 134 5.1.4 Analyze MSX-MUSIC ..... 135 5.1.5 Challenge Creating a Score Using FM Sound Source 137

5.2 Control FM sound source 139

5.2.1 Try producing sound with a machine language program 139
5.2.2 Explain the outline of the library 141
5.2.3 Let’s compile with MSX-C 149

5.3 Structure of FM sound source data 150

5.3.1 Let’s create FM sound data 150
5.3.2 To specify percussion sound data 152
5.3.3 Let’s specify musical instrument sound data 154
5.3.4 Things you can’t do with the OPLL driver 156
5.3.5 Let’s add tone data .....
↪ 156
5.3.6 Explain sample data .....
↪ 158

5.4 Various FM sound related materials 160

5.4.1 Description of the powerful usage method 160
5.4.2 List of MSX-MUSIC tone data 162

A R800 Instruction Table 165

A.1 Let’s use the instruction table this way 166
A.2 8-bit transfer command .....
↪ 168
A.3 16-bit transfer command .....
↪ 169
A.4 Exchange command .....
↪ 171
A.5 Stack operation command 171
A.6 Block transfer command 172
A.7 Block search command 172
A.8 Multiplication command 172
A.9 Addition command .....
↪ 173
A.10 Subtraction command .....
↪ 175
A.11 Comparison command 176
A.12 Logical calculation command 177
A.13 Bit manipulation command 178

A.14 Rotate command 179
A.15 Shift command 181
A.16 Branch command 182
A.17 Call command 183
A.18 Output command 185
A.19 CPU control command 186

I. Table of Contents

1.1 Configuration of the MSX turbo R system ..... 17 1.2 Changes in ROM configuration on the MSX turbo R ..... 19 1.3 Internal block diagram of the R800 ..... 25 1.4 Differences in memory access methods between Z80 and R800 ... 26

2.1 Z80 CPU memory ..... 49 2.2 MSX slot configuration (Part 1) ..... 51 2.3 MSX slot configuration (Part 2) ..... 52 2.4 Example of MSX2+ slot configuration (Expanding only slot 3) . 54 2.5 Example of MSX2+ slot configuration (Expanding slots other than slot 3) ..... 55 2.6 How to specify the slot number ..... 58 2.7 Device table ..... 60 2.8 Slot configuration of MSX turbo R 68

3.1 Kanji driver operating principles ..... 75 3.2 Switching screen modes 78

4.1 Horizontal scroll (SP2=0 case)..... 85 4.2 Horizontal scroll (SP2=1 case)..... 86 4.3 List of functions of the V9958 control register added with ..... 96 4.4 Mechanism of the two types of horizontal scroll ..... 97 4.5 RGB screen data structure ..... 99 4.6 YJK screen data structure ..... 101 4.7 Mixed screen data structure ..... 101 4.8 How to draw interlaced lines on a TV screen ..... 110 4.9 This is what happens in interlace mode! ..... 112 4.10 The principle of load cutting is 114

Table of Contents

4.11 Procedure for Raster Line Slicing ..... 115 4.12 VDP Register that Generates Raster Line Slicing ..... 116 4.13 VDP Register that Captures Raster Line Slicing ..... 117 4.14 VDP Register that Controls Screen Switching ..... 117 4.15 Mechanism of Hardware Vertical Scroll 118

5.1 Exploring the Structure of Four Types of Electronic Instruments ..... 131 5.2 Analyzing Basic Tones ..... 132 5.3 Envelope of Instruments and Synthesizers ..... 134 5.4 Drum Tone Data ..... 153 5.5 Tone Data ..... 157 5.6 List of OPLL Registers 161

1.1 I/O Map of MSX turbo R 21
1.2 Comparison of Operation Speeds between Z80 and R800 24
1.3 List of Changes in BIOS and BASIC on MSX turbo R 32
1.4 I/O Ports for PCM 45

2.1 System Work Area Related to Slots 61
2.2 I/O Ports of MSX2+ 64

3.1 List of MSX-JE Built-in Hardware 73
3.2 Screen Modes of Kanji BASIC 77
3.3 Hooks used by Kanji Drivers 79

4.1 Screen Modes of VDP and BASIC 82
4.2 Mode Register 83
4.3 Command Register 84
4.4 Status Register 84
4.5 Changes to V9958 Terminals 90
4.6 Characteristics of V9958 90
4.7 Screen Modes of MSX2+ 91
4.8 I/O Ports of VDP 92
4.9 Locations to Save Control Registers 93
4.10 Other Convenient System Work Areas 94
4.11 Detailed Explanation of Added and Modified System Work Areas on MSX2+ 94
4.12 Details of the 0FAFCH Bank (MODE) 104
4.13 Logical Screen Operations 104

5.1 Comparison of Performance of Electronic Instruments 131
5.2 Relationship between Pitch and Frequency 134

5.3 List of Tones that Can Be Set with MSX-Music ..... 135 5.4 Data Structure of 6 Instrument + 1 Drum Sound ..... 150 5.5 Example of Data Structure of 6 Instruments + 1 Drum Sound ... 151 5.6 Data Structure of 9 Instruments ..... 152 5.7 Example of Data of Instruments ..... 155 5.8 Example of Instrument Data ..... 155 5.9 List of Preset Data 163

Chapter 1 MSX turbo R

This chapter is a re-edited compilation of articles from the MSX Magazine, November 1990 issue and December 1990 issue, ”MSX Turbo R: Technical & Analysis” and ”PCM Limit Usage.”

1.1 MSX Turbo R Hardware

With the new 16-bit CPU ”R800” and 256KB of main RAM, and standard support for MSX-DOS2, including hierarchical directories, the MSX Turbo R is a topic of much discussion. An overview of the system configuration of this noteworthy machine is introduced below.

1.1.1 The Features of MSX Turbo R

- By equipping it with the high-speed ”R800” CPU, which is more than five times faster than the Z80, the speed is increased by approximately 4 to 5 times, and at its peak up to 10 times (compared to MSX2+).
- In addition to MSX-DOS1, it is equipped with Japanese MSX-DOS2 kanji drivers and supports environments such as hierarchical directories and DOS compatible disks.
- Equipped with 256KB of main RAM that supports a memory mapper. The structure of the machine has also been standardized.
- Equipped with PCM recording/playback capabilities as standard. Previously, this was optional for MSX-MUSIC, but is now a standard feature.

1.1.2 MSX Turbo R System Configuration

The hardware configuration of the MSX Turbo R (hereinafter referred to as Turbo R) is as shown in Figure 1.1. The conventional MSX equipped with the ”Z80” compatible CPU has been combined with the newly developed ”R800” CPU. Despite industry rumors of “the

next MSX will have a clock twice that of the Z80, or a high-speed CPU compatible with Z80 such as the Hitachi HD64180," there was no clear information on who developed the CPU.

The performance of this new hardware is comparably superior to the recently developed 16-bit CPU V30 (the NEC-developed 16-bit CPU). The performance transformation is also included in ROM, and by keeping the RAM and device capacity optimized, the traditional MSX design philosophy has been carried forward. For now, Turbo R hardware will be evaluated as "aiming to be a practical system," unlike the latest notebook PCs.

1.1 MSX turbo R Hardware

Figure 1.1: MSX turbo R System Configuration

To explain the hardware configuration of turbo R in more simplified terms, disregarding the detailed control signals. The V9958 outputs video signals. The TC9769 (Z80) connects various interfaces to the board. The output of TC9769 is connected to the printer port. The output of MIX is audio signal.

If we go into a little more detail about the hardware configuration, first of all, the "TC9769" in the figure, from the model number, it can be inferred that it is a CMOS-LSI (a type of LSI with low power consumption) from Toshiba, commonly referred to as the "MSX-Engine". It includes a Z80 5th generation CPU, PSG sound, etc. In the following, whenever "Z80" is mentioned in this book, it refers to this LSI.

The "273" below it, is a driver for controlling the printer. "OPLL" is for FM sound, "FDC" is for the Floppy Disk Controller. Additionally, "SRAM" is memory for saving results of kanji learning without turning off the power, but as far as write cycle characteristics are concerned, it has a vendor-specific optional feature to prevent errors. As for R800 and main RAM, the S1990 is the key, and it's involved (not shown in the figure, it's connected to S1990). For example, when using R800's VDP, it temporarily suspends S1990 signals, and if further necessary, temporarily suspends R800's specific signals and operates in sync with Z80's signal timing or performs this operation. The S1990 suspends R800 signals based on the central memory mapping processing.

The turbo R was made with such a complex configuration because it was aimed to maintain compatibility with all conventional hardware and software. It seems that was the case. Additionally, while the number of parts in turbo R is not small, it has a very complicated internal structure. Furthermore, while the S1990 is in a 160-pin flat package, normal assembly is impossible. The fact that it was a special product can also be indicative of the times when hardware crafting during the personal computer's dawn was a craftmanship technique. However, despite this, the MSX cartridge slot remains the same as it always has been, so henceforth MSX will probably continue to be a teaching material for beginners in hardware.

1.1.3 Elegant CPU switching

In Victor's MSX2 machines, the HC-90 and HC-95 could switch between two types of CPUs. However, the turbo R has the LSI "S1990", which was specially developed for this system control. This enables hardware CPU switching to execute programs compiled for different CPUs effortlessly.

With this hardware feature, conventional MSX software that uses the Z80 mode, and turbo R specific software targeting the R800 mode, can execute automatically, allowing seamless switching and automatic execution of applications. Hence, with slight modifications, programs for MSX2 and turbo R/R800 can be developed, extending the scope of software development from traditional Z80 to turbo R/R800.

1.1.4 The overcrowded ROM configuration of MSX turbo R

The turbo R is expected to have many built-in ROMs; however, upon opening it, you'll find fewer ROMs than expected. This is because the S1990 possesses ROM control capability. In MSX2, ROMs are commonly plain main ROMs and sub-ROMs attached to specific slots. However, kanji ROMs attach to I/O ports, preventing a crowded slot composition, ensuring they are conventional ROMs.

Instead of needing 4 ROMs of 32 kilobytes or 1 ROM of 128 kilobytes, simplifying board size, power consumption, and cost, the turbo R integrates few ROMs but achieves efficiency. Below version 1.20, the turbo R houses one 512-kilobyte ROM. This ROM includes the main and sub BIOS, OPLL driver, DOS, and version 1-level primary software and secondary software, overseeing the CPU ROM tasks. Thus, S1990's ROM control effectively manages internal software state.

Since both the primary ROM is connected to D8H and sub ROM connected to D9H slots are visible as single ROM, it simplifies ROM management. Thus, combining primary, DOS (consisting of MSX-DOS2 with 48 kilobytes), and various segments occupies slot 3-2, interchanging between 16-kilobyte space, effectively managing internal storage.

Fig 1.2: ROM Structure Changes in MSX turbo R

[Diagram]

MSX2+ ROM Structure

- Z80

```
- Main: 32 kilobytes
- Sub: 48 kilobytes (Japanese Kanji driver)
- First, second dictionary: 128 kilobytes each
```

- MSX turbo R ROM Structure

```
- R800
- S1990
- Z80
- Main: 512 kilobytes (Japanese Kanji driver, OPLL driver, DOS, first and second
↪  dictionaries)
- Text conversion prediction: 512 kilobytes
```

The connections have been replaced.

1.1.5 System Timer to Adjust Speed

When using R800 and V9958 (an LSI that controls the display) at intervals of 8 microseconds or less, the R800 waits automatically upon receiving an interrupt from the VDP interface circuit built into the S1990. This prevents the V9958 from malfunctioning due to the CPU being too fast. However, other LSIs do not have automatic wait functions, so software self-adjustment timing is required. Many traditional software use:

EX (SP), HL EX (SP), HL

or

PUSH HL POP HL

Though it takes time, there is a method of incorporating complex instruction sequences into the program to obtain accurate timing. However, as explained earlier, the instruction execution time of the R800 is not consistent, so it is impossible to take timings using such a method. Therefore, the turbo R has a ”system timer” to accurately take readings. This is a 16-bit counter whose value increments every 3.911 microseconds. The lower byte is

assigned to I/O port 0E6H, and the upper byte is assigned to I/O port 0E7H. However, if you try to read the full 16-bit value, the count may change midway and the reading will not be reliable, so use only either the lower-byte or the upper-byte value. Below is an example program that waits for (the value of the B register) x 3.911 microseconds. By using the upper-byte value instead of the lower-byte value and writing the program in the same way, you can wait for (the value of the B register) x 1001.2 microseconds.

List 1.1 (TIMER.Z80)

```
.Z80
COUNTLOW  EQU   0E6H   ; Lower 8 bits of the counter
COUNTHIGH EQU   0E7H   ; Upper 8 bits of the counter
; Waits for the value of the B register x 3.911 µs
; Accuracy: x -3.911 µs, +0 µs
; The B register must be initialized
; The C, A, and F registers are destroyed

WAIT: IN A,(COUNTLOW) ; Get the current value of the counter
     LD   C,A              ; Save it
WAIT_LOOP:
     IN   A,(COUNTLOW)     ; Get the current value of the counter
     SUB  C
     CP   B                ; When it reaches the specified value
     JR   C,WAIT_LOOP      ; If not yet reached, loop
     RET
```

1.1.6 I/O Ports of the MSX turbo R

The original turbo R documentation did not include an I/O map, so based on information obtained by disassembling and analyzing the hardware, I created an I/O map referring to the MSX2+ I/O map. The remark ”add” is used for the I/O ports newly added in the turbo R.

First, the ”A/D converter” is an I/O port used to stop a program by a key click while playing back PCM recordings using the BIOS. Another example, ”snooze control,” is an I/O port used to allow stopping a program with the snooze key during disk operation. These will be explained in detail later.

1.1 Hardware of MSX turbo R

Table 1.1: I/O Map of MSX turbo R

Address Range	Usage	Notes
00H~3FH	Custom Hardware	
40H~7BH	Manufacturer Option	
7CH~7DH	OPLL	B
80H~87H	RS-232C	1 B, X
88H~8BH	External VDP	2 XX
90H~93H	Printer	1
98H~9BH	VDP	1 +
A0H~A2H	PSG	1 +
A4H~A5H	A/D Converter	R
A7H	Pause-key control	
A8H~ABH	8255	1 B
ACH~AFH	MSX-Engine	2
B0H~B3H	SONY or SRAM	2 XX
B4H	Clock	2
B5H~B8H	Light Pen	2 XX
B9H	VHD Control	2 XX
B0H~C1H	MSX-Audio	2 XX
C8H~CCH	MSX-Interface	2 XX
D0H~D7H	Floppy Disk 2	XX

Address Range	Usage	Notes
D8H~D9H	3rd Kanji ROM	2
DAH~DBH	4th Kanji ROM	2 XX
DCH	Kanji ROM Expansion	R, X
E3H~E5H	?	?
E6H~E7H	System Timer	?
EAH~FAH	Reset Status	R, B
F5H	Device Name Table 2	2
F6H~F7H	AV Control	X
FCH~FFH	Memory Mapper	2 B

According to the MSX Magazine editorial department's investigation:

1. Compatible with MSX1.
2. Compatible with MSX2.
3. Compatible with MSX2+.

R: Updated in turbo R. B: Operable only with BIOS. +: Application programs may not operate. X: Manufacturer's Option. However, even though it is tentative, I have confirmed that the new functions of turbo R are provisionally implemented. XX: Included in the previous specification, but deleted in the turbo R specification. ?: Registered as a specification, but not described.

”The expansion of the Kanji ROM” refers to a 24-dot Kanji ROM and might include a future JIS No. 3 Kanji ROM, as well as pre-reserved functions. Although these contents are not yet confirmed in the documentation at hand, I conducted an experiment within MSX turbo R’s hardware that makes it accessible from a ROM port. This has been verified from data transferred between the Turbo R’s hardware and the I/O port. Furthermore, ”System Timer” has been recognized as a standard public characteristic.

Though not listed in this table, turbo R’s speed adjustment may affect PSG, joystick, mouse, printer, keyboard, and clock (battery backup).

22 Chapter 1 MSX turbo R

The operation of the clock IC should be performed via the BIOS.

Next, although it is a general function of MSX2+, I would like to give some additional explanation about the ”reset status.” This is used to distinguish, at the point of jumping to the main ROM 0 area, whether the reset was a hardware reset or a software reboot. Specifically, by calling the ITAR address in the main ROM, the reset status value is read into a register, and by calling the ITDR address, the reset status value is written from a register. For example:

CALL ITAR OR 80H CALL ITDR RST 0H

With this procedure, by setting bit 7 of the reset status to 1 and then jumping to the 0 area, the MSX can reboot safely.

So why must we go through the BIOS to handle the reset status? The reason is that there are differences in how the hardware manages the reset status at power-on between machines, and the BIOS is used to absorb these differences. Since the appearance of DOS2 and the turbo R, important components such as the memory mapper, along with the BIOS routines that extend the hardware, have become necessary. Functions and tables that were not used in the past have now become essential. There are also functions that were important in the past but are no longer found in recent MSX machines. Conversely, today's MSX machines include functions that were not available before. The recent MSX has been

overtaken by modern computers, and the writer feels there is less enthusiasm around it, but how about you?

1.1.7 DRAM mode for faster performance

The minimum time required for a memory read or write operation is called the ”access time.” If the CPU runs faster, the CPU must insert wait states to match the memory speed. This access time varies from device to device, and higher-speed memory is more expensive. In general, RAM has a shorter access time than ROM.

When using the R800 for speed, would it not be better to run programs from RAM rather than ROM? Therefore, the contents of the BIOS, BASIC, sub-ROM, and kanji-driver ROM are transferred (copied) to DRAM (main RAM) for use. This is called ”DRAM mode.” In this mode, 64 Kbytes of main RAM is detached from the CPU address space, the ROM contents are written into that RAM, and then it is reconnected to the CPU.

From the CPU's point of view, it appears as if an ordinary ROM has been replaced with a high-speed ROM. When executing a program written in BASIC, the ROM containing the BIOS and BASIC interpreter is in use, so the speed of DRAM mode could not be fully utilized. However, when running a machine-language program, especially a DOS program, the time the ROM is used is relatively short. Therefore, rather than using DRAM mode, it may be more advantageous to use the freed-up memory for a RAM disk or the like.

Also, transferring a ROM-cartridge program to RAM lets it run faster, but with the turbo R, disk-based software will likely become the mainstream even more than before.

1.1.8 R800 Features—This Is It!

- Object-code compatible with the Z80. Therefore Z80 software also runs, except for the parts that depend on CPU timing.
- The CPU clock frequency is 7.16 MHz. However, since the number of clocks per instruction is greatly reduced compared to the Z80, in Z80-equivalent terms it corresponds to 29 MHz (only when there are no wait states).
- Supports a multiply instruction with 16-bit x 16-bit -> 32-bit precision. This makes a major improvement in arithmetic processing speed possible.
- Officially guarantees access to the upper/lower 8 bits of the IX/IY registers, which was undefined on the Z80.

1.1.9 All About the R800

The R800, adopted as the CPU of the turbo R, is a high-speed CPU that is upward software-compatible with the conventional Z80. In other words, software developed for the Z80 can be run as-is at high speed on the R800. As features added to the Z80, it formally provides a 16-bit multiply instruction and instructions for byte access to the IX/IY registers, which were treated as ”tricks” on the Z80. The instruction set of the R800 is given later, so please refer to it.

The conventional MSX clock frequency is 3.58 MHz, while the turbo R clock frequency is 7.16 MHz. However, just looking at this it might seem that the speed has doubled, but that is not actually the case. On the R800 the number of clocks required to execute a single instruction is reduced, and furthermore no M1-cycle wait occurs when accessing RAM, so the program execution speed becomes even faster. To achieve the same processing speed as the R800 on a conventional Z80

To achieve this, the clock frequency is increased to approximately 29 MHz, which results in a significant speed up.

Table 1.2: Comparison of Operating Speeds of Z80 and R800

Command	MSX2+ (units: μs)	Turbo R (units: μs)	Multiplier
LD r,s	1.40	0.14	x10.0
LD r,(HL)	2.23	0.42	x5.3
LD r,(IX+n)	8.87	1.06	x8.4
PUSH qq	6.35	0.56	x11.3
LDIR (BC ≠ 0)	6.35	0.96	x6.6
ADD A,r	1.40	0.14	x10.0
INC r	1.40	0.14	x10.0
ADD HL,ss	3.95	0.14	x24.4
INC ss	3.36	0.14	x24.0
JP	3.07	0.42	x7.3
JR	3.00	0.42	x7.1
DJNZ (B ≠ 0)	3.91	0.42	x9.3
CALL	6.35	0.56	x11.3
RET	3.07	0.56	x5.5
MULTU A,r	—	1.96	—
MULTUW HL,rr	—	5.03	—

So, the comparison of the speed between the Z80 and R800 for various commands is shown in Table 1.2. The fact that register-to-register transfer (LD command) and addition speeds are 10 times faster is noteworthy. However, please note that the values in this table are measured when the R800 operates in waitless mode. There may be times when speed drops due to wait, so be careful. We will explain in detail later how waits occur and how to measure speed under such conditions.

The internal clock frequency of the R800 is about 1.3 times faster. On the R800, the data bus is 8-bit externally, but the internal data bus is 16-bit. Therefore, 16-bit calculation commands can be processed with one cycle.

Looking at the memory configuration, the R800 has a 16-bit CPU, unlike the 8-bit CPU of the Z80, with an external data bus of 8 bits and a 16-bit internal CPU. It appears to be similar to the Intel ”8088” or the Motorola ”MC68008”.

The turbo R also has a memory mapper function, but these functions are not only intended for use with the R800 or MSX. When used with turbo R, the R800 will control a slot control function mounted on the S1990 and an expanded memory mapper system.

Next, we will explain ”DRAM Page Access” in detail. First, this---

1.3: Internal Block Diagram of R800

Clock Generator

MSX Control

Address Extension (Mapper)

Extended Address Bus

Address Bus

Data Bus

Memory I/O Port

CPU

```
The method for memory access using the Z80 is shown in Figure 1.4 below. The upper byte
 ↪  of the address (row address) is sent to the DRAM, the RAS (row address strobe)
 ↪  signal is set to LOW, the lower byte of the address (column address) is sent to the
 ↪  DRAM, and the CAS (column address strobe) signal is set to LOW. This specifies the
 ↪  address of the memory.

On the other hand, the page access of DRAM in the R800 is shown in Figure 1.4 above. The
 ↪  upper byte of the address is fixed with the RAS signal, and the CAS signal of the
 ↪  lower byte of the address is changed to send to the DRAM at twice the speed of the
 ↪  conventional method. In this way, the R800 can automatically perform page access
 ↪  continuously while fixing the upper byte of the address.

Next, the types of DRAM used when connected to the R800 are listed. The varieties of
 ↪  DRAM that can be used are 256 kilobits (32 kilobytes), 1 megabit (128 kilobytes), 4
 ↪  megabits (512 kilobytes), etc. The minimum main RAM capacity is 256 kilobytes.
```

Figure 1.4: Difference in memory access methods between Z80 and R800

Method of memory access with R800: [\text{Memory cycle time} \approx 140\text{nS}]

Method of conventional memory access: [\text{Memory cycle time} \approx 280\text{nS}]

Even with the turbo R, having just two 1-megabit DRAMs is sufficient. The first MSX developed in 1983 used 16-kilobit DRAM chips, totaling 8, yet even that provided ample RAM capacity with 16 kilobytes. Looking back, it's astonishing to think that just two 256-kilobit DRAMs were enough. While the MSX functionality has indeed increased, the size and power consumption of the hardware have decreased. It is noteworthy achievements of the latest Japanese semiconductor technologies that have resulted in the recent emergence of popular notebook computers and the turbo R.

1.2 Effective Use of MSX turbo R

1.2.1 Programming to Maximize the Speed of R800

Certainly, the R800 is fast, but to make the most of its speed, methodical programming is necessary. It should be understood that there are three types of wait states related to accessing external slots, internal ROM, internal DRAM in sequential access, and a wait state that occurs with the second cycle when accessing internal DRAM. Additionally, there is a wait state unrelated to the address for sequential instruction execution or jump instructions and their destinations.

For efficient use, it's advantageous to have the program stored in internal RAM, which overlaps with 256-byte boundaries. For example, if the program is stored within the internal RAM's address range (within the paging unit), and if the larger instruction fetch occurs and the CPU reads necessary dataed, it results in a single memory access cycle. This avoids additional wait states necessary for external ROM access. Optimally programming with such considerations ensures the CPU operates at maximum speed, such as:

PUSH HL

The execution time of an instruction depends on whether the opcode placement in memory and the positioning of the stack pointer are aligned with the upper byte of internal RAM. If aligned, it takes four clocks; if not, five clocks, though adjustments may be required for optimized programming. Situational discrepancies in instruction execution time are important, so bear them in mind.

1.2.2 Precautions and Issues in Using R800

For the Z80, a DRAM refresh occurs each time an instruction is executed. In the R800, DRAM refresh occurs every 31 microseconds and within 280 cycles per second, necessi-

tating attention to the time required for these refresh cycles. As previously explained, due to DRAM access conditions, it is difficult to consistently measure the program execution timing for the R800.

Therefore, to adjust program speed, use something like a ”system timer.” Additional explanations on using this system timer, as well as adjusting speed related to the CPU and VDP, will be explained, so please wait.

Also, although this explanation is about programming and problems, consider that inadequate development materials make developing software challenging.

Note that some context or specific technical terms might need further clarification by someone with specialized knowledge in this field.

It is inconvenient that powerful ”ICE (in-circuit emulator)” cannot be used for debugging.

Therefore, in order to create software for turbo R, it is first necessary to thoroughly debug the software for conventional MSX and Z80 using ICE, verify that the program operates correctly, and then convert it to turbo R. A straightforward approach might be to first create a program for Z80, verify its operation, and then convert the result to a program for R800. Alternatively, it's possible to first verify the overall operation with an assembler, then write the software to check the operation. Finally, compile the entire program and verify its operation... One might consider looking at the source list.

1.2.3 Explanation of Added BIOS and Functions

To control the new hardware features of turbo R, a BIOS for switching the CPU and replaying PCM recording has been added.

Here, the description order for BIOS names (labels), entry addresses, and registers used for each function are explained. The notation to express BIOS functions in this document is outlined below. E denotes the register where the current value to be called from BIOS is stored, R refers to the register where the BIOS returns the value, and M is the register where BIOS writes arbitrary values, meaning original contents may change. Furthermore, IYH refers to the higher byte of the IY register, and the content below indicates relevant notation.

CHGCPU 0180H address Function: Switch the CPU. E register bits b1 and b0 set the mode as follows. Mode “R800 DRAM” means transferring the BIOS content from ROM to DRAM and using that mode.

b7 b6 b5 b4 b3 b2 b1 b0 L 0 0 0 0 0 0 M

Mode

```
00  Z80
01  R800 ROM
10  R800 DRAM
```

LED: Always write 0.

1.2 How to Utilize MSX turbo R 25

Also, if bit 7 of register A is set to 1, the LED that shows which CPU is running changes. Conversely, if bit 7 of register A is 0, the CPU is switched but the LED does not change.

Input: AF Output: none

Note: The contents of the registers before the CPU switch, other than AF and A, are carried over to the newly switched CPU. It is also permitted to switch the CPU during an in-

struction sequence while preserving these registers. Detailed precautions regarding CPU switching are explained later.

GETCPU 0183H address Function: Checks the operating CPU. Input: none Output: A

Depending on the operating CPU, the following value is returned in register A.

A	CPU
0	Z80
1	R800 ROM
2	R800 DRAM

Note: Before calling this BIOS, it is necessary to confirm that the hardware is a turbo R as described later.

PCMPLY 0186H address Function: Plays PCM sound. Input: A

b7 b6 b5 b4 b3 b2 b1 b0 R 0 0 0 0 0 0 F F

Frequency value (Write 0 if none) -VRAM / MRAM

EHL (Data address) DBC (Data length)

30 Chapter 1 MSX turbo R

If bit 7 of the A register is set to 1, data is placed in video RAM, and if set to 0, data is placed in main RAM or PCM sound-source data. Note that the values of the D and E registers are meaningful only when the data is in video RAM.

Bits 1 and 0 of the A register are used to set the sampling frequency. However, 15.75 kHz can be specified only when the turbo R is operating in R800 DRAM mode.

```
00   15.75 kHz
01    7.875 kHz
10    5.25 kHz
11    3.9375 kHz
```

Output: Carry flag

```
0  Normal completion
1  Abnormal completion

   A (Reason for abnormality)
       1  Frequency specification error
       2  Interrupted by STOP key
```

EHL (Interrupt base) Note: all

PCMREC 0189H address Function: Records PCM sound. Input: A

b7 b6 b5 b4 b3 b2 b1 b0 R T T T T C F F

Frequency Compression Trigger level VRAM / MRAM

EHL (Data address) DBC (Data length)

The settings of bits 7, 1, and 0 of the A register are the same as those explained for PCMPLY. Bits 6 to 3 of the A register specify the ”trigger level.”

1.2 Utilizing MSX turbo R

Specifies the sound level that triggers the start of recording. If this value is 0, recording will start immediately.

Also, if bit 2 of the register is set to 1, the recording data will be compressed. If it is 0, it will not be compressed.

R

- Carrier Flag

```
- 0 Normal Termination
- 1 Abnormal Termination
```

A (Reason for Abnormal Termination)

```
- 1 Interruption by peripheral detection
- 2 Interruption by STOP key
```

EHL (Interrupt Habitat)

M

- All

1.2.4 Regarding changed or removed BIOS

Regarding the BIOs that have been changed or removed with the turbo R, as shown in Table 1.3, they will be explained briefly.

First, with the turbo R, the cassette tape interfaces are no longer present. Thus, the conventional BIOS, "TAPION", "TAPIN", "TAPIOFF", "TAPOON", "TAPOUT", and "TAPOFF" are now just set carrier flags and return errors when called. Also, "STMOTR" does nothing when called and returns.

Additionally, as a new feature, to change the size of the main ROM, the new BIOS "GTPDLD" is added for battle simulation games. When called, they read register A, clear A as it always read, and return to standby. Similarly, "GTPADB" or "NEWPAD" will read the value 8-11 from the register A and return to standby.

For altered BIOs, in order to know the version of the MSX used, the contents of the main ROM at address 002DH is accessed, which should be changed to 03H. When developing software for Turbo R, it is important to first ensure that the contents at 002DH are 03H or higher and then program accordingly. Otherwise, MSX2 software should display an alert and stop to prevent errors.

The meaning of the revision in 002DH as 03H is significant as it guarantees future software compatibility for the MSX. Consider it necessary to ensure it is functional at 03H or higher. Generally, it's important to consider the version of hardware and OS.

Table 1.3: List of BIOS and BASIC changes in MSX turbo R

Added BIOS Entries

- CHGCPU 0180H
- GETCPU 0183H
- PCMPLY 0186H
- PCMREC 0189H

Modified BIOS Entry

- ROM version ID 002DH

Deleted BIOS Entries

- GTPDL 00EDH
- TAPION 00E1H

- TAPIN 00E4H
- TAPIOF 00E7H
- TAPOON 00EAH
- TAPOUT 00EDH
- TAPOOF 00FOH
- STMOTR 00F3H
- GTPAD 00DBH
- NEWPAD SUB 01ADH

Added Statements

- CALL PCMREC
- CALL PCMPLAY
- CALL PAUSE

Modified Statement

- COPY

Deleted Statements

- CLOAD
- CSAVE
- MOTOR

Now, once you obtain the values above the version numbers you need, let's try creating the software so that it operates.

Although this is a bit extra and not truly necessary, the MSX version check is implemented so that MSX2+ exclusive programs and educational feature-included MSX-JE combined applications, etc., do not run. To avoid that situation, remember to ”Operate at 03H and above.”

Furthermore, similar to BIOS, BASIC functionalities in turbo R have also been added, modified, or deleted. For that, refer to Table 1.3 and the BASIC manual attached to the machine.

1.2.5 Notes on Application Development

MSX turbo R does not always operate solely on R800 exclusive hardware. When accessing a peripheral slot, there's a wait, and accessing the ROM or internal DRAM is accompanied by paging waits. Therefore, to reduce these waits as much as possible, the program must be created with utmost care. In terms of precautions, please note them succinctly.

1.2 Utilizing MSX turbo R

First of all, transferring the program itself to RAM and then executing it. Software provided by floppy disks will definitely operate in RAM, so there are no issues here. However, attention must be paid to programs supplied in ROM cartridges. By transferring the necessary parts to RAM and then executing from there, considerably high-speed execution becomes possible.

In R800, coding should be done to avoid causing page breaks, as it has a specialized page access feature in DRAM. Make full use of this feature. Specifically, programming should avoid changes in the top 8 bits of the address so that access is within the continuous memory range of ??00H-??FFH, or 256 bytes. It's effective to program in such a way.

Incidentally, avoiding page breaks means avoiding memory access that exceeds these boundaries, meaning avoiding changes in the top 8 bits of the address during execution.

As mentioned earlier, even though turbo R is different from MSX2 and others, during the loading phase of the program, it doesn't significantly impact the execution time of commands. This is because, even in Z80, commands are not executed during DRAM refresh cycles.

Furthermore, to create programs that operate on both the turbo R and MSX2+ while ensuring timing through software loops, it's recommended to use turbo R's newly included system timer, which counts up every 3.911 microseconds. From now on, let's use this system timer for timing.

1.2.6 Examples of Programs That Switch the CPU

List 1.2 shows the source list of "CHGCPU.COM" that switches the CPU. When DOS2 is operating on the turbo R,

CHGCPU 0

Switches to Z80 mode,

CHGCPU 1

Switches to R800 in ROM mode,

CHGCPU 2

The R800's DRAM mode or ROM mode can be selected respectively. When explaining the contents of the program, the program checks the contents of memory area 5DH (technically correct, the default PCB area) in the DOS work area, and sets (or reads) values to the A register according to the first letter of the command.

The BIOS at main ROM address 180H has a function called "CHGCPU".

Also, to verify the practicality of the program, the DOS version number is checked and processed. Specifically, if the contents of the 2DH address at the DOS work area's main-ROM is more than 03H, it means it's turbo R.

It confirms that the machine is turbo R, uses DOS system call 6FH, and ensures that the DOS kernel version number is two or higher.

List 1.2 (CHGCPU.Z80)

Z80

```
RDSLT EQU 0000CH ; inter slot read
CALLST EQU 00001H ; inter slot call
EXPTBL EQU 0FC1H ; slot # of main ROM

    ld a,(EXPTBL) ; address to read
    ld hl,2dh ; read version
    call RDSLT
    jr nc,TURBOR
    de,MSG_NOTR ;_STROUT
    ld c,9
    call 5 ; return to DOS
    rst

TURBOR:
    ld c,6fh ;_DOSVER
    call 5 ; version of DOS kernel
    ld a,b
    cp 2
    jr c,NOTDOS2
    ld c,7f ; version of MSXDOS.SYS
    call 5
```

```
NOTDOS2:
    ld de,MSG_NOTDOS2 ;_STROUT
    ld c,9
```


call 5 rst 0 ; return to DOS

```
;
MSG_NOTR:
  DB    'not MSX turbo R', 0Dh, 0Ah, '$'
MSG_NOTDOS2:
  DB    'not MSX-DOS 2', 0Dh, 0Ah, '$'
END
```

Similarly, in the next List 1.3, it's a source list of the ”GAMEBOOT.COM” program which causes an MSX2 program to run in R800 mode. This program ensures that it runs in a state where DOS2 is started and R800 is selected. In other words, it ensures that the program runs in an environment where DOS2 system is started (games, etc.), even if it is not a program on the disk.

First, the program is disassembled and a message is displayed, indicating that the disk is exchanged. Next, the boot sector of the replaced disk is read and executed. The environment at that time is the same as when boot sectors are copied twice in the usual way, and the first page contains DOS in ROM, while the other page contains RAM, with the character program being set to RAM.

Additionally, to save the pointer of the error handling program and DOS working area (”3238H area”) in the HL register, the program switches pages from RAM to DOS in ROM, and sets the DE register to the location of the program (”3688H area”).

List 1.3 (GAME.BOOT.Z80)

```
.z80
_conin      equ   01h
_strout     equ   09h
_setdta     equ   1ah
_rdabs      equ   2fh

dos         equ   0005h
enaslt      equ   0024h

notfirst    equ   0f340h
master      equ   0f348h

             ld    sp, (6)
             ld    de, prompt            ; print prompt message
             ld    c, _strout
             call  dos
             ld    c, _conin             ; wait for key in
             call  dos
             ld    de, 0c000h            ; read boot sector at 0c000h
             ld    c, _setdta
             call  dos
             ld    de, 0                 ; logical sector 0
             ld    l, 0                  ; drive A:
             ld    h, 1                  ; read 1 sector
             ld    c, _rdabs
             call  dos
             ld    h, 40h
             ld    a, (master)
```

```
call  enaslt
ld    hl, 0f323h
ld    de, 0f368h
xor   a
ld    (notfirst), a
scf
jp    0c01eh
```

prompt: db 'Insert game disk in drive A:', 0dh, 0ah db 'and press any key $'

end

1.3 Making Full Use of PCM to the Limit

The new feature added to turbo R is PCM. As a feature that Sensei was particular about applying fully, it is Mr. Koji's intention to use this feature to the limit. Here, we introduce how to utilize PCM from BASIC to machine language, making full use of its unique qualities, including watermarked sound-data copying.

1.3.1 Basics - How to Use with BASIC

First, let's introduce the basic method using BASIC, to start with. PCM converts and memorizes the sound input from a microphone, etc., into digital, then replay it as desired. In the case of turbo R, PCM data is stored in the main RAM and video RAM. The sampling rate can be selected from four types: 15.75 kHz, 7.875 kHz, 5.25 kHz, and 3.9375 kHz. The higher this value, the higher the quality of sampling.

When using PCM with BASIC, it's sufficient to remember just one command. However, if you don't memorize this command, it will be difficult to use. So, for starters, paying great attention to where in the PCM range the data is written, and its start address and end address settings are necessary.

Firstly, you need to secure the memory area for PCM data using the "CLEAR" command in BASIC; otherwise, it will cause an incorrect operation. For example, when utilizing the addresses C000H - D000H for PCM data, it looks like this:

```
CLEAR 200, &HC000
```

In short, let's try to load the sample program from List 1.4 and play around with it after typing it in.

Of course, if using video RAM for PCM data, you can store in any desired address, without worrying about the start and end addresses. In this case, checking PCM data on the screen becomes possible. Initially, by:

```
SCREEN 8
```

setting the screen mode and then recording PCM sound, the data will be displayed on the screen, and it should be easy to follow.

The basic method for PCM recording and reproduction is as described. Also, adjusting the reproduction sampling rate allows for reproduction at various speeds. However, the problem is, when replaying PCM, the processing load on turbo R is quite intense, so adjustments are needed.

It becomes difficult, so unfortunately, you cannot do anything else while playing PCM.

List 1.4 (PCM1.BAS)

```
10 CLEAR100,&H9000
20 PRINT "Micropower Login System.";
30 A$=INPUTS(1):PRINT
40 _PCMREC (@&H9000,&HCFFF,0)
50 PRINT "#Y Key System.";
60 A$=INPUTS(1):PRINT
70 _PCMPLAY (@&H9000,&HCFFF,0)
80 GOTO 20
```

1.3.2 BASIC Commands Related to PCM

CALL PCMREC

Syntax

- Recording to main RAM or video RAM:

  CALL PCMREC(@start_address, end_address, sample_rate [, [trigger_level, compression_switch] [, S]])

- Recording to an array:

  CALL PCMREC(array_name, [length], sample_rate [, [trigger_level, compression_switch] [, S]])

Specification	Sample Rate Setting
0	15.7500KHz
1	7.8750KHz
2	5.2500KHz
3	3.9375KHz

Trigger Level: When the recording level exceeds the specified level at the start of recording. The value (between 0-127) starts the recording if the input level exceeds it. If not specified, recording starts immediately. The compression switch is for compressing PCM voice data by 1/2 if set to 1. Setting it to 0 or leaving it blank will record without compression.

CALL PCMPLAY

Syntax

- Playing from main RAM or video RAM:

  CALL PCMPLAY(@start_address, end_address, sample_rate [, S])

- Playing from an array:

  CALL PCMSLAY(array_name, [length], sample_rate)

1.3 Clever Uses of PCM

When using PCMREC and PCMPLAY, if they are not in high-speed mode, they will briefly switch to high-speed mode before running, and return to the previous state after finishing. Also, in R800 or ROM mode at 15.75KHz, an error will occur if selected.

When stopping or pausing during recording or playback, the [STOP] key should be pressed. If pressed, the program execution will be interrupted, and the PCM data format is such that values range from 0 to 255 as normal data, with specific values that output 1 byte of continuous repetition and a level (127).

1.3.3 Make the BEEP Sound with PCM!

When running BASIC programs, pressing the [CTRL] and [STOP] keys simultaneously to interrupt the program will produce a ”beep” sound. This is known. By displaying the ”LIST” command, and pressing [CTRL] and [STOP] again, you hear the ”beep” sound once more. In BASIC, you can also use the ”SET BEEP” command to change the sound, but there are subtle tricks to making the BEEP sound with PCM, and that can make the sound quite large and fun.

Indeed, if you execute the program listed in the appendix of ”1.5,” you will be able to make the BEEP sound with PCM. However, this requires a turbo R machine. It is a long program, so please make an effort to input it diligently.

Please note that this program should be placed in the main RAM area (from 4000H to 60FFH). After running the program, use the ”CLEAR” command to set the user area above address B000H. However, don't use it for memory disk-related matters. Therefore, don't just thoughtlessly execute ”CALL MEMINIT” and so forth. So when you use the ”BEEP” command in BASIC:

1. PRINT CHR$(7)

Use it instead of that command. Be aware not to let the BEEP sound become PCM.

Here’s the usage of the program:

1. PCM BEEP Set From then on, the BEEP sound will become PCM. When executed once, the setting will be valid until the power is turned off or the CLEAR command settings are changed.
2. PCM BEEP Reset Restore the BEEP sound to its original state. Execute this command when switching to DOS or DOS2 with ”CALL SYSTEM.”

40 Chapter 1 MSX turbo R

3 PCM Data Playback Plays back the currently set PCM data. Use it for confirmation.

4 PCM Data Recording Records PCM data at 15.75 kHz. Recording uses the memory area from B000H to CFFFH.

5 PCM Data LOAD Loads PCM data that was saved in BSAVE format with the extension ”.PCM”.

6 PCM Data SAVE Saves the PCM data recorded with ”PCM Data Recording” to disk.

0 END Ends the program. Of course, you can also stop it with [CTRL] + [STOP].

In addition, simple messages are displayed on screen, so refer to them.

List 1.5 (PCM2.BAS)

```
10 SCREEN0:WIDTH40:DEFINT A-Z
20 CLEAR100,&HD800
30 DEFUSR=&HD800:DEFUSR1=&HD806:DEFUSR2=&HD803
40 FOR I=&HD800 TO &HD87F
50 READ A$:POKE I,VAL("&H"+A$):NEXT
100 PRINT
110 PRINT"1) PCM BEEP Set "
120 PRINT"2) PCM BEEP Reset "
130 PRINT"3) PCM DATA Playback"
140 PRINT"4) PCM DATA Record"
150 PRINT"5) PCM DATA LOAD"
160 PRINT"6) PCM DATA SAVE"
170 PRINT"0) END"
180 PRINT" ...HIT 0-6 KEY=";
190 A$=INPUT$(1):I=ASC(A$)-ASC("0")+1
```

```
200 IF I>0 AND I<8 THEN ELSE190
210 ON I GOTO 230,240,310,220,340,390,430
220 PRINTCHR$(7):GOTO 190
230 GOSUB 470:END
240 GOSUB 470:I=USR1(0):I=USR(0)
250 PRINT"You can use PCM BEEP."
260 PRINT"When using DOS or DOS2, always reset PCM BEEP again."
270 PRINT"PCM BEEP data is in page 1 (4100H to 60FFH)."
280 PRINT"Setting B000H or above with the CLEAR command is OK, but memory-disk commands such
 ↪  as CALL MEMINI cannot be used."
```

1.3 PCM utilization method to the limit

290 PRINT ”Na”, BEEP, BEEP, and press return”;PRINT CHR$(7) 300 END 310 GOSUB 470:POKE &HFD4,&HC9 320 PRINT ”PCM BEEP Reset device.” 330 GOTO 100 340 GOSUB 470 350 PRINT ”PCM Recording Start. (HIT ANY KEY):”; 360 A$=INPUT$(1):PRINT:PCMREC(&HB000,& 370 PRINT ”PCM Recording End.” 380 GOTO 100 390 GOSUB 470 400 PRINT ”PCM Data LOAD” 410 INPUT ”FILE NAME(8 characters):”;A$ 420 BLOAD A$+”.PCM”,R=&HB000:GOTO 100 430 GOSUB 470 440 PRINT ”PCM Data SAVE” 450 INPUT ”FILE NAME(8 characters):”;A$ 460 I=USR2(0):BSAVE A$+”.PCM”,&HB000,&HCFFF:GOTO 100 470 PRINT CHR$(I+47):PRINT:PRINT:RETURN 480 DATA C3,40,D8,C3,5F,D8,CB,6D 490 DATA D8,3A,42,F3,32,24,D8,21 500 DATA DE,18,00,40,00,00,46,ED 510 DATA ED,B0,C1,76,B1,05,2F 520 DATA 14E1,DC,00,1D,ED,B0 530 DATA C9,F7,F0,00,CA,09,4E,01 540 DATA C0,01,00,20,21,00,41,3E 550 DATA 0A,CD,F8,F3,3A,B4,D4,E5 560 DATA 3B,FA,7E,D3,A4,23,DB,79 570 DATA 20,7F,FB,C9,C9,6D,D,7B 580 DATA 21,00,B9,11,00,41,C9 590 DATA 20,E6,ED,CD,76,B8,C9 600 DATA 6D,E8,D1,20,11,40,41,00,B0 610 DATA C9,00,18,4E,31,23,00 620 DATA 21,00,40,C3,24,00,3A,C1 630 DATA FC,21,40,03,C3,24,00

1.3.4 Advanced version: PCM with machine language!

When using PCM with machine language, the quickest method is to use BIOS. The settings for sampling rate and trigger level are the same as in BASIC, so no problem there.

Since this is the advanced version, we will introduce a program that can record PCM and play it back using BIOS.

When using BIOS, you can choose between four types of sampling rates: 15.75 kHz, 7.875 kHz, 5.25 kHz, and 3.9375 kHz. One type changes every 63.5 microseconds. This counter value is changed every 3.911 microseconds so the system cannot adjust the counter, and the counter value changes every 3.911 microseconds.

It is a program introduced here that replaces the timer.

First, let's explain how to use the recording program. HL register contains the starting address of the PCM data to be recorded, BC register contains the size of the data to be recorded, and E register contains the wait count that decides how many system timer ticks to include. Setting this to 16 corresponds to a frequency of 15.75 kHz.

The playback program works similarly. HL register contains the starting address of the PCM data for playback, BC register contains the size of the data, and E register contains the wait count.

There is a curious command within the PCM recording program list:

0EDH,70H

This is:

IN (HL),(C)

It's an instruction, and it's an R800 specific command to read a value from port C register and reflect it in the flag.

The principle of the program is as such, try using it. You'll be able to enjoy the changes in sound by altering the value of the E register.

List 1.6 (PCMREC.MAC)

```
PMDAC EQU 0A4H
PMCNTL EQU 0A4H
PMCNTL EQU 0A5H
PMSTAT EQU 0A5H
SYSTML EQU 0E6H ; system timer port

REC:
    LD A,00001100B
    OUT (PMCNTL),A ; A/D MODE
    DI
    XOR A
    OUT (SYSTML),A ; reset timer

REC1:
    IN A, (SYSTML)
    CP E
    JR C,REC1 ; wait
    XOR A
    OUT (SYSTML),A ; reset timer

    PUSH BC
```

”1.3 Efficient Use of PCM Interface Methods”

DEFB 0EDH,70H JP M,RECAD7 AND 11111110B

RECAD7: OR 00000000B

LD (HL),A LD A,00001100B OUT (PMCNTL),A

POP BC INC HL DEC BC LD A,C OR B JR NZ,REC1 LD A,0000011B OUT (PMCNTL),A ; D/A MODE EI RET

END

List 1.7 (PCMPLAY.MAC)

```
PMDAC EQU 0A4H
PMCNT EQU 0A4H
PMCNTL EQU 0A5H
PMSTAT EQU 0A5H
SYSTML EQU 00E6H     ; system timer port

PLAY:
```

LD A,0000011B OUT (PMCNTL),A ; D/A MODE DI XOR A OUT (SYSTML),A ; reset timer

PLAY1: IN A,(SYSTML) CP E JR C,PLAY1 ; wait XOR A OUT (SYSTML),A ; reset timer

LD A,(HL) OUT (PMDAC),A ; play 1 byte INC HL DEC BC LD A,C OR B ; end of data? JR NZ,PLAY1 ; next data

1.3 Making the Most of PCM (to its limits) — p.45

```
    EI
    RET

    END
```

Table 1.4: I/O Ports for PCM

Address	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0A5H Write	0	0	0	SMPL	SEL	FILT	MUTE	ADDA
0A5H Read	COMP	0	0	SMPL	SEL	FILT	MUTE	BUFF
0A4H Write	DA7	DA6	DA5	DA4	DA3	DA2	DA1	DA0
0A4H Read	0	0	0	0	0	0	CT1	CT0

- ADDA (BUFF): Buffer mode. Specifies the D/A converter output. For D/A use, set to 0 (double buffer); for A/D use, set to 1 (single buffer). Note that on reset it is in the double-buffer state.
- MUTE: Muting control. Turns the entire system's audio output on or off. 0: audio output off (reset state). 1: audio output on.
- FILT: Selection of the sample-and-hold circuit input signal. During A/D, selects whether the signal fed into the sample-and-hold circuit is the filter's output signal or the reference signal. 0 = reference signal, 1 = filter output signal. Reset value is 0.
- SEL: Selection of the filter input signal. Selects whether the signal fed into the low-pass filter is the D/A converter output signal or the mic-amp output signal. 0 = D/A converter output signal, 1 = mic-amp output signal.
- SMPL: Sample-and-hold signal. Selects whether to sample or hold the input signal. 0: sample (reset state). 1: hold.
- COMP: Comparator output signal. Compares the sample-and-hold output signal with the D/A converter output signal. 0: D/A output > sample-and-hold output. 1: D/A output < sample-and-hold output.
- DA7~DA0: D/A output data. When playing back PCM data, outputting the prepared data here reproduces the PCM sound. The data format is absolute binary, with 127 corresponding to the 0 level.
- CT1, CT0: Counter data. Counts up every 63.5 microseconds. During D/A it synchronizes with the count-up, and the data written at address 0A4H is output repeatedly. Writing data to 0A4H also clears the counter.

Table 1.1: I/O ports for PCM

Address	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0A5H Write	0	0	0	SMPL	SEL	FILT	MUTE	ADDA
0A5H Read	COMP	0	0	SMPL	SEL	FILT	MUTE	BUFF
0A4H Write	DA7	DA6	DA5	DA4	DA3	DA2	DA1	DA0
0A4H Read	0	0	0	0	0	0	CT1	CT0

```
; play 1 note
; end of data
; wave data
```

Chapter 2 SLOT

Chapter 2 SLOT

This chapter is a re-edition of articles ”MSX Magazine February 1989 issue,” ”MSX2+ Technical Exploration” from the March 1989 issue, and ”Technical Analysis” from the November 1990 issue.

2.1 What is a Slot

The slot for setting the cartridge in the MSX is called a ”cartridge slot,” but the slot is also the function for managing the memory of the MSX. In this chapter, we will explain the important and yet difficult concept of slots in an easier way.

2.1.1 How are CPU and memory connected

One of the most important components that make up a computer is the “CPU” and memory. CPU is an abbreviation for ”Central Processing Unit”, a device that manages the whole computer and performs calculations. On the other hand, memory refers to a device that temporarily stores information that the CPU handles.

Information handled by computers is known to be represented by combinations of 0s and 1s, called ”binary.” A binary digit is called a bit. Also, within a program listing, as writing binary as is can be cumbersome, 4-bit binary numbers are used as abbreviated hexadecimal which are represented by characters 0-9 and A-F called ”hexadecimal.”

This is not to be confused, but in the computer world, the unit “kilo (K)” is not 1000 times, but 1024 times. For instance, 64 kilobytes of memory is 64 x 1024 = 65536 bytes of memory. Adding, 65536 x 8 equals 524288 bits.

To manage such memory, many microcomputers have separate numbers for each byte in memory. That is called the “number” or “address.” For example, in a machine language program, it is written as ”execution start address 8000H.”

2.1.2 Exploring the internals of 8-bit CPU Z80

As mentioned in 2.1, CPU and memory are connected by an ”address bus” and a ”data bus”. The address bus is a signal that specifies the memory address that the CPU wants to read or write, and the CPU sends it to the memory. The data bus is a signal used to send the contents of the memory. It should be noted that the address bus from CPU to memory is unidirectional, whereas the data bus is bidirectional.

2.1 What is a Slot?

Figure 2.1: Memory of a Z80 CPU

(Z80 CPU Image Diagram)

- Address bus: 16 bits
- Data bus: 8 bits
- Memory 64KB
    - 16 bits address (0000H to FFFFH)

The Z80 CPU used in MSX can basically access 64 kilobytes of memory. The address bus is 16 bits, which allows it to address each byte in the memory range from 0000H to FFFFH. The data bus, which retrieves data, is 8 bits.

The CPU in MSXs before the turbo R, the ”Z80”, has an 8-bit data bus (physically, there are 8 electrical lines) and a 16-bit address bus. This allows it to read up to 64 kilobytes of memory one byte at a time. This type of CPU is called a ”8-bit CPU.” A computer with an 8-bit CPU is called an ”8-bit computer.” Therefore, MSX is referred to as an 8-bit computer.

To explain the address in more detail, the memory address that can be specified by the 16-bit address bus is binary

0000000000000000B

to

1111111111111111B

(In binary notation, ”B” is used to denote binary numbers)

This covers address ranges from 0 to 65535 in decimal, which means it can address from 0000H to FFFFH in hexadecimal notation (H denotes hexadecimal). In decimal, it ranges from 0 to 255. In addition, this memory in bytes is 64 kilobytes, as it is multiplied by 1024 bytes (1 kilobyte).

Previously, we talked about MSX being an 8-bit computer. Recently, 16-bit computers (computers with a 16-bit CPU) have become mainstream. In these cases, the data bus is 16 bits, so they can read twice the information at once compared to 8-bit computers. However, the memory address is still 16 bits, so they have the same 64-kilobyte memory connection capacity. Combining the speed of memory reading with the performance advantages means that this is the current state of affairs. Furthermore, many large computers have 32-bit or 64-bit data buses and address buses.

The MSX turbo R has the 16-bit CPU R800. To allow it to connect to the conventional cartridges, it still has an 8-bit data bus.

2.1.3 Various Types of Memory and Their Functions

There are many types of memory. First, they are categorized by the type or product. ”ROM” and ”RAM.” ROM (Read Only Memory) is memory whose contents cannot be altered and remains preserved even when power is off. MSX defines different types of ROM, including software like BASIC, and all the software provided on cartridges is written in ROM. For instance, the specialized ROM for Japanese characters included in MSX2 is another example of this type of ROM.

RAM (Random Access Memory) is memory that allows freely writing and reading its contents. However, its contents are erased when power is cut off. RAM is used to store intermediate calculations during programming or for manipulations in disk operations. For example, in a game where cheat commands are input, they are stored in RAM.

Additionally, ”SRAM” is a type of RAM that retains contents as long as power is supplied. It is used in things like versatile format jump and battery backup cartridge for MSX.

Besides this, memory can also be categorized according to its method of use. As shown in Figure 2.1, there's ”main memory” connected to the CPU - in other words, memory other than the main ROM and 64-kilobyte main RAM in the MSX framework.

MSX also includes ”video RAM (VRAM),” which is used to store images and text for television displays. Depending on the computer's specifications, this video RAM may be directly connected to the CPU, but in MSX it's linked to the video display processor (VDP). The details are left to the VDP, which then links with CPU and video RAM.

Though this section discusses MSX specifically, these basic concepts apply broadly to all computers. For more detailed explanations, the BASIC introductory book that comes with the MSX might be useful.

2.1.4 What are the Slots in MSX?

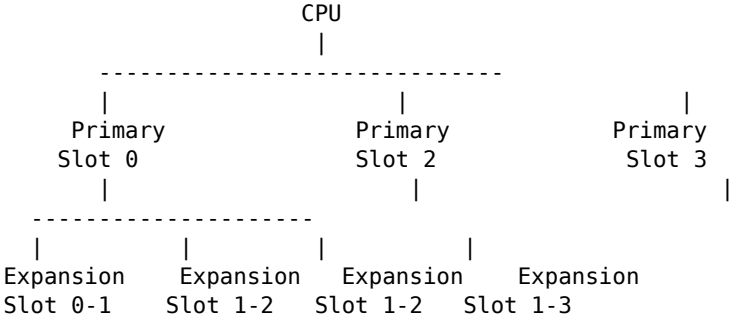
As mentioned earlier, the 8-bit CPU can handle up to 64 KB of main memory. Since that wasn't enough, the initial 8-bit computers used various methods to handle memory ex-

pansion. Nowadays, technology allows for more than 64 KB of memory expansion, and, as explained in section 2.2, MSX allows switching between different 64 KB memory slots, making it possible to use up to 256 KB. Eventually, a system was created allowing the configuration and switching of 4 slots, permitting the maximum use of 256 KB.

2.1 What are Slots?

This makes it possible to handle more memory. This memory is called the ”primary slot” and is assigned to the cartridge slots provided in the machine.

Fig. 2.2: MSX Slot Configuration (Part 1)



The MSX is designed to handle memory beyond 64 kilobytes by using a method called slot switching. This method involves dividing the memory into four 64-kilobyte segments and switching between them, allowing a maximum of 256 kilobytes of memory to be managed. Furthermore, each of the four primary slots can be expanded to include additional ”expansion slots,” such as those in Fig. 2.2, enabling even more memory to be connected.

Additionally, instead of one primary slot, you can use four groups of ”expansion slots” to handle memory beyond 64 kilobytes. This means that a total of 16 groups can be connected, allowing for a maximum of 1 megabyte (1024 kilobytes) of memory. However, it's not possible to further expand these expansion slots.

Now, while you can handle memory beyond 64 kilobytes by switching slots, it is actually difficult to switch the entire memory simultaneously. Therefore, the MSX divides the memory into units called ”pages” for management.

The 16 kilobytes of memory from 0000H to 3FFFH are managed as ”page 0,” the 16 kilobytes from 4000H to 7FFFH as ”page 1,” and the 16 kilobytes from 8000H to BFFFH as ”page 2.” Finally, the 16 kilobytes from C000H to FFFFH are page 3. You can select a slot for each page. For example, when processing BASIC disk output commands, you switch page 0 to the BASIC interpreter's ROM, and page 2, which is responsible for text interfaces, to RAM (see Fig. 2.3).

Chapter 2: SLOT

Figure 2.3: MSX Slot Configuration (Part 2)

CPU Memory View

0000H Slot 0 Slot 3-0 Slot 3-1 Page 0 Main ROM RAM Disk Main 4000H Page 1 ROM Disk ROM 8000H Page 2 Disk ROM RAM C000H Page 3 RAM FFFFH RAM

64 kilobytes of memory address is divided into four 16 kilobytes pages, and each can be selected in slots. For example, when processing disk output commands, Page 0 is BASIC's main ROM, Page 1 is disk ROM, Page 2 and 3 are main RAM.

A BASIC interpreter is a program that processes programs written in BASIC, and it is written in the ROM of the MSX system itself. For MSX1, it was in a 32 kilobytes ROM, but for MSX2, it consists of 48 kilobytes. For MSX2, the part that overlaps with MSX1 is a 32 kilobytes ”Main ROM”, and the expanded functionality part is written in a 16 kilobytes ”Sub ROM”.

Moreover, in the case of MSX with disk interface, the external disk drive interface cartridge is equipped with its own ROM, and a 16 kilobytes ROM for interfacing with the BASIC disk output program (DISK-BASIC). This is the state as depicted in Figure 2.3.

2.1.5 The Secret of MSX Extension Slot

The MSX slot is not just used for memory expansion, but also for expanding the MSX functionality. There are slots for connecting game cartridges, modem cartridges, etc.

As mentioned earlier, MSX with a disk interface cartridge equipped has a 16 kilobytes ROM slot in the disk interface. When performing disk output, the memory automatically switches to the disk interface slot ROM. Even if the disk interface is inherently equipped, it programmatically switches if connected via cartridge.

2.1 What is a slot?

There is no obstacle to its operation.

Also, when connecting a disk interface, the ”CALL FORMAT” command or the expansion BASIC command for ”CALL TELCOM” can be used when connected to a communications cartridge. These expansion commands are processed as ROMs for cartridges. On many MSX computers, peripheral devices can automatically expand BASIC commands just by connecting an interface cartridge.

Additionally, there are other expansions, such as “Japanese MSX-DOS2,” the integrated VT font “HALNOTE,” and “MSXView,” a GUI (Graphical User Interface) for the turbo R series, which is also software provided via cartridges. By connecting ROM cartridges to slots, upgraded functionalities can be easily added, showcasing one of the MSX features.

Slots are convenient features, but developing programs utilizing them can be challenging. For Z80 CPU MSX programmers, it might take some time to fully grasp the concept of slots.

2.1.6 How the MSX2+ slot has changed

As mentioned earlier, one of the appealing features of the MSX machine was having a rich set of expansion slots. However, this was also one of MSX's weaknesses. Depending on the machine, there were differences in slot structures, and this caused operational problems with some software.

For instance, certain software didn't work when the cartridge slot was divided into ROM and RAM expansion slots. This made DOS or other ROM features unusable, causing problems across multiple MSX machines. Although such issues should have been resolved, the differences in slot structures caused serious problems for programmers.

With the MSX2+, a certain level of standardization was achieved within the slot structure, as shown in Figures 2.4 and 2.5. An example of the MSX2+ slot configuration is shown, with internal slots divided into SLOT 0 and SLOT 1 & 2, and with slot expansions being classified into SLOT 0-1 and SLOT 3 expansions.

2.4 shows distinguishing features when SLOT 3 is expanded. When the standard slot is used, it divides the BASIC’s main ROM into SLOT 1 and SLOT 2, each functioning as partial cartridge ROMs.

Page 54

Figure 2.4: Example of MSX2+ Slot Configuration (When Expanding Only Slot 3)

The highlighted parts are standard specifications. The highlighted parts are optional specifications.

Slots 1 and 2 are external cartridge slots. This figure shows the standard slot configuration of MSX2+. However, note that there may be numeric differences depending on the machine (see the main text).

Slot 0-0 Page 0 0000H Main ROM 4000H Main ROM Page 1 8000H C000H
Page 2 FFFFH

Slot 3-0 0000H RAM 4000H RAM 8000H RAM C000H RAM FFFFH

Slot 3-1 0000H Sub ROM 4000H Kanji Driver 8000H Single Kanji Conversion License C000H FFFFH

Slot 3-2 0000H 4000H 8000H Disk ROM C000H FFFFH

Slot 3-3 0000H 4000H 8000H C000H FFFFH MUSIC

And so, in one of the expanded slots of slot 3, place a 64KB RAM (main RAM) range spanning pages 0-3 that fits completely within the slot. Similarly, place a 48KB range of sub ROM, Kanji driver, and single kanji conversion license of the corresponding slots. This means that, in the same environment as shown in the figure, RAM would be placed in the expanded slot 0 (the expanded slot of standard slot 3-0), and ROM would be placed in 3-1. However, this varies depending on the type of machine.

In response to this, assuming slots 1 and 3 expansions occur simultaneously, the Main ROM will be placed in the extended slot of 0-0 similar to Figure 2.4. Be careful not to exceed the standard 3-0 when placing the disk interface.

Note that the numbers and letters in Figures 2.4 and 2.5 correspond to different colors and parts.

MSX-MUSIC (FM sound generation) communication and multi-area conversion license ROM are marked as optional specifications for MSX2+.

Now, for MSX2+ with internal software equivalent to Figure 2.5, and only the main ROM connected to the external cartridge.

In the same manner as described for Figure 2.4, MSX2+ can theoretically hold internal software identical to Figure 2.5.

Figure 2.5: Example of MSX2+ Slot Configuration (when extending slots 0 and 3)

```
                Slot 0-0        Slot 0-1        Slot 0-2        Slot 0-3
```

0000H Page 0 Main ROM (empty) (empty) Communication ROM 4000H Page 1 Main ROM MUSIC Communication 8000H Page 2 (empty) (empty) (empty) C000H Page 3 (empty) (empty) (empty) FFFFH

```
                Slot 3-0        Slot 3-1        Slot 3-2        Slot 3-3
```

0000H Page 0 RAM Sub ROM Disk ROM Option 4000H Page 1 RAM Kanji Driver (empty) Option 8000H Page 2 RAM Single Kanji Conversion ROM (empty) Option C000H Page 3 RAM (empty) (empty) Option FFFFH

[Dark shaded parts are standard specifications] [Light shaded parts are optional specifications]

When communication software, etc., is built into the MSX2+ itself, slots 0 and 3 are expanded. Manufacturer-specific applications (like word processors) may also be included in slot 3-3. This means that the slot capacity can be increased, thereby reducing the number of combinations compared to MSX2. By placing sub-ROM and Kanji drivers or single kanji conversion in this slot, it's expected that the MSX2+'s kanji input/output performance will be significantly improved.

2.1.7 Expanding Slots

The external cartridge slot prepared exclusively for the MSX machine (the place to insert regular game cartridges) is a basic slot. Here, by connecting a ”slot expander,” you can expand to four extension slots. For example, the two*pi slots...

Page 56

Chapter 2 SLOT

If slot expanders are connected to both slots, a total of 8 slots will be made available.

One thing to be aware of is that some cartridges do not work with the expander slots. Examples include the type of Japanese MSX-DOS2 (the type with RAM built into the cartridge). Verify the software to be used and confirm whether it works with the expander slots before using it. Note that the Japanese MSX-DOS2 that does not have RAM built into the cartridge works fine with the expander slots.

The product name of the slot expander is ”MSX Expansion Slot Box - EX-4”. It is priced at 29,800 yen (excluding tax) and is available from Nippon Electronics (phone 03-3486-4181). In addition, several MSX manufacturers released slot expanders, but currently, it is almost impossible to obtain them.

2.2. Challenge of Slot Switching

MSX cannot be discussed without addressing the concept of slots. Here, we will center our discussion on how to control software to switch slots and the specification changes in MSX2+.

2.2.1. About Slot Switching

Although the significance of slots in MSX has been explained before, the method of switching slots has not been explained. From here on, we will introduce such methods. Of course, it's not about physically switching like a human would do with a switch, but about programmatically switching.

First, the CPU used in MSX is the Z80, and this has what is called ”I/O Ports.” This allows the CPU to communicate with various peripheral devices like VDP and FM sound sources through electrical signals. The Z80 has 256 such I/O ports in total, each numbered from 0 to 255 (in hexadecimal, from 00H to FFH), and they are distinguished by this numbering.

In BASIC, the handling of these ports is done with the `INP` function, which reads the 1-byte value of an I/O port, and the `OUT` command in machine language writes a 1-byte value to an I/O port.

If a value is written to the I/O port 0A8H, which is for slot switching, you can understand the current slot status by reading the value specified for each slot. bit 7 of the value corresponds to page 3, bit 6 to page 2, bit 5 to page 1, and bit 4 to page 0. Writing the value 11110000B (B indicates binary) to this port switches page 3 to slot 3, page 2 to slot 2, page

1 to slot 1, and page 0 to slot 0. As such, advanced slot control and memory utilization in FFFFH areas rely on complex routines, but we omit these here.

Also, while programs can directly switch slots, it might not be the best or the most fun, but it's crucial for machine operations. In practice, "BIOS" is used for slot switching. BIOS stands for "Basic Input Output System." This is a collection of routines stored in ROM for software management. Let's check the various functions handled by BIOS beyond slot switching as well.

2.2.2. How to Specify Slot Numbers

When switching slots using BIOS, you need to specify the base slot number and the expanded slot number. Usually, this is sufficient for identifying each slot, but occasionally, there can be confusion due to two CPU-level registers (temporary storage of data inside the CPU). Figure

Page 58

2.6 Slot numbers are specified by effectively using each bit of 8 bits (1 byte) to distinguish between primary slots and expansion slots. For instance, to specify primary slot 0, a value like 0000000B (in hexadecimal, this is 00H) is set, and to specify expansion slot 1 of primary slot 3, a value like 10000111B (87H) is set.

Figure 2.6: How to specify slot numbers

In the case of primary slots:

```
 b7  b6  b5  b4  b3  b2  b1  b0
[ 0 | x | x | x | x | x | x | x]
<== Primary slot number
       Meaningless
```

In the case of expansion slots:

```
 b7  b6  b5  b4  b3  b2  b1  b0
[ 1 | x | x | x | x | x | x | x]
<== Primary slot number ==>
              <== Expansion slot number
                     Meaningless
```

Slot numbers are expressed as 8-bit (1 byte) values as shown in the figure above. For example, to specify primary slot 0, the value 0000000B is used, and to specify expansion slot 1 of primary slot 3, the value 10000111B is used. Note that the content of meaningless bits is ignored.

2.2.3 BIOS functions that manipulate slots

First, memorize the notation to represent the functions of the BIOS. Let's set a value E that should be read before issuing a call to the BIOS instruction. R indicates a register to which the BIOS returns a value. M indicates that the BIOS loads a meaningless value. In other words, it states that the content of the register to be overwritten is ignored. The same applies when Y and X bytes are used. The specified slot is read by the HL register.

Function: RDSLT Address 000CH

A slot number specified in the register A, the address read by the HL register is read.

E: Slot number A HL: Address

R M Note: The value read out by A register. AF, BC, DE Overwriting is prohibited.

WRSLT 0014H Address Function: Write the contents of the E register to the address specified in the slot specified by the A register and the address specified by the HL register. E A Slot Number HL Address E Content to Write None AF, BC, D Overwriting is prohibited.

CALSLT 001CH Address Function: Call a subroutine in another slot. E IX Call Address IYH Slot Number Called party from IX, IY, back registers R M Note: Save the state of the current slot to the stack and execute the purpose subroutine. The contents of AF, BC, DE, HL registers will be passed to the subroutine as is, the subroutine will execute the RET command and then return to the original program. At this time, the values of the AF, BC, DE, HL registers passed from the subroutine. Whether a multi-byte stack is used or not depends on the slot configuration.

ENASLT 0024H Address Function: Switch slots. E A Slot Number H Page (upper 2 bits) None AF, BC, DE, HL

For example, to switch the page, it's sufficient to set the value in the H register to 80H~BFH. Do not set values outside this range.

CALLF Function 0030H address

```
Calls the subroutine in another slot.
Write the slot number and address continuously with the "RST 30H" instruction as
↪  shown in the program below.

RST 30H
DB  Slot number
DW  address

Caller according to
IX, IY, Shadow register
```

Note Except for the method of specifying slot and address, it is the same as CALSLT. It is used for special purposes (hacks).

EXTROM Function 0150H address

```
Calls the Sub ROM.
IX call address

Caller according to
IX, IY, Shadow register
```

Note Unless a sub-ROM slot is automatically selected, it works the same as CALSLT.

In summary, the BIOS introduced above has certain limitations. No matter which page, it cannot be used for page 3. For pages 0 to 2, it can be used without problems especially if DOS is calling the main ROM. Be careful not to run your programs using special slot structures on your MSX. Note that calling a sub-ROM from DOS and setting the slot configuration and disk interface type correctly may make the difference between whether your program runs or not.

2.2.4 Method to know the slot structure As mentioned earlier, the MSX slot structure varies depending on the machine due to options like Disk Interface. As a result, each machines' slot configuration...

2.2.5 Exploring the System Work Area

There are some game software for MSX2 that, if operated with MSX2+, display the title screen in SCREEN 12. Additionally, despite not having a modem cartridge, there are also ones that attempt to communicate.

As a result, there are programs that display error messages without exception. To create such software, we will introduce methods to examine the types and features of programs and cards.

First, ”How many disks are there?” To investigate, read the contents of the FFATH section. If it contains C9H, it is a disk. If not, it is the value after that.

To check the ”type of MSX,” read the main ROM's 2DH section. In the case of MSX1, it is MSX2. If it is MSX2+, it is MSX2+. If it is 3, it is called turbo.

Generally, the contents of 2BH and 2CH sections, but ”made overseas exported MSX” has numbers for keyboard and communication standard types. Exporting software should be careful. The two sections are confirmed.

Additionally, it is necessary to know that MSX computers produce relative frequency numbers at schools in Europe and America, including Zimbabwe, and also adjacent countries in Asia. For example, in the case of international machines, it was understood.

Now, ”To check the amount of video RAM,” read the FAFCH section. The value of bits 2 and 1 for bits 0 to 63 is the number of RAM. 00 means 16K ROM, 01 means 64K ROM, and 10 means 128K ROM. The other bits are used for other purposes, so they do not matter now.

Referring to the next page list, the value of ”AND 6” between bits 2 and 1 can be obtained.

In this case, if there is ”expansion BIOS”, it can be checked for. This is necessary to control typical modems, FM sound sources, Chinese characters, and optional hardware. The contents of FFCBH's bits indicate the program's slot number.

If no value is defined, it is not a problem. The slot numbers and bit values show what is included. Therefore, unnecessary matter can be confirmed using ”AND.”

Although the machine is similar, differences exist in expansion slots. Hence, try with your own machine or on a friend's machine. It is recommended to test on many machines to see the results.

2.2 Slot switching challenge 63

List 2.1 (WHO_AM_I.BAS)

```
100 ' Analyzing slot structure of MSX
110 ' by nao-j on 9. Jan. 1989
120 CLEAR : DEFINT A-Z : CLS
130 VE = PEEK(&H2D) : ' Version No. of BASIC
140 IF VE=0 THEN PRINT "I am MSX1"
150 IF VE=1 THEN PRINT "I am MSX2"
160 IF VE=2 THEN PRINT "I am MSX2+"
165 IF VE=3 THEN PRINT "I am MSX turbo R"
170 IF VE>3 THEN PRINT "Who am I ?"
180 VR = (PEEK(&HFAFC) AND 6) ¥ 2 : ' size of VRAM
190 IF VR=0 THEN PRINT "VRAM 16KB"
200 IF VR=1 THEN PRINT "VRAM 64KB"
210 IF VR=2 THEN PRINT "VRAM 128KB"
220 MR = PEEK(&HFC49) : ' size of main RAM
230 IF MR < &H20 THEN PRINT "RAM 8KB"
240 IF MR < &H40 AND MR >= &H20 THEN PRINT "RAM 16KB"
250 IF MR < &H80 THEN PRINT "RAM >= 32KB"
260 FOR SS = 0 TO 3 : EXPTBL
270 PRINT USING "Slot # is "; SS;
280 FF = PEEK(&HFCc1 + SS) AND 128
290 IF FF THEN PRINT "expanded slot" ELSE PRINT "primary slot"
300 NEXT SS
310 PRINT
```

```
320 SS = PEEK(&HFCc1) : PRINT "Main ROM is in "; : GOSUB 530
330 SS = PEEK(&HFAF8) : PRINT "Sub  ROM is in "; : GOSUB 530
340 IF PEEK(&HFF7) <> &HC9 THEN 360
350 PRINT "I have no disk." : GOTO 450
360 PRINT "I have disk(s)."
370 SS = PEEK(&HF348) : PRINT "FDC ROM is in "; : GOSUB 530
380 SS = PEEK(&HFc41) : PRINT "P0  RAM is in "; : GOSUB 530
400 SS = PEEK(&HFc42) : PRINT "P1  RAM is in "; : GOSUB 530
410 SS = PEEK(&HFc43) : PRINT "P2  RAM is in "; : GOSUB 530
420 SS = PEEK(&HFc44) : PRINT "P3  RAM is in "; : GOSUB 530
430 PRINT "Bottom address of disk work area is ";
440 PRINT RIGHT$("00"+HEX$(PEEK(&HFC4B)),2);
450 PRINT RIGHT$("00"+HEX$(PEEK(&HFC4A)),2)
460 ' detecting extended BIOS
470 IF (PEEK(&HFB20) AND 1) = 0 THEN GOTO 520
480 IF PEEK(&HFcBE) = &HC9 THEN GOTO 520
490 PRINT : PRINT "I have extended BIOS."
500 SS = PEEK(&HFFcB)
510 PRINT "ROM of the extended BIOS may be in "
520 END
530 ' displaying slot number
540 PRINT USING "primary slot #"; SS AND 3;
560 IF (SS AND 128) = 0 THEN 570
560 PRINT USING " extended slot #"; (SS AND 12) ¥ 4;
570 PRINT : RETURN
```

The text mainly includes lines of code and comments from a BASIC program that indicates it analyzes the slot structure of an MSX computer system.

2.2.6 MSX2+ Hardware Specifications

For the MSX2+, minor improvements have been added to the hardware. Table 2.2 lists the new specifications and additional I/O ports.

Table 2.2: MSX2+ I/O Ports | I/O Address | Usage | |-------------|--------------------------| | 7CH | Internal FM Sound Power | | 7DH | Internal FM Sound | | DAH | 2nd Megabit ROM | | D8H | 2nd Megabit ROM | | F4H | Initial Diagnostics | | F5H | Device Enable |

These added specifications included improvements such as the following. If your software does not use I/O ports directly but instead uses BIOS, these new specifications will be properly applied without the need for special programming.

The addresses 7CH and 7DH are I/O addresses used to control the internal FM sound power. This is because the FM sound provided by the cartridge will still use the same I/O port as the ”FM-PAC” package.

To check whether the FM sound is built into the unit, you can examine if the program from address 401FH matches the pattern stored in the ROM of the FM sound controller slot. If it matches, the FM sound is built into the unit.

On the other hand, for FM sound cartridges, the data from address 4018H includes characters such as “PAC” or “OPLL” to denote different models.

Some modem cartridges, like ”MSX-Write”, may display the MSX title screen with ”BASIC” in the initial menu. This is because they jump to the main ROM's 0 address for a reset.

In the past, it was uncertain how to validate a genuine reset to address 0 in slot jumping. Therefore, from MSX2+, hardware to check the reset status of the F4H I/O port has been incorporated.

As an example, BIOS added to the main ROM of MSX2+ is: CALL 17AH OR 80H CALL 17DH JP 0

When called during initialization, the ROM cartridge program is:

2.2 Challenging Slot Switching 63

CALL 17AH performs this. Then, if bit 7 of the A register is 0, it is a real reset; if it is 1, it means the reset was invoked by oneself.

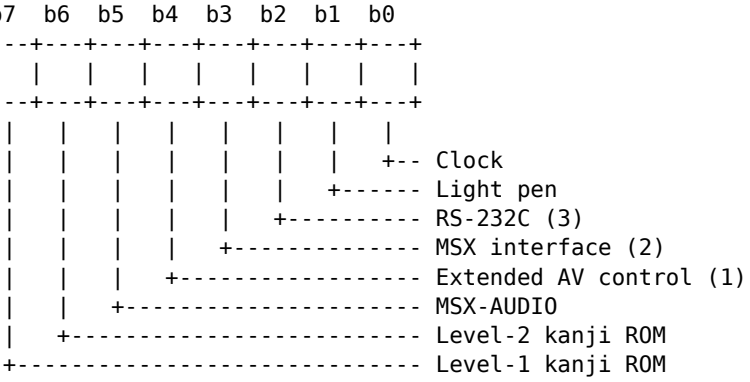
2.2.7 Device Enable to Prevent Conflicts

When you connect to an MSX, for example, a kanji-ROM cartridge that contains a kanji ROM, there is a risk that hardware addresses will conflict and cause a malfunction. To prevent this, there is a function called ”Device Enable,” controlled by I/O port number F5H.

The hardware shown in Figure 2.7 is disconnected during a reset. By writing an 8-bit value to I/O port number F5H, the hardware corresponding to the set bits is connected to the bus. The writing of this value (in software terms, performed before a reset or before jumping to address 0 of the main ROM) automatically enables the hardware.

In MSX2 and earlier, even when hardware was connected, there was no rule to disconnect the hardware during a reset.

Figure 2.7: Device Enable



With I/O port number F5H, the hardware in the device can be enabled or disabled at will according to the value written (the hardware corresponding to a bit set to 1 is enabled). (1) Superimpose function; controlled by I/O port number F7H, etc. (2) Not yet implemented in the specification. (3) Not related to modems.

Write” etc., if you try to re-initialize the BIOS by jumping to address 0 in the ROM, there could be confusion. Therefore, starting from MSX2, it was standardized so that internal hardware can be disconnected by writing 0. By doing this, utilizing the F4H and F5H ports of the I/O ports and creating basic software for MSX2, compatibility and reliability have improved compared to before.

2.3 Slot Configuration of MSX turbo R

In this section, we will explain the slot configuration of the newly released MSX turbo R. It is significant to note that, at long last, the slot configuration has been unified. This is indeed a profoundly meaningful matter.

2.3.1 Slot Configuration Finally Unified

Fig. 2.8 depicts the slot configuration of the turbo R. To accommodate the higher CPU speeds and to facilitate the development and debugging of application programs, the slot

configuration has been unified.

In this figure, Slot 3 appears to have 64KB of RAM, but in actuality, it is equipped with 128KB of RAM, as 256KB is mounted overall (including slot space). Portions exceeding 64KB are used ordinarily in ”D-RAM mode,” explained in the DOS2 firmware and RAM disk sections, but programs can swap in another set of RAM using expanded BIOS.

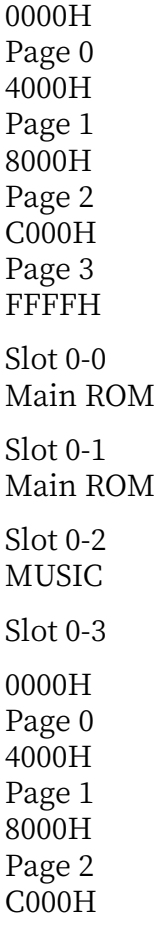
Furthermore, page 1 of slot 3 has a DOS system ROM installed. In other words, slot 3-1 contains the same MSX-DOS2 ROM as the traditional MSX-DOS2. It automatically switches when necessary.

The greatest advantage of the standardized slot configuration is that the standard DOS programs can now control the ROM and its distribution by dividing the segments and placing them in appropriate slots. An article in a previous issue of our magazine indicated that in the past, with extended slots containing RAM and Sub-ROM, there would be issues with program split and processing. However, that shouldn’t be a problem with the turbo R thanks to the expanded slot capabilities in RAM and Sub-ROM.

Additionally, the drivers for OPLL, that is, FM-BIOS ROM, are significantly placed in slot 0-2. Therefore, the FM-BIOS search routine for turbo R-specific software is simplified, as specific slot searches need not be prioritized.

With the unification of the slot configuration, there are notable benefits for game processing, especially the ”Kompare” for initializing, but also loading slots 1 and 2, and resolving the issues some MSX1 and MSX2 games had that prevented them from running. Now, with the MSX2+ and turbo R, it is a condition where unique games and application software developed easily.

Figure 2.8: MSX turbo R Slot Configuration



Page 3
FFFFH

Slot 1

0000H
Page 0
4000H
Page 1
8000H
Page 2
C000H
Page 3
FFFFH

Slot 2

0000H
Page 0
4000H
Page 1
8000H
Page 2
C000H
Page 3
FFFFH

Slot 3-0
RAM
(Note 1)

Slot 3-1
Sub ROM
Kanji Driver

Slot 3-2
DOS
(Note 2)

Slot 3-3
Manufacturer Option

Until now, although there were patterns of slot configurations, in the turbo R, it has been standardized in this way. (Note 1) Main RAM means 256 kilobytes of RAM assigned to memory mapper units. In (Note 2), MSX-DOS1 (16 kilobytes) and MSX-DOS2 (48 kilobytes) are referenced, and they can be automatically switched according to the state.

2.3 MSX turbo R's Slot Configuration

Additionally, for software houses, the unified slot configuration means that issues arising from particular slot configurations are greatly reduced, which is the biggest benefit. From the standpoint of developing software, having a unified slot configuration is much more convenient than having upgraded CPUs and increased RAM capacity. Long live turbo R!

(This page is too faint in the original to transcribe.)

Chapter 3 Kanji BASIC

72 Chapter 3 Kanji BASIC

This chapter is a re-edited version of an article titled ”Exploring MSX2+ Technical Topics” from the April 1989 issue of MSX Magazine.

3.1 Analyzing Kanji BASIC

One of the features of MSX2+ and later machines, as well as DOS2, is the improved ease of using kanji. In addition to strings and file names in BASIC programs, you can also use kanji characters. This chapter reports on the functions of Kanji BASIC.

3.1.1 Hardware Required for Kanji BASIC

How can Kanji BASIC be used? First, the basic requirement is to have an MSX2+ or turbo R main unit, or to insert a DOS2 cartridge. In both cases, the Kanji BASIC ROM is built in. The differences on the hardware side are that the MSX2+ and turbo R can naturally use the SCREEN 10 to 12 modes, and that the kanji ROM is built into the main unit.

Also, something to pay special attention to is that Kanji BASIC and a kanji ROM are built into the Japanese-language study cartridge ”HBI-J1” released by Sony. Therefore, by adding a kanji ROM to an MSX2, you can use Kanji BASIC.

DOS2, MSX2+, and turbo R all have a ”kanji conversion” function. This uses a ”reading” or a ”JIS code” to specify and convert a single kanji character. Until now, inputting long sentences was cumbersome, but by adding the ”MSX-JE” input method with its advanced sentence-conversion function, it became possible to convert long sentences into kanji.

For example, ”ashita wa hare deshou” is converted into ”明日は晴れでしょう” (it will probably be fine tomorrow), enabling the appropriate kanji to be inserted into the text. This kind of conversion is widely used in dedicated word processors. Table 3.1 shows a list of hardware that includes the MSX-JE function.

Also, some of these support the conversion function, so by combining them with an MSX2+, turbo R, DOS2, etc., it is possible to use Kanji BASIC.

MSX-JE has a learning function. For instance, for a frequently used word such as ”kanban” (signboard), the preferred kanji are displayed as the leading candidates. In addition, once the user manually selects a preferred kanji conversion, it is automatically given priority in future conversions, making sentence input smooth. This learning data is stored in SRAM, up to a maximum of 8 KB. Because the SRAM contents are retained even when the power is turned off, the more you use it, the easier it becomes to use.

Table 3.1: MSX-JE Built-in Hardware List

Manufacturer	Product Name	Type	Kanji ROM	SRAM
Panasonic	FS-A1ST	MSX turbo R	1, 2	Yes
Panasonic	FS-A1WSX	MSX2+	1, 2	Yes
Panasonic	FS-A1WX	MSX2+	1, 2	Yes
Panasonic	FS-SR021	Cartridge (1)	1, 2	Yes
Panasonic	FS-4600F	MSX2	-	No
Panasonic	FS-PW1	Printer (2)	-	No
Sony	HB-F1XV	MSX2+	1, 2	Yes
Sony	HB-F1XDJ	MSX2+	1, 2	Yes
Sony	HBJ-J1	Cartridge	1, 2	Yes
Sanyo	PHC-77	MSX2	-	No
HAL Laboratory	HALNOTE	Cartridge (3)	1, 2	Yes
ASCII	MSX-Write	MSX-Write	-	No
ASCII	MSX-Write II	Cartridge	1, 2	Yes

For Kanji ROM, ”1” and ”2” indicate the presence of the first and second kanji fonts, respectively. The ”SRAM” column indicates whether the hardware is equipped with SRAM or not. (1) AWX with ROM and cartridge (2) Cartridge and dedicated printer set (3) Cartridge with dedicated OS HALOS.

3.1.2 What is the Software Compatible with MSX-JE?

MSX-JE is not just a word processor or expanded BASIC, but a configuration combining various applications to utilize the convenient functions of continuous conversion and others. Though Kanji ROMs and continuous conversion dictionaries with ROM are relatively high-priced, the fact that many software applications for MSX-JE are shared means they are economically beneficial. An application software manual is sold with MSX-JE utilization steps explained in detail. The software labeled as ”MSX-JE compatible” can be combined with any MSX-JE. However, HALNOTE in Table 3.1 uses a specific OS called ”HALOS” when combined with a cartridge-stored disk, so it has limitations when combined with non-HALOS software.

3.1.3 Explanation of the Operation Principle of Kanji Driver

Regarding MSX, let's briefly explain the method to display kanji inside the computer. Firstly, although single-byte katakana characters (8-bit) can be represented, kanji characters require 2 bytes (16-bit) of characters to be displayed. According to JIS (Japan Industrial Standards), kanji characters can be handled as follows:

are represented by a 2-byte code:

亜 … 3021H (Level 1) 腕 … 4F53H (Level 1) 哀 … 5021H (Level 2) 塙 … 737EH (Level 2)

However, this ”JIS code” has the problem of being cumbersome when mixing alphabetic characters and kanji. Therefore, the JIS code was reworked into a form that is easier to process, called ”Shift JIS code,” and this was used in personal computers including the MSX. In some places it is referred to as ”Microsoft kanji code,” though this is not entirely accurate.

Shift JIS code is designed so that software originally created for English text can be modified only slightly to handle Japanese as well. Conveniently, a single byte represents a half-width character and two bytes represent a full-width character. Operating systems such as MS-DOS and OS-9, as well as recent operating systems, have incorporated Shift JIS (the JIS kanji code). Therefore, different computers can exchange files encoded in Shift JIS without too much trouble.

Now, if you look at the codes, you will notice that confusion is avoided by using 2-byte codes for kanji and other such characters, while alphabetic and numeric characters are represented by single-byte codes.

So, characters represented by a single byte are ”half-width characters,” while those represented by a 2-byte code are ”full-width characters.” It is important to be aware of this; characters like ”ABCD” should be treated as half-width characters.

That said, to get to the main subject: this ability to output kanji has been achieved on the MSX2+ and turbo R by means of DOS2 and the kanji driver. Both BASIC and DOS can take advantage of this kanji driver, and application programs can use it as well.

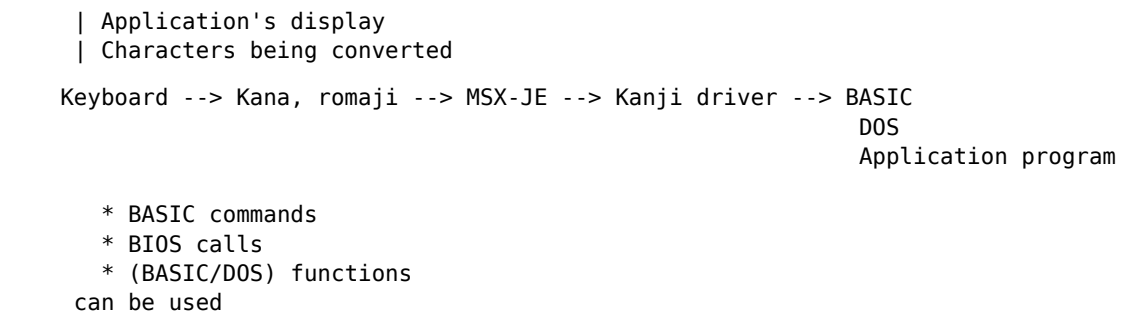
Figure 3.1 explains the operation of the kanji driver step by step. First, the kanji entered into the input buffer is analyzed (judged) by JE, then displayed on the screen. Next, after conversion by KE, the converted kanji code is sent to the application program (BASIC or DOS).

Thus, when an application or system program displays text on the screen, the character code is sent to the kanji driver, which handles the processing.

This kanji driver is designed to add kanji-handling capability to application programs by converting JIS codes, DOS commands, and so on, just as the BIOS does. Although it may look as if only kanji-handling functionality has been added, applications can now make good use of it.

3.1 Analyzing Kanji BASIC 75

Figure 3.1: Operating principle of the kanji driver



One of the major advantages of the kanji driver is that the application program itself does not need to operate the MSX-JE dictionary.

3.1.4 JE-Compatible Hardware and Software

For reference, here we have summarized the hardware and software released so far that are compatible with MSX-JE.

- Modem cartridges

Panasonic	FS-CM1
Panasonic	FS-CM820
Sony	HB-1200
Canon	VM-300
Akashi Denki	V-3

- MSX with built-in modem

Panasonic	FS-A1FM
Sony	HB-T7
Sony	HB-T600
Mitsubishi	ML-TS2H

Page 76

• Software

Sony - Shugisha Sakueamon
Sony - Hagaki Kakikiyomon
ASCII - MSX-TERM
ASCII - MSX-DOS2 TOOLS
ASCII - MSXView

3.1.5 Various screen modes available in Kanji BASIC

As the screen modes of MSX2+ and turbo R are slightly increased, the screen modes of Kanji BASIC are also diversified. Table 3.2 displays a list of them. In BASIC,

```
CALL KANJI
WIDTH
```

The commands are used to specify each screen mode. For example,

```
CALL KANJI: WIDTH 32
```

By executing the command, the screen will display 32 characters by 12 lines. To do the same thing in DOS2,

```
KMODE 0
MODE 32
```

can be used. However, if you build an English system, the Kanji driver is not read, so you need to call the Kanji mode via BASIC once.

Now, I will briefly explain the meaning of Table 3.2. First, ”screen dot count” refers to the number of dots (points) displayed on the screen. The combination of Kanji characters is also expressed with the same dots. The 512x424 dot screen mode uses the ”interlace” display method. This is a technique where two screens with half vertical shifts are displayed alternately, effectively presenting one screen. This method also has a disadvantage of causing flickering, but it helps in increasing the resolution.

”Kanji dot count” refers to the number of dots used to represent one Kanji character. Normal Kanji characters are presented using 16 x 16 dots, but they are compressed to 12 x 12 in the 512-dot screen width mode. Additionally, this mode can display 40 characters horizontally. Since the pattern ROM (Kanji ROM) is automatically selected when connected, the 12 x 12 dot Kanji ROM already embedded is selected.

Table 3.2: Screen Modes of Kanji BASIC

Screen Dot Count	Kanji Character Count	Half-Width Character Count	Setting Method
256 x 212	16 x 16	32 x 12	CALL KANJI0: WIDTH 32
256 x 212	16 x 16	40 x 12	CALL KANJI1: WIDTH 40
256 x 424	16 x 16	32 x 24	CALL KANJI2: WIDTH 32
256 x 424	16 x 16	40 x 24	CALL KANJI3: WIDTH 40
512 x 212	16 x 16	64 x 12	CALL KANJI0: WIDTH 64
512 x 212	16 x 16	80 x 12	CALL KANJI1: WIDTH 80
512 x 424	16 x 16	64 x 24	CALL KANJI2: WIDTH 64
512 x 424	12 x 16	80 x 24	CALL KANJI3: WIDTH 80

”Half-Width Character Count” is the number of columns and rows of half-width characters displayed on the screen. The values are naturally half of the Kanji character count in the table.

One thing to be cautious about is that not all rows in the table can be used with BASIC. This is because the rows used for displaying function keys and Kanji conversion are occupied, making the bottom 1-2 rows unavailable on the screen.

3.1.6 Kanji Text and Kanji Graphics

Now, the screen mode seems simple but involves a bit of complexity. In BASIC:

```
CALL KANJI1
WIDTH 40
SCREEN 0
```

When you execute this command, it refers to the 2nd screen mode from the top of Table 3.2. Notice that although ”SCREEN 0” is specified, this sets VDP to ”SCREEN 5” mode with a 256 x 212 dot configuration. This state is called ”Kanji Text Mode.”

In this mode, you can edit BASIC programs and input using the INPUT command or PRINT command, among others. However, graphics features such as LINE and PAINT cannot be used.

To operate graphics in Kanji mode, you need to switch the screen to ”Kanji Graphic Mode” using commands like ”SCREEN 5.” However, please remember that in this mode graphics features and Kanji text cannot be used simultaneously. To avoid confusion, the screen mode switch is summarized in Table 3.2 above.

Figure 3.2: Switching Display Modes

	CALL KANJI	SCREEN 2~12
ANK	Text	
	Kanji Text	Kanji Graphic

CALL ANK	SCREEN 0~1
Program Halt	
INPUT Command	

When a program starts running or when it stops or encounters an error, it is confirmed that the pre-set display mode is set. If you mistakenly press ”CALL KANJI” while the program is working properly, it will work correctly just after inputting the program, but there have been uncomfortable cases where it does not work properly if you turn off the power once and reload (the program). Also, in Kanji graphic mode, when using the ”INPUT” command or ”LINE INPUT” command, it automatically returns to the Kanji text mode. To avoid this, read from the ”INPUTS” or ”INKEY$” function keyboard.

3.1.7 The Correct Use of the Kanji Driver

There is a common belief that the Kanji functionality in MSX is not practical. It is a so-called ”extravagant” software. But since it is attached to BASIC, there is pain and trouble for the users to get used to. Not only the text explanation but also the awareness of the Kanji text and Kanji graphic switching is a great inconvenience. Here, we look back on the cautions of how to use such a Kanji driver.

When the ”CALL KANJI” command is executed after setting the MSX key, the work area that uses Kanji drivers to set MSX-JE etc. is initialized when the area is used. But, for example,

```
10 A = 1
20 CALL KANJI
30 PRINT A
```

When this program is executed, the value ”0” is displayed the first time (in other words, the variable has become reset). But from the second time onwards, the value ”1” is displayed correctly.

10 GOSUB 80 ...

```
70 END
80 CALL KANJI0
90 RETURN
```

When running a program like this, if ”CALL KANJI0” is executed, and there is no place to return with the ”RETURN” command, the program will behave strangely.

In addition to this, if the Kanji driver secures a work area in memory, there is a problem that the work area for BASIC is greatly reduced. With the typical program, machine language routine, and memory-inadequate programs, you might not be able to run them completely.

The Kanji driver has a function to convert the program's internal characters to JIS code and then print them with a Kanji printer. This function applies to BASIC's ”LPRINT” command and BIOS's ”LPT OUT”. However, to print to a printer, the cooling control is required, and in the case of ”bit image printing” where graphics are printed, it may spoil the subtle tone. Therefore, before bit image printing starts, you must write a value other than zero to system RAM work area in location F418H (called RAWPRT) to prevent the conversion by the Kanji driver.

From now on, the story is for advanced programmers. First of all, in Table 3.3.

Table 3.3: Hooks used by the Kanji driver

Number	Name	Function
FD9FH	H.TIM1	Timer input
FDA4H	H.CHPU	Display one character on the screen
FDA9H	H.DSPC	Display cursor
FDAEH	H.ERAC	Erase the cursor
FDB3H	H.DSPF	Display function key
FDB8H	H.ERAF	Erase function key
FDBDH	H.TOTE	Switch the screen text mode
FDC2H	H.CHGE	Write one character on the screen
FDDBH	H.PINL	Read one BASIC line text editor
FDE5H	H.INL	Read one line
FF84H	H.LWID	Handle BASIC width command
FF89H	H.LPTO	Print one character to the printer
FFCOH	H.SCRE	Handle BASIC screen command
FFCAH	FCALL	BIOS extension

If an application program uses any of these hooks, it is likely that the Kanji drivers may not work correctly.

This should give you an idea of what the text says.

A list of hooks that can replace the Kanji driver. If another program replaces these hooks, be aware that the Kanji driver may not work correctly.

Next, if the Kanji driver is invoked from an interrupt call, the contents of registers IX and IY will be corrupted. Thus, for BIOS related to Kanji input/output, assume that the contents of these registers must be preserved, and programs made with this in mind will not work properly in Kanji mode.

Chapter 4 V9958 VDP

--- Page 82 | Chapter 4: V9958 VDP ---

From sections 1 to 4 of this chapter, the editor has recompiled the ”V9938 MSX-VIDEO Technical Data Book” and ”V9958 Specifications.” The common functions of the V9938 and V9958 are described here, so please refer to sources like ”MSX-Datapack.”

Sections 5 to 7 of this chapter re-edit articles from ”MSX Magazine” from December 1988, January 1989, November 1989, December 1989, and January 1990 issues, which were edited by the ”MSX Technical Examination Team.”

Hardware documentation like the ”V9958 Specifications” commonly explains the ”VDP Modes,” but in this book, the screen modes of MSX BASIC are explained in line with MSX

Magazine articles. The relationship between the VDP modes and the screen modes of MSX BASIC is shown in Table 4.1.

Table 4.1: Correspondence between VDP Modes and BASIC Screen Modes

VDP Mode	BASIC Screen Mode
TEXT 1	SCREEN 0: WIDTH 40
TEXT 2	SCREEN 0: WIDTH 80
MULTI COLOR	SCREEN 3
GRAPHIC 1	SCREEN 1
GRAPHIC 2	SCREEN 2
GRAPHIC 3	SCREEN 4
GRAPHIC 4	SCREEN 5
GRAPHIC 5	SCREEN 6
GRAPHIC 6	SCREEN 7
GRAPHIC 7	SCREEN 8 (SCREEN 10 ~ 12)

4.1 V9958 Register List

Table 4.2: Mode Registers

|R#| |--|--|--|--|--|--|--|--| |b7|b6|b5|b4|b3|b2|b1|b0| |Mode 0 | |R#0|0|DG|IE#1|IE0|Ms|M4|M3|0|
| |Mode 1 | |R#1|0|BL|IE0|Mh|M2|0|SI|MAG| |Pattern name T.B.A.| |R#2|0|A16|A15|A14|A13|A12|A11|A10|
| |Color T.B.A. (Low)| |R#3|0|A13|A12|A11|A10|0|A9|A8| |Pattern gen. T.B.A.| |R#4|0|0|A16|A15|A14|A13|A
	Sprite attr. T.B.A. (Low)		R#5	0	A14	A13	A12	A11	A10	A9	A8	A7		Sprite pat. gen. T.B.A.
R#6	0	0	A16	A15	A14	A13	A12	A11		Text / Back drop color		R#7	MS#	LP#	TP	TP*	VR*	SPD	BM*	BW*
	Mode 2		R#8	LN	0	S#	0	IL	EO	NT	DC*			Mode 3		R#9	0	0	0	A15	A14	A13	A12	A11
	Color T.B.A. (High)		R#10	0	0	0	A10	A9	A8	A7	A6	A5			Sprite attr. T.B.A. (High)
R#11	T23	T22	T21	T20	BC3	BC2	BC1	BC0		Text / Back color		R#12	ON3	ON2	ON1	ON0	OF3	OF2	OF1	OF
	Blinking period		R#13	0	0	0	0	A13	A12	A11	A10		VRAM access base addr.		R#14	0	0	0	0	S3	S2	S1	S0
	Status reg. pointer		R#15	0	0	0	CO	B3	B2	B1	B0		Color palette addr.		R#16	0	0	0	RS3	RS2	RS1	RS0
	Control reg. pointer		R#17	V3	V2	V1	VIS	C3	C2	C1	C0		Display adjust		R#18	IL7	IL6	IL5	IL4	IL3	IL2	I
	Interrupt line		R#19	0	0	0	0	0	1	1			Color burst 1		R#20	0	1	1	1	1	1			Color burst 2
R#21	0	1	0	1	0	1	0			Color burst 3		R#22	D7	D6	D5	D4	D3	D2	D1	D0			Display offset
R#23	0	0	VDS*	YAE	YJK	WTE	MSK	SP2			R#24			R#25	HO5	HO7	HO6	HO5	HO4	HO3	
Horizontal scroll (High)		R#26	H0			R#27			Horizontal scroll (Low)

T.B.A.: table base address

- In bits where ”0” is written in this table, make sure to write 0. (Note: Hardware control use only, do not change for normal application programs.) † Flags do not exist in V9938, so be sure to write 0. Additional register of the new V9958. Their initial values are 0, for the V9958 function, same as V9938. Register 24 is reserved.

Table 4.3: Command Registers

br	b6	b5	b4	b3	b2	b1	b0	Function
R#32	Sx7	Sx6	Sx5	Sx4	Sx3	Sx2	Sx1	Sx0
R#33	Sx15	Sx14	Sx13	Sx12	Sx11	Sx10	Sx9	Sx8
R#34	Sy7	Sy6	Sy5	Sy4	Sy3	Sy2	Sy1	Sy0
R#35	Sy15	Sy14	Sy13	Sy12	Sy11	Sy10	Sy9	Sy8
R#36	Dx7	Dx6	Dx5	Dx4	Dx3	Dx2	Dx1	Dx0
R#37	Dx15	Dx14	Dx13	Dx12	Dx11	Dx10	Dx9	Dx8
R#38	Dy7	Dy6	Dy5	Dy4	Dy3	Dy2	Dy1	Dy0
R#39	Dy15	Dy14	Dy13	Dy12	Dy11	Dy10	Dy9	Dy8
R#40	Nx7	Nx6	Nx5	Nx4	Nx3	Nx2	Nx1	Nx0
R#41	Nx15	Nx14	Nx13	Nx12	Nx11	Nx10	Nx9	Nx8

br	b6	b5	b4	b3	b2	b1	b0	Function
R#42	Ny7	Ny6	Ny5	Ny4	Ny3	Ny2	Ny1	Ny0
R#43	Ny15	Ny14	Ny13	Ny12	Ny11	Ny10	Ny9	Ny8
R#44	..	..	CH	CH0	CL3	CL2	CL1	CL0
R#45	MXC	MXD	MXS	DTY	DTX	EQ	MAJ	..
R#46	CM3	CM2	CM1	CM0	L03	L02	L01	L00

Table 4.4: Status Registers

br	b6	b5	b4	b3	b2	b1	b0	Function
S#0	F7	F5-F	..	S54	S53	S52	S51	S50
S#1	FL	FI	SD	..	IDA	T8	TDo	FH
S#2	TR	VR	HR	BD	..	E0	CE	-
S#3	X7	X6	X5	X4	X3	X2	X1	X0
S#4	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
S#5	-	-	-	Y7	Y6	Y5	Y4	Y3
S#6	-	-	-	Y2	Y1	Y0	-	-
S#7	C7	C6	C5	C4	C3	C2	C1	C0
S#8	BX7	BX6	BX5	BX4	BX3	BX2	BX1	BX0
S#9	-	1	1	1	1	BX8	BX9	-

※ The significant bits for functions that exist in the V9938 but not in the V9958 are meaningless for the V9958. The ID for the V9958 is 00010B.

4.2 New Features of V9958

4.2.1 Horizontal Scroll

HO☒ HO☒ HO☒ HO☒ are the horizontal screen scroll amounts; set in SCREEN 6 in 2 dot units and in other screen modes in 1 dot unit:

```
SP2 = 0 (default), horizontal screen size is 1 page.
SP2 = 1, horizontal screen size is 2 pages.

MSK = 0 (default), left edge mask not applied.

MSK = 1, 16 dots on the left edge in SCREEN 6 and 8 dots in other screen modes are masked,
↪  displaying the border color.
```

HO☒~HO☒ settings scroll the display screen to the left by the set amount: 8 dot units (SCREEN 6) or 16 dot units in others.

Table 4.1: Horizontal Scroll (When SP2 = 0)

Display Screen

HO7-3 8dot

0 0 1 30 31 1 line

```
    1 2 31 0 1
    . . . . . .
    31 0 29 30 1
```

Figure 4.2: Horizontal Scroll (In the case of SP2 = 1) Display Screen

HO 8-3 | 8dot | 0 | 1 | 31 | 32 | 62 | 63 | 1 line

```
  0    |        | [0] | [1]
  1    |        | [1] | [2] | 32  | 33 | 63  | [0]
               .
               .
```

```
 31    |        | 31  | 32  | 62  | 63 | 29  | 30
 32    |        | 32  | 33  | 63  | [0] | 30  | 31
                .
                .
 63    |        | 63  | [0] | [0] | 30 | 31  | 61  | 62
```

When SP2 = 0, the data of 1 screen will be displayed with a horizontal scroll. HO☒ is ignored.

When SP2 = 1, the data of 2 screens will be displayed with a horizontal scroll. Set 1 as the base address of the pattern name table A15. As the base address of the pattern name table, 0-31 means it is set to 0, 32-63 means it is set to 1. The base address of the pattern generator table and the color table remain the same and do not change depending on the scroll.

For HO☒ to HO☒, the display value shifts to the right based on the set value, by 1 dot unit (in SCREEN6 and 7 it shifts by 2 dot units) for each shift.

4.2.2 WAIT

R#25

b7	b6	b5	b4	b3	b2	b1	b0
0	CMD	VDS	YAE	YJK	WTE	MSK	SP2

```
WTE = 0 (initial value) disables the wait function.

WTE = 1 enables the wait function. When the CPU accesses VRAM, the V9958's VRAM access must
↪  complete until the wait is applied to all access to the V9958 ports. The wait only
↪  applies to access to the register and color palette; the wait does not apply to command
↪  data transfer.
```

4.2.3 COMMAND

R#25

b7	b6	b5	b4	b3	b2	b1	b0
0	CMD	VDS	YAE	YJK	WTE	MSK	SP2

```
CMD = 0 (initial value) enables the COMMAND function only in SCREEN 5 to 12 modes.

CMD = 1 enables the COMMAND function in all screen modes. In modes other than SCREEN 5 to
↪  12, it operates as SCREEN 8. Thus, parameters can be set as X-Y coordinates of SCREEN 8.
```

4.2.4 YJK Display Mode

Register Setting

R#25

b7	b6	b5	b4	b3	b2	b1	b0
0	CMD	VDS	YAE	YJK	WTE	MSK	SP2

```
YJK = 0 (initial value) treats the data on VRAM in RGB mode (3, 3, 2-bit modes). Sprite
↪  colors are the same as before.

YJK = 1 converts the data on VRAM to YJK mode and then to RGB signals (each 5-bit),
↪  outputting it analogously from the RGB terminal. The number of sprite display colors
↪  increases to 16 colors.
```

YAE selects the YJK data format.

When YAE = 0

No attribute. The data format is shown next. Loop the next four dots continuously.

For YAE = 1 case

There is an attribute per dot. The data format is shown below. Continuous 4 dots are grouped and expressed.

```
C7    C6    C5    C4    C3    C2    C1    C0
                          K_L
```

Y K_H

```
                Y                          J_L
                                       Y    J_H

C7    C6    C5    C4    C3    C2    C1    C0
                          K_L
```

Y A K_H

```
                Y    A                     J_L
                                       Y    A    J_H
```

```
A = 0 (initial value), then Y, J, K are all YJK format data.
A = 1, Y data becomes color code, and the color palette is output as RGB. J and K are YJK
↪  format data.
```

YJK Format and Conversion to RGB (Reference)

```
R = Y + J

G = Y + K

B = 5/4Y - 1/2J - 1/4K

Y = 1/8B + 1/2R + 3/8G

J = R - Y

K = G - Y
```

(Editorial note) The value of Y is an integer from 0 to 31 if there are no attributes. If there are attributes, it is an even number from 0 to 30. The values of J and K are integers from -32 to 31. Conversion results from YJK to RGB are clipped to values 0 to 31.

”4.3 V9958 Abolished Functions

The following functions that existed in V9938 have been abolished in V9958.

- Composite video output
- Mouse/light pen interface

(Note from the editor) The mouse for MSX does not use the mouse interface function of V9938, so the removal of this function does not have an impact.”

Page 90 Chapter 4 V9958 VDP

4.4 V9958 Hardware Specifications (Changes)

Table 4.5: Changes in V9958 Terminals

No. | Name | I/O | Description | V9958 | V9938 | Name | I/O

```
4   | VRESET | I  | HSYNC/CSYNC 3-state logic input separation | VDS   | O |
5   | HSYNC  | O  | HSYNC output or burst flag output          | HSYNC | I/O
6   | CSYNC  | O  |                                              | CSYNC | I/O
8   | CPUCLK/VDS | O | CPUCLK output or VDS output              | CPUCLK | O |
24  | AVDD(DAC)   | O | Analog power supply                     | VIDEO  | O
26  | WAIT  | O/I | I/O WAIT output                               | LPS    | I
27  | HRESET | I  | HSYNC/CSYNC 3-state logic input separation    | LPD    | I
```

If the VDS flag of the control register 25-bit is 0, pin 8 outputs CPUCLK, if the VDS flag is 1, pin 8 outputs VDS.

Table 4.6: V9958 Current Characteristics

VRESET, HRESET Number | Item | Minimum | Standard | Maximum | Unit

```
VIL   | Low-level input voltage  | -0.3   |    0.8   |          | V
VIH   | High-level input voltage |  2.2   |     Vcc   |          | V
```

HSYNC, CSYNC, CPUCLK/VDS, WAIT Number | Item | Measurement Conditions | Minimum | Standard | Maximum | Unit

```
VOL   | Low-level output voltage | IOL = 1.6mA           |          |   0.4   |          | V
VOH   | High-level output voltage| IOH = 0.1mA          |  2.4     |          |          | V

                            G, R, B
```

Number | Item | Measurement Conditions | Minimum | Standard | Maximum | Unit

```
VRGB31  | Maximum output voltage | RL = 470Ω             |          |   2.8   |          | V
VRGB30  | Maximum output voltage | RL = 470Ω             |          |   2.0   |          | V
VP-P    | VRGB31-VRGB30          | RL = 470Ω             | 0.8      |         |          | V
DRGB   | VP-P Value              | RL = 470Ω             |          |          |    5 %   |
```

4.5 V9958 and MSX2+

The component controlling the screen display of the MSX is called the ”Video Display Processor”, abbreviated as VDP. The VDP of the MSX2 was a device called ”V9938”, but in machines from MSX2+ onwards, it has been upgraded to ”V9958”. Here, we introduce the functions added to this V9958.

4.5.1 There are 12 types of screen modes

Screen mode is the state of the screen that can be switched with the SCREEN command in BASIC. For example, if you want to write a BASIC program to display 40 columns horizontally, you would switch the screen to text (character display) mode with

```
SCREEN 0: WIDTH 40
```

and if you want to draw graphics, you would switch to graphics mode with

```
SCREEN 8.
```

When there are many screen modes, it means that you can, for the time being, choose the one that suits your needs. But there's a reason why there are many screen modes in MSX2+.

Number	Display Method	Resolution	Number of Colors
0	Character (1)	80 x 24 characters	Specify text color and background color
1	Character (2)	32 x 24 characters	Specify text color and background color
2	Table (3)	16 x 192 dots	2 colors per 16 horizontal dots
3	Bitmap (4)	64 x 48 dots	16 colors
4	Bitmap	256 x 192 dots	16 colors
5	Bitmap	256 x 212 dots	16 colors
6	Bitmap	512 x 212 dots	4 colors
7	Bitmap	512 x 212 dots	16 colors

Number	Display Method	Resolution	Number of Colors
8	Bitmap	256 x 212 dots	256 colors (Dedicated screen)
9	Bitmap	256 x 212 dots	16,369 colors (Dedicated screen)
10	YJK / RGB	256 x 212 dots	12,499 colors (Dedicated screen)
11	YJK / RGB	256 x 212 dots	12,499 colors (Dedicated screen)
12	YJK	256 x 212 dots	19,268 colors (Dedicated screen)

Notes:

1. Displays 1 character in 6 x 8 dots, can display either English letters or Katakana.
2. Displays 1 character in 8 x 8 dots, can display either English letters or Katakana.
3. Creates a screen display by combining patterns of 8 x 8 dots.
4. In bitmap display, the color of the dots can be displayed regardless of the color of the surrounding dots.

Chapter 4: V9958 VDP

The first issue is the problem of distance. Outputting text on the graphics mode screen takes time. Therefore, when entering a program, text mode, which does not use graphical representations, is advantageous.

The second reason is functionality. Numerous screens with lots of colors or potentially numerous display elements, for example, SCREEN 8's screen data, require up to 54,272 bytes, equivalent to 12 double-sided disks. In games utilizing ROM cartridges, color limitations and fewer data requirements result in faster processing. Using SCREEN 2 can conserve memory.

The third is the addition of features to maintain compatibility and coherence; in earlier MSX models, only four screens (from SCREEN 0 to SCREEN 3) were available. However, the MSX2 introduced SCREEN 4 and higher for high-resolution graphics, and MSX2+ added SCREEN 10 and above for increased colors.

Table 4.7 summarizes the color screens. Though actual data concludes vertically, newer modes with doubled vertical resolution added in the MSX2+ included the "kanji mode" for Chinese characters and others.

4.5.2 Controlling VDP Registers

When controlling MSX's screen display, the VDP, analogous to the CPU's heart, employs "registers." By sending signals to these VDP registers through I/O ports, the CPU can execute various functions. Main among these registers is the "Control Register," which allows the CPU to understand the state of the VDP, and the "Status Register," signifying the current VDP status to the CPU. There's another "Command Register" for the VDP's highly flexible instructions, though fundamental programs might not frequently use these.

Typically, the I/O port addresses to connect the CPU to the VDP extend from 98H to 9BH. Detailed in section 4.8, Table 4.8 itemizes usage for ports 6 and 7 in ROM:

```
Table 4.8: VDP I/O Ports

| Port Name | R/W | I/O Address | Usage |
| --------- | --- | ----------- | ----- |
| Port 0 | READ | ROM offset 6 | VRAM read |
| Port 1 | READ | ROM offset 6+1 | Status register |
| Port 2 | READ | ROM offset 6+2 | VRAM read |
| Port 3 | WRITE | ROM offset 7 | Control register |
| Port 4 | WRITE | ROM offset 7+1 | Palette register |
| Port 5 | WRITE | ROM offset 7+2 | Connector register |
```

This is determined. This may be due to the consideration of increasing the VDP in the MSX itself. Note that when writing and reading to the same port, the bank difference in the I/O port may be different.

Next, to set the value of the VDP control register, first write the data to the port 48 in Table 4.8, and then write the register number + 128 (to indicate continuation). It is important to satisfy the condition of ”continuation”. If an interruption occurs between the writes, the VDP will be confused. Therefore, in this case, it is commonly used to prohibit interruptions by “DI” command before writing a series of data.

By the way, once the control register value is written, it cannot be read. Therefore, if there's a case where you want to write only a specific bit of the register, you first read the value from RAM, modify only the necessary bits, and then write again to the control register. For detailed explanations, refer to Table 4.9 which shows how to write the value into the system work area (RAM). For example, if you modify the 4th bit of control register 1, read the content of address F3E0H of RAM and write it to address F3E0H after modification. This operation of writing control register 1 to RAM address F3E0H is also introduced as ”WR1WRVDP” in the sample program of list 4.9. This routine has been confirmed to operate correctly.

Next, concerning reading the value of the status register. Here, the value corresponding to the number read from status register 15 is used. The procedure prohibits interruption, reads the value of port 1 of the VDP, and then returns control register 15 to 0. This procedure is also introduced in the sample program of list 4.9 as ”VDPSTA”.

Table 4.9: Control Register Storage Locations

Control Register Number | VDP Related Number | Storage Address | Label

```
0 | : | : | F3DFH | RG0SAV
1 | : | : | :

:

7 | : | F3E6H | RG7SAV
8 | : | FF7EH | RG8SAV

:

23 | 24 | FFF6H | RG23SA
25 | FFFAH | RG25SA
26 | 27 | FFF8H | RG26SA
28 | FFFCH | RG27SA
```


Table 4.10: Other useful system work areas

Address	Label	Meaning
F341H	RAMAD0	Slot number of page 0 RAM
F342H	RAMAD1	Slot number of page 1 RAM
F343H	RAMAD2	Slot number of page 2 RAM
F344H	RAMAD3	Slot number of page 3 RAM
FAF5H	DPPAGE	Disk page number
FAF6H	ACPAGE	Active page number
FD9AH	H.KEY1	Interrupt hook
FD9FH	H.TIMI	Timer interrupt hook

- Only valid when a disk is available.

Table 4.11: System work areas added and modified in MSX2+

Address	Label	Meaning
0FAFCH	MODE	Next display mode
0FAFDH	NORUSE	Kanji driver work area
0FDOAH	SLTWRK+1	Kanji driver work area
0FD0FH	SLTWRK+6	Kanji driver work area
0FFFAH	RG25SA	VDP register save
0FFFBH	RG26SA	VDP register save
0FFFCH	RG27SA	VDP register save

Table 4.12: Details of address 0FAFCH (MODE)

Bit	Meaning
b7	1 if Katakana, 0 if Hiragana
b6	1 if external (waterproofing ROM present)
b5	1 if SCREEN 11, 0 if SCREEN 10
b4	Used internally
b3	
b2	Mask VRAM address to 3FFFFH for SCREEN 0~3 VRAM
b1	VRAM capacity 00: 16KB 01: 64KB 10: 128KB
b0	1 if Roman character conversion

By the way, the ROM of MSX2+ contains BIOS with similar functions to those of subroutines, but generally, BIOS is not used to create subroutines. However, these BIOS are ROM in MSX1, called by the processor as subroutines, so ...

There is concern that it might take some time. Therefore, it seems there are bad scenarios where video problems get mixed in, and BIOS could not be used. For tables 4.10 to 4.12, a summary of the system work area has been prepared, so please refer to it.

4.5.3 V9958 Registers

Fig. 4.3 shows the three control registers added to the V9958. These registers are intended for writing only, and the values written are recorded in the system work area of 4.9. Let's pay attention to the differences and similarities between system work area registers and BASIC or VDP related registers.

Most of the added features of the V9958 are controlled by control register 25. We will introduce the well-known YJK (natural screen) display and horizontal scroll later, but for now, let's introduce some of the remaining features.

In register 25, if bit 7 is written, the bit is called VDS and it controls 8 functions of the VDP. However, in usual programming, changing this bit is prohibited.

Moreover, if you write to a register simultaneously with the V9938, you can use the SCREEN5 to 8 functions, only if the values match. This means you can use VDP commands as well in these graphic modes.

Additionally, the VDP commands in these screen modes can replicate BASIC's COPY command functionality besides VDP features. In detail, VDP commands are designated to SCREEN 5 to 8 outside the parameters, and the bit values of 0 to 255 are determined by the XY coordinates of 128K video RAM positions.

Furthermore, if bit 1 is written in, the VDP's wait function gets enabled. This function involves sending a WAIT signal from VDP when the CPU reads video RAM and waits for the CPU's operation completion. However, high-speed transfers to the palette register, writing VDP commands, or copying cannot use the wait function.

4.5.4 Horizontal Scroll by VDP

The horizontal scroll has two methods: using one screen of image data or using two screens of image data. We will sequentially explain each method.

To start, control register 25, if bit 0 (SP2) is the case, then as shown in figure 4.4, data for 1 screen is horizontally scrolled. The number of scroll dots is specified in registers 26 and 27. Writing values from 0 to 63 in register 26 and that value in register 27 allows scrolling the screen horizontally by 8-dot units.

Page 96 Chapter 4: V9958 VDP

Fig. 4.3: List of functions of the control registers added to the V9958

Register 25

b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0 CMD | VDS | YAE | YJK | WTE | MSK | SP2

- 1: 2-page horizontal scroll
- 1: Mask left 8 dots
- 1: Wait function enabled
- 1: YJK mode
- 1: YJK/RGB mixed mode
- Changes pin 8 function
- 1: Write command in full-screen mode
- Always write 0

Register 26

b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0 0 | 0 | HO5 | HO4 | HO3

- Horizontal scroll dot count
- Always write 0

Register 27

b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0 0 | 0 | 0 | 0 | HO2 | HO1 | HO0

- Horizontal scroll dot count
- Always write 0

Writing a number from 0 to 7 moves the screen to the right by the specified number of dots. However, be careful that in SCREEN 6 and 7, the number of dots that scroll is twice the specified number. The scroll count increases from 0 to 255, and when it overflows, the scrolled part on the left side of the screen disappears and then reappears on the right side.

When the b7 bit of register 25 is set to 1, as shown in Fig. 4.4, the scrolled part of the image data is displayed, enabling the 2-page horizontal scroll. The image data at this time is stored in page 0 and 1 of the video RAM, or in pages 2 and 3.

Figure 4.4: Structure of 2 types of horizontal scrolling

When SP2 = 0

- Number of dots specified in register 26 and 27

```
- Image
- Image
- Left end
- Right end
- Content of VRAM
```

The number of dots specified in control registers 26 and 27 is increased, and the image displayed on the screen scrolls to the left, making the part that protrudes from the left edge appear from the right end. It's a situation where one screen is connected on the left and right.

When SP2 = 1

- Number of dots specified in register 26 and 27

```
- Image
- Image
- Left end
- Page 0 in VRAM
- Page 1 in VRAM
```

The video RAM's pages 0 and 1 (or 2 and 3) are combined to create two horizontal screen images, and only the specified parts are displayed alternately on the screen. The images move horizontally as if scrolling the screen, but the two images are connected and move.

When the page is 0 and 1, set display pages to 1 and 2, and when 2 and 3, set display pages to 3. The number of horizontal scroll dots ranges from 0 to 511 (which is twice the number of screen horizontal scrolls).

Also, if bit 1 of register 25 is set to 1, 8 dots at the edge of the screen (16 dots in SCREEN6) are invalid, and the border color of the screen will be displayed there. This horizontal scroll is especially useful when horizontally scrolling data on one screen, as the part that overflows from the screen appears immediately from the opposite side. Refer to the program example of horizontal scroll in listing 4.1 (which is twice the screen horizontal scroll) for reference.

LIST 4.1 (HSCROLL.BAS)

```
100 'hscroll.bas
110 'by nao-i on 26. Oct. 1989
120 '
130 ONSTOP GOSUB 290: STOP ON
140 DEFINT A-Z: SCREEN 5
150 COLOR 15,1,1: CLS
160 LINE (0,0)-(255,211)
170 SET PAGE 1,1: COLOR 15,8,1: CLS
180 LINE (0,0)-(255,211)
190 V6=VDP(26):VDP(26)=V6 OR 3
200 ' *** scroll ***
210 FOR V7=0 TO 63
220 GOSUB 320: VDP(27)=V7: VDP(28)=7
230 FOR V8=6 TO 0 STEP -1
240   GOSUB 320: VDP(28)=V8
250 NEXT V8
260 NEXT V7
270 GOTO 200
280 ' *** restore VDP ***
290 STOP OFF: VDP(26)=V6: SET PAGE 0,0
300 COLOR 15,4,7: SCREEN 0: END
310 ' *** wait next vsync ***
320 TIME=0
330 IF TIME=0 GOTO 330
340 IF (VDP(-2) AND 64)=0 GOTO 340
350 RETURN
```

4.5.5 Do not use backdoor methods regardless of the situation

There are bits in VDP's registers that can be written to normally, and some that are specified to be written as '1' forcibly. Ignoring these specifications and writing abnormal val-

ues, one cannot predict what kind of problems may occur, and it is absolutely not recommended to perform such actions.

Moreover, like the combination of YJK=0 and YAE=1, there are settings decided by VDP's specifications. If such specifications are not adhered to, unless you know well what you are doing, it could cause problems in the movement. Let's clarify: an MSX machine that runs correctly is different. Also, there is no guarantee that code for V9958 would work properly on a future VDP. It should be remembered that V9958 VDP made by some manufacturer might behave slightly differently if the method of taking advantage of 'backdoor techniques' is ignored. Currently, there are valid backdoor techniques for V9958, but these may not work in the future.

There was an example in the past where VDP of an MSX machine used Z80 backdoor techniques, causing software runaways especially on HC-90 compatible boards. This was a famous example where the VDP was run on an upgraded CPU (HD64180), leading to troubles that stemmed from ignoring specifications.

4.6 Deciphering the YJK Method

4.6.1 Television Broadcast and YJK Method

In the natural graphic mode of MSX2+, the traditional RGB method is replaced by the YJK method. This YJK method is similar to the method used in color television broadcast.

Many readers may not know this, but in the past, television broadcasts were black and white, and naturally television images were black and white signals. Later, when color broadcasts began, they faced the problem of how to handle this. One solution was to separate the screen into three channels: RGB (red, green, and blue), but this required a device capable of receiving color broadcasts.

So instead, a method leveraged the characteristics of human vision. Human vision perceives fine details from brightness levels but does not perceive fine details from color levels. Thus, using fewer channels for color and more for brightness, television broadcast could handle both black and white and color signals. This method combined UV signals representing ”hue” and ”chroma” with Y signals representing ”brightness,” [using YUV signals]. When the human eye perceives these signals, a normal color image is visible.

In this way, to increase the number of channels (waves) for television, the YUV method was utilized. On the other hand, on the MSX2+, the YJK method, similar to the YUV method, is used to increase the number of colors effectively using video RAM.

4.6.2 Data Structure of RGB Method and YJK Method RGB Method (SCREEN 8)

In SCREEN 8, brightness levels of R, G, and B are represented with 3, 3, and 2 bits respectively, displaying any of 256 colors for each dot.

Figure 4.5: RGB Method Screen Data Structure

b7 b6 b5 b4 b3 b2 b1 b0 G G R R R B B B

YJK Method (SCREEN 12)

In SCREEN 12, the YJK method that considers 4 horizontal dots as a unit is used. From Y0 to Y3, the brightness of each dot is indicated by 5 bits, and K and J are represented by 12-bit color hue data for all dots (4096 colors). This is how YJK data is converted to RGB format.

[R = Y + J] [G = Y + K] [B = 1.25Y - 0.5J - 0.25K]

However, the value of Y ranges from 0 to 31, and the values of J and K range from -32 to 31. If the RGB values calculated by the above formulas are smaller than 0 or larger than 31, they will be clipped to 0 and 31 respectively. This process is called ”clipping” to within 0 to 31.

For example, let's describe the 4-byte data where Y = 0, J = K = -32 representing black on the electric board.

[\begin{array}{cccccccc} b_7 & b_6 & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
\end{array}]

Conversely, let's consider the process of converting RGB data to YJK data. According to the formula, RGB can be converted to Y, J, and K through the following three formulas:

[Y = (2R + G + 4B) / 8] [J = (6R - G - 4B) / 8] [K = (-2R + TG - 4B) / 8]

When displaying the image using the YJK method, the signals output by the VDP are converted to composite (video) signals similar to the conventional analog RGB signals, so a special monitor TV is not necessary for MSX2. The YJK image is fundamentally similar to SCREEN 8, with 1 byte corresponding to 1 dot. However, since the values of J and K are specified for each group of horizontal dots, for instance, when executing PSET(0,0,0), the K value for dots from (0,0) to (3,0) will change. Therefore, using commands such as LINE in SCREEN 12 will cause ”color changes.” I will explain this again later.

4.6 Deciphering the YJK Method

Fig 4.6: YJK Method Screen Data Structure

![Data Table]

0	b7	b6	b5	b4	b3	b2	b1	b0
1	Y0			K_L				
2	Y1			K_H				
3	Y2			J_L				
4	Y3			J_H				

Assigning colors every 4 dots allows you to specify the luminance for each dot.

0 1 2 3 252 253 254 255 Y K_L Y K_H Y J_L Y J_H Y K_L Y K_H Y J_L Y J_H ... Y K_L Y K_H Y J_L Y J_H

Mixed Method (SCREEN 10, 11)

The drawback of the YJK method is that it can only specify colors every 4 horizontal dots, making it difficult to align characters or lines naturally. Therefore, SCREEN 10 and 11 which combine the advantages of RGB (SCREEN 5 mode) and YJK (SCREEN 12 mode) were introduced to address this. These screen modes naturally allow overlapping of characters with less color issues compared to SCREEN 12.

Fig 4.7: Mixed Method Screen Data Structure

0	b7	b6	b5	b4	b3	b2	b1	b0
1	Y0			A0	K_L			
2	Y1			A1	K_H			
3	Y2			A2	J_L			
4	Y3			A3	J_H			

In one dot, you can display in YJK or RGB mode. With the screening of VRAM's respective horizontal lines, by utilizing YJK method with the extended range (eight segments as one) of SCREEN 12, the mixed method is realized by the way of RGB mode allocation.

4.6.3 Color Slant Program

The advantage of the MSX2+ is its ability to display 19,268 colors on SCREEN 12, allowing for an expanded range (same 128kB as MSX2) of color expressions through a method called YJK.

As explained earlier, the method of expressing Y, which represents brightness (brightness), combined with color phases B and R values, is used to specify colors. List 4.2 is a program that displays all colors that can be expressed in YJK. However, to set the value in the later half of X, 32,768 times PSET is used, and it seems that sometimes an error occurs. Therefore, the PSET value on the left side of the X value is written with the PSET command, and the part corrected with the COPY command is copied using the COPY command.

List 4.2 (YJK1.BAS)

```
100 'screen 11 & 12 display by PSET
110 ' by nao-1 on 12, Nov. 1988
120 CALL KANJI0:WIDTH 64:DEFINT A-Z:YY=0
130 CLEAR:DEFINT A-Z:YY=0
140 ON STOP GOSUB 400:STOP ON
150 SCREEN 12:COLOR 4,#8,0,0:CALL CLS
160 'Set X in VRAM Y
170 FOR K=32 TO 31:YP=112+K+K
180 FOR J=32 TO 31:XP=128+J+4
190 PSET (XP,YP),(JX AND 7
210 PSET (XP+1,YP),(K AND 56)*#8
220 PSET (XP+1,YP+1),(K AND 56)*#8
230 PSET (XP+2,YP), (J AND 7
240 PSET (XP+2,YP+1), (J AND 7
250 PSET (XP+3,YP),(J AND 56)#8
260 PSET (XP+3,YP+1),(J AND 56)#8
270 NEXT:NEXT
280 'Change value of Y
290 SC=12:NS=1:SCREEN 12
300 FOR Y=1 TO 31:GOSUB 360:NEXT
310 FOR Y=30 TO 0 STEP -1:GOSUB 360:NEXT
320 SC=11:NS=1:SCREEN 11
330 FOR Y=2 TO 30 STEP 2:GOSUB 360:NEXT
340 FOR Y=28 TO 0 STEP -2:GOSUB 360:NEXT
350 GOTO 280
360 'Subroutine to write Y in VRAM
370 IF NS THEN LOCATE 1,0:PRINT USING'SCREE
```

NN #';SC:NS=0

```
380 LOCATE 1,0:1:PRINT USING'Y''s value (brightness) as "#'#;YY
390 LINE(0,48)-(255,175),(Y XOR YY)*#8,BF,XO
```

XX:YY=Y:RETURN

```
400 'Stop for on stop
410 SCREEN 0:COLOR 15,4,7:END
```

4.6 Decoding the YJK Method

List 4.3 (YJK2.BAS)

```
100 'screen 11 and 12 color sample
110 'by nao-i on 2. Nov. 1988
120 CALL KANJIO:WIDTH 64:DEFINT A-Z:YY=0
130 CLEAR:DEFINT A-Z:YY=0
```

```
140 ON STOP GOSUB 380:STOP ON
150 SCREEN 12:COLOR &HF8F,0,0:CALL CLS
160 ' Return J and K to VRAM
170 FOR K=32 TO 31:YP=112+K
180 PSET (0,YP),K AND 7
190 PSET (1,YP),K AND 7
200 PSET (1,YP),(K AND 56)*8
210 PSET (1,YP+1),(K AND 56)*8:NEXT
220 FOR J=32 TO 31:XP=128+J*4
230 LINE(XP+2,48)-(XP+2,175),J AND 7
240 LINE(XP+3,48)-(XP+3,175),(J AND 56)*8
250 COPY (0,48)-(-1,175)TO(XP,YO):NEXT
260 'Change color sample
270 SC=12:NS=1:SCREEN 12
280 FOR Y=1 TO 31:GOSUB 340:NEXT
290 FOR Y=30 TO 0 STEP -1:GOSUB 340:NEXT
300 SC=11:NS=-1:SCREEN 11
310 FOR Y=72 TO 30 STEP 2:GOSUB 340:NEXT
320 FOR Y=28 TO 0 STEP -1:GOSUB 340:NEXT
330 GOTO 260
340 'Sample routine to write to VRAM
350 IF NS THEN LOCATE 1,0:PRINT USING"SCREE NN ∞";SC:NS=0
360 LOCATE 1,1:PRINT USING"Y (coordinate)"Y
370 LINE(0,48)-(255,175),(Y XOR YY)*8,BF,X0
```

R:YY:Y=RETURN

```
380 on stop goto 390
390 SCREEN 0:COLOR 15,4,7:END
```

4.6.4 Required Logical Operations

Logical operations refer to logical arithmetic in Japanese, meaning calculations performed bit by bit in binary. These include AND, OR, XOR, and NOT, the four types of logical operations.

For example, X%=&B00000011 Y%=&B00000101 AND operation only processes the bits, so it only keeps the rightmost bit 1 in X% and Y%:

XX', AND YY'=&B00000001

That's what it means. Similarly, OR calculation is such that both values or one of the two values becomes 1, while in XOR calculation, the bit becomes 1 if one of the two values becomes 1. Also, NOT is a calculation that inverts each bit of a value, and writing image data inverted is called the PRESET calculation. To make this clear through words is difficult, so I have summarized the contents in Table 4.13:

Table 4.13: Logical Operations

Symbol	Meaning	Example
PSET	Write the specified color as it is	
AND	Write the logical AND of the original colors	0011 AND 0101 → 0001
OR	Write the logical OR of the original colors	0011 OR 0101 → 0111
XOR	Write the logical XOR of the original colors	0011 XOR 0101 → 0110
PRESET	Write the inverted specified color	NOT 0101 → 1010

Now, to handle the video RAM with MSX2 or MSX2+, you need to think in this way about logical operations. For example, in the program listed in 4.3, consider the value of line K in the video RAM, from 1 (binary 00001) to 2 (binary 00010),

Thus, Video RAM content: b7 b6 b5 b4 b3 b2 b1 b0 0 0 0 0 0 1 ? ? ?

becomes, b7 b6 b5 b4 b3 b2 b1 b0 0 0 0 1 ? ? ?

The values of J to K enter from bit 1, so this has to change. Appointing the two XOR values from Y0, each 8 times,

LINE(0, 48)–(255, 175), 24, XOR

becomes so, and each 1 write will change Y values from 1 to K. This corresponds to list 4.3 in 370 rows.

LINE(0, Y0)–(255, Y0 +127), (Y XOR YY)*8, BF, XOR

Meaning, Y represents the destination Y value, while YY keeps the original Y value. When specifying a logical operation (bitwise operation) with the LINE command, the result of writing the two values calculated will be written to video RAM. If it is SCREEN 12,

4.6 Controlling the YJK Method

LINE (0,0)-(255,211), &B11111100, BF, AND

When executed, bits 7 to 3 of the video RAM remain unchanged while bits 2 to 0 become 0, resulting in the entire display showing shades of black.

4.6.5 So-called Coloring

The data structure of the YJK screen is summarized in Figure 4.6. Similar to SCREEN 8, generally, each dot in the screen corresponds to one byte of video RAM. For a screen with a resolution of 256×212 dots, 54,272 bytes of video RAM are used for displaying. However, when specifying colors, values for YJK are defined per every horizontal 4 dots; for instance, in SCREEN 12:

PSET (3,0),3

When executed, not only the single dot at (3,0) but also the J values for four dots from (0,0) to (3,0) will change, causing overlaps. After copying the content of ”sample.s12” to a file on a SCREEN 12 screen and then executing list 4.4, the ”coloring” will occur. Let’s test it out. Originally, SCREEN 12 could not display text lines.

List 4.4 (S12.BAS)

```
10 ' Example of coloring
20 CALL KANJI
30 SCREEN 12:COLOR &HF9,2
40 BLOAD"sample.s12",S
50 LINE (0,0)-(211,211),3
60 LOCATE 4,6:COLOR &HF9,2
70 PRINT "COLORING occurs in SCREEN 12."
80 GOTO 80
```

4.6.6 What is the Difference Between SCREEN 10 and 11?

The data structure of SCREEN 10 and 11 is summarized in Figure 4.7; in terms of functionality, both are identical. When displaying a BLOAD screen, the differences in the hardware used are not apparent. So, what's the difference? When writing text or characters to the screen using BASIC commands, problems arise. For example, a program to display the differences using List 4.5, where a screen saved using the YJK method is loaded:

CIRCLE (128,106),100,6

Let's give it a try. In SCREEN 10, bit 3 of the video RAM is set to 1, causing the red color number specified by the circle command to be written (in binary, 0110). Thanks to the YJK palette background, a red circle can be drawn. However, in SCREEN 11, since the

video RAM connects directly (i.e., follows an 8-bit pattern like 00000110), a ”color shift” occurs.

As shown in the example, drawing characters or shapes on the YJK screen by stacking them requires the use of SCREEN 10. To process YJK screen data, SCREEN 11 is needed.

List 4.5 (S10.BAS)

```
100 REM * To avoid color shift *
110 SCREEN 12
120 BLOAD "sample.s12",S
130 SCREEN 10
140 CIRCLE (128,106),100,6
150 FOR I=1 TO 50:BEEP:NEXT
160 SCREEN 12
170 BLOAD "sample.s12",S
180 SCREEN 11
190 CIRCLE (128,106),100,6
200 GOTO 200
```

4.6.7 Using text even in SCREEN 11

List 4.6 is a program that writes text on the SCREEN 11 screen. Although it was mentioned previously that SCREEN 10 should be used for displaying text, depending on the PRINT command's characteristics, SCREEN 11 may also be useful at times.

For example, using the LINE or PUT KANJI command on SCREEN 10 causes YJK screen RGB characters to be printed. However, using the PRINT command on that background causes a box frame to appear, resulting in the printed characters being cut off. In such cases, it is better to use it directly as text.

What you can do is switch to SCREEN 11, use the SET PAGE command to set video RAM to page 1, and BLOAD background screen data there. Then, after returning to that page, switch background color to 0 using the PRINT command to write text on the background. Also, for characters, use the COPY command with ”TAND” specified to write on overlapping parts of page 1’s image data.

TAND allows performing overlapping operations on the transparent (non-colored) parts where colors match. Therefore, text written in one color is ignored as if written in white and displayed correctly.

4.6 Decoding the YJK Method

Next, write the same character on page 0 in the color specified by (Cx16+8). This value is the number entered at the beginning of the program.

As shown in the figure, the bits from bit 7 to bit 4 are color numbers, and bit 3 specifies RGB display; the remaining bits from bit 2 to bit 0 become the TOR. When this character is specified as TOR, the contents of the video RAM for the character part are as follows:

This means displaying characters in RGB mode without breaking the YJK screen. This fills in the point of the logical operation feature that was lacking when copying characters with the COPY command to the pen location.

It may be difficult to understand YJK method in an MSX2 computer, but when using this method, you can achieve excellent effects. Let's all do our best to study and research this.

List 4.6 (S11.BAS)

```
100 ' SCREEN 11 Opening
110 ' by nao-i on 4. Nov. 1988
120 SCREEN 0: WIDTH 32: CALL KANJIO
130 DEFINT A-Z: ON STOP GOSUB 300: STOP ON
```

```
140 FILES: PRINT: INPUT "file name"; FF$
150 OPEN FF$ FOR INPUT AS #1: CLOSE #1
160 INPUT "string"; SS$
170 INPUT "color(1..15)"; C
180 INPUT "X(0..240)"; X
190 INPUT "Y(0..196)"; Y
200 SCREEN 12: SET PAGE 1,1: BLOAD FF$, S
210 SCREEN 11
220 SET PAGE 0,0: COLOR 7,0: CALL CLS
230 L = LEN(SS$)-1
240 LOCATE 0, 0: PRINT SS$
250 COPY (0, 0)-(L, 15), 0 TO (X, Y), 1, TAND
260 LOCATE 0, 0: COLOR C x 16 + 8: PRINT SS$
270 COPY (0, 0)-(L, 15), 0 TO (X, Y), 1, TOR
280 SET PAGE 1, 1
290 GOTO 290
300 '* * * called by STOP * * *
310 SET PAGE 0, 0: COLOR 15, 4, 7
```

4.6.8 The Trick to Display Text on SCREEN 12

In SCREEN 12 in YJK mode, it is inherently impossible to display text or lines separately. However, by utilizing logical operations, we will introduce a trick to display text. It's Program 4.7. The structure of the program itself is almost the same as List 4.6. Please note, though: COLOR &HF8, 0 and COPY ..., TOR are the two different parts. With this, you can set the Y value of the part where the text is displayed to a maximum of 31, and display text in a nearly white color on a black background. Conversely, if you set the text color to a logical AND, the text will be displayed in black. When you run the program, a list of disk files will first be displayed, so specify the image file that you will use as the background. Next, write the log you want to display on the screen in position coordinates. With this, the log will be displayed on the SCREEN 12 screen. Let's modify the program to have a lot of fun with logs.

List 4.7 (S12T.BAS)

```
100 ' SCREEN 12 Log
110 ' by nao-i on 4. Nov. 1988
120 SCREEN 0:WIDTH 32:CALL KANJIO
130 DEFINT A-Z:ON STOP GOSUB260:STOP ON
140 FILES:PRINT:INPUT "file name:",FF$
150 OPEN FF$ FOR INPUT AS #1:CLOSE #1
160 INPUT "string",SS$
170 INPUT "X(0..240)",X
180 INPUT "Y(0..196)",Y
190 SCREEN 12:SET PAGE 1,1:BLOAD FF$,S
200 SET PAGE 0,0:COLOR &HF8,0:CALL CLS
210 L=LEN(SS$)*8-1
220 LOCATE 0,0:PRINT SS$
230 COPY (0,0)-(L,15),0 TO (X,Y),1,TOR
240 SET PAGE 1,1
250 GOTO 260
260 ' called by STOP ***
270 SET PAGE 0,0:COLOR 15,4,7
```

4.6.9 YJK Mode VDP Registers

Instead of displaying in YJK mode and in BASIC on SCREEN 10-12, let's introduce a method to manipulate VDP registers and execute lines with machine language programs. Control Registers...

4.6. Decoding the YJK Method

With bit 3 ”YJK” of Register 25 and bit 4 ”YAE”, you can select between screen display by

RGB method and YJK method.

First, let's set registers other than register 25 similar to the case of SCREEN 8. In BASIC language, by specifying screen modes 10 to 12, you can choose the YJK method. However, according to the functions of VDP, it is easier to understand if you think that SCREEN 8 screen is set to RGB method and is ”changed later”. If bit 3 of YJK is 0 and bit 4 of YAE is also 0, the settings of SCREEN 8 are determined. Then, with other registers as they are, changing bit 3 of YJK to 1 sets the display setting to YJK on SCREEN 12.

To display the mixed method screen of YJK and RGB on SCREEN 10 or 11, set YJK to 1 and YAE to 1. Note that in the case of YJK being 0, the sprite color does not change due to the impact of the palette, but it can be changed to a palette if YJK is 1.

4.7 Examining Raster Lines

4.7.1 What is the Mechanism for Displaying Monitor Screens?

You know that the SCREEN5 screen of the MSX2 is expressed by a collection of 256 x 212 dots (pixels). Figure 4.8 illustrates how this is actually represented on a monitor. By sending pixel data sequentially from left to right, and line by line from top to bottom, it effectively displays the screen. This series of 256 pixel collections is called 'raster lines'. In short, the horizontal collection of 256 pixels makes up a raster line, and the collection of 212 such raster lines vertically makes up the screen.

Figure 4.8: Example of Raster Lines on a TV Screen

Not only for computer screens, but ordinary television broadcasts use this mechanism for displaying images. The 'NTSC' system used in Japan and America employs a display method consisting of 525 raster lines. However, the actual visible lines in the image are roughly 490. The remaining raster lines are used for control signals known as 'sync signals', for overlaying text and broadcast information. Thirty such screens are displayed per second, amounting to 525 x 30 = 15750 raster lines per second on a single screen. The broadcasting system and the MSX's screen composition both involve this principle, with broadcast frequency being 30Hz and horizontal raster frequency at 15.75kHz.

Modern 16-bit computers often have higher screen resolutions, such as 640 x 400 dots, due to the need to display more dots and require monitors with a horizontal raster frequency of 24kHz or higher. Also, the latest computers have finer screen displays.

Non-interlaced display method. Raster line gaps can be clearly seen.

4.7 Researching Horizontal Scanning Frequency As the horizontal scanning frequency increases, monitors with frequencies such as 31.5 kHz and 34 kHz are used. Here, a monitor that accommodates various horizontal scanning frequencies on a single screen is called a ”multi-scan monitor.” If it accommodates frequencies from 15.75 kHz to 34 kHz, or if it can switch between two types such as 15.75 kHz and 24 kHz, there are various models. Therefore, when buying a new monitor, it's better to choose a multi-scan monitor that corresponds to the horizontal scanning frequency of the computer you currently have if you don't plan to buy a new one.

For reference, it's worth noting that in Western Europe, formats like ”PAL” and in France and the Soviet Union they use ”SECAM”, and monitors from these regions can accommodate different scanning frequencies from NTSC. For this reason, European computer graphic boards cannot display images directly when connected to Japanese monitors. Video projectors marketed in Japan are often compatible with NTSC, PAL, and SECAM, but tests for specific uses with output image quality from the video projector would require special properties, so they are priced quite high (recently, a worldwide video deck capable of replaying videos from different regions was released by Sony).

Also, it's necessary to consider the resolution and video performance displayed by VTRs and video discs. ”Vertical resolution” refers to the number of scanning lines. For monitors, the finer the number of dots (dots per inch) in the screen's horizontal direction, the higher the detail display. High-definition images refer to the higher resolution number of lines. Even for VTRs, ”ED Beta” and ”SVHS” show better vertical resolution than conventional VTRs.

4.7.2 Television Broadcasting Using the Interlace Method When displaying computer screens, the composition resembles Figure 4.8, but in the case of television broadcasting, things are slightly different from what we've discussed earlier. First, please look at Figure 4.9.

This method is known as ”interlace,” a way of describing screen display methods. In fact, this is widely used in television broadcasting. First, the odd-numbered scanning lines of the screen are displayed, such as lines 1, 3, 5, etc., as shown in Figure 4.8. After that, the even-numbered scanning lines like 2, 4, 6, etc. are displayed. In the current TV system, scanning lines are divided to display 212, 213, 214 lines in sequence. By displaying in this ”interlace” way with reduced visible scanning lines in every other execution, extra flicker due to video line gaps is significantly reduced. As shown in Figure 4.9, this even and odd line-by-line display method known as the ”interlace system” is applied.

In conclusion, considering the nature of the television broadcast adopting this interlace way, the discussion is essential regarding how television broadcasting adopts the interlace system.

”Figure 4.9: This is what happens in interlace mode

```
212   - 212
213   - 213
```

- -

2 - 1

- -

```
210  - 210
423  - 423
424  - 424
```

This is the interlace

- Two fields display method. The line spacing becomes visible.

To reduce the screen disturbance caused by noise of various origins, which occurs due to factors such as the flickering of the screen, the interlace display method is useful. For example, if the 0th and 1st scanning lines flicker a lot, and the 212th scanning lines, which are displayed immediately after them, flicker, the screen disturbance becomes noticeable. Noise generated when radio waves fly through the air, noise generated during the chemical manufacturing process, noise generated at the instant of connection to power outlets, and believe it or not, noise generated from disturbance of TV broadcasts can easily affect it. This interlace mode is a useful method for reducing such noise effects.

However, the downside to this interlace method is that the positions of the adjacent scanning lines can appear slightly shifted, making the screen seem shaky. More than in broadcast TV, when fine text is displayed in a still condition on computer screens, the shaking can be quite noticeable and easily catch the eyes of the user. As a result, many computers use the non-interlace mode for displaying screens as a common practice.

4.7.3 Interlace Screens on MSX2

The reason why many computers use ”normal” (non-interlace) screens is that MSX2 can use interlace screens... The aim here is for MSX2 and its successors to adopt the interlace method. For MSX2 to display vertical 424 dot screens, it needs a display monitor dedicated for computers (referred to as computer-only monitors unlike general household TV-called TV monitors, intended for vertical 424-dot display). As MSX2 advanced, along with multi-threaded horizontal synchronizing frequencies greater than the existing 15.75kHz or the newer 24kHz monitors that cost over 100,000 yen, it has become the norm. Therefore, with MSX2, combined with multiple...”

4.7 Investigating the Principles of Scan Line Interruption 113

As for the monitor, typically domestic TVs with a horizontal scanning frequency of 15.75 kHz low-resolution monitors were mainstream. For example, in the case of MSX2+,

CALL KANJI3

By executing this command, one could switch the screen to interlace mode, enabling 24-line kanji display. However, as mentioned earlier, as kanji display in interlaced mode is event-driven and consumes large memory bloats, one often feels it occasionally becomes slow.

The second purpose is for seamless image continuity, connecting the MSX2 to a TV screen as closely as possible, making ”superimpose” and ”video digitization” feasible. The horizontal scanning frequency of 15.75 kHz, being the same as the interlace method, superimposes the computer screen on the MSX2 screen atop TV broadcasts or video camera images, and video digitization becomes possible. Computers like the FS-5500 with built-in such functionality or attaching hardware like the VISC to the HC-95 and copying picture data for processing show a significant progress trend in this area.

The third point about the interlace method is it easily shows the gaps when printed or photographed. That is, the interlace screen has highly distinguishable scan line gaps, known as ”flicker.” However, interlace screens prefer the characteristic of line gaps without flicker.

On MSX2 and later machines, one can switch the screen to interlace mode by command. However, the kanji mode specification applies only to MSX2+.

In the case of an MSX2 machine SCREEN „„1 For MSX2+ machines and beyond CALL KANJI3

4.7.4 Investigating the Principles of Scan Line Interruption

”Interruption” is to change the flow of the program by external conditions. For example, when a joystick button is pressed, the BASIC program flow is changed by the command ”ON STRIG GOSUB,” which is one type of such interruption handling.

”Scan line interruption” is the same principle – the processing changes by specifying the scan line number where the display ends. This requires high-speed processing for scan line interruption handling, fitting for machine-language programs rather than BASIC programs.

Page 114

Chapter 4 V9958 VDP

Here, we will briefly explain the basic concept of interrupt processing. First, let's consider a VDP that controls the screen display, and, under certain conditions, can send an interrupt signal to the CPU. There are three types of interrupts: ”vertical synchronization,” ”horizontal synchronization,” and ”scanline.” However, on the MSX2, the scanline

interrupt is not used.

First, "vertical synchronization interrupt" occurs when the display of the bottom of the screen is completed, and the display of the top of the screen is to begin. It is an interrupt that occurs 1/60 of a second. This is often used to adjust the timing of music playback in games. In the MSX, the "timer interrupt" is essentially the same thing. The problem arises with a specific horizontal synchronization when the display ends, called the scanline interrupt. This interrupt occurs every 1/60 of a second.

Figure 4.10: The principle of scanline interrupt

Page 0: Interrupt occurs, triggering the scanline

Page 1: First, display here→

Next, display here→

By combining vertical synchronization interrupts and scanline interrupts, it is possible to display arbitrary scanlines at the top and bottom of the screen while maintaining interrupts. Thus, the upper part can be divided into vertical synchronization interrupts, the lower part into scanline interrupts. In other words, the screen can be divided into two parts.

On MSX2, multiple screens can be used. With 128 kilobytes of VRAM, SCREEN 5 retains 4 screens, SCREEN 7 and 8 contain 2 screens each. By using the BASIC "SET PAGE" command to switch, it is possible to immediately switch between multiple screens. Therefore, the scanline interrupt utilized since the early days of computers can be used in various ways. By creating a sprite interrupt processing program and switching pages with a scanline interrupt, it is possible to display different screens in the upper and lower parts of the screen. By combining with VDP scroll functions, it is also possible to scroll one part of the screen to the left or right, while drawing another part.

4.7.5 Introducing an example of scanline interrupt

To be honest, I first saw the VDP specifications and thought, "What would the scanline interrupt feature be useful for?" Although it was introduced in references such as the "MSX2 Technical Handbook," showing there was a scanline interrupt...

4.7 Researching Raster Split Insertion

Figure 4.11: Procedure for Raster Split Insertion

1. Prepare screen data for pages 0 and 1
2. Specify which raster to initiate the split
3. Display the page 0 screen data with the upper half split
4. Display the page 1 screen data with raster split insertion

By preparing the screen data for pages 0 and 1, and specifying the raster for the split insertion, the display can be switched between them.

There were no concrete program examples or applications published. The case where this was actually used in game software, released shortly after the compiler's development, was a game called "Zaxxon EX." One of the expanded functions in MSX2 is the "hardware horizontal scroll." This function was very useful for creating shooting games, displaying the upper part of the screen with fixed scores while scrolling the lower high-speed section of the screen. In "Zaxxon EX," they displayed the fixed upper part scores while performing high-speed scrolling. At that time, the MSX MAG editor had talked about this hardware horizontal scrolling in "Gradius."

So, once noticed, one might consider the horizontal scrolling with ”Gradius” as having key differences. Hereafter, shooting games using raster split insertion began to be developed one after another. Raster split insertion was used in game software to stop the pause display, and then suddenly, the game's two screens would be displayed. Let's actually try it.

Additionally, in MSX2+, the combination of raster split insertion and ”hardware horizontal scrolling” is being used to display only part of the screen with horizontal scrolling. This is advertised with the ”F1 Spirit 3D Special” game, which scrolls only the game screen horizontally while the F1 machine-like cockpit is displayed at the bottom of the screen with raster split insertion.

(Page 116) Chapter 4: V9958 VDP

4.7.6 Let's move on to the practical section!

Let's introduce a program example that uses interrupt handling in practice. The details will be explained later, but this program is made in assembler. However, it can be called from the BASIC language, so it can be used flexibly in your own games and other applications. In addition, source code is also made public, so if you understand the assembler, please try to analyze it.

4.7.7 VDP Registers Used in Interrupt Handling

The procedure for initiating a line interrupt is as follows. First, write the line number of the interrupt you want to generate into control register 19 and set bit 4 of control register 0. Then, when the display of the specified line is finished, the VDP will send an interrupt to the CPU. Additionally, if an interrupt occurs, the bit in status register 1 will be set, so you can check whether the interrupt was caused by a line interrupt. The summarized procedures are shown in Figures 4.12 and 4.13.

(Figure 4.12: VDP Registers Generating Line Interrupts)

VDP Control Register 0

b7 b6 b5 b4 b3 b2 b1 b0 0 DG IE2 IE1 M5 M4 M3 0

- IE1: Enable line interrupt
- M5, M4, M3: Screen mode

VDP Control Register 19

b7 b6 b5 b4 b3 b2 b1 b0 IL7 IL6 IL5 IL4 IL3 IL2 IL1 IL0

- IL: Line interrupt number

*Feature on V9938 only, always write 0 on V9958.

4.7 Investigate Horizontal Interruptions

Figure 4.13: VDP Register for Detecting Horizontal Interruptions

VDP Status Register - 1

```
| by | bs | b6 | b5 | b4 | b3 | b2 | b1 | b0  |
| FL | LP | I5 | I4 | I3 | I2 | I1 | ID | FH  |
```

- Detects horizontal interruptions
- VDP version number
- Used in Light Pen *

- Function of V9938 only, V9958 is irrelevant.

Now, we have explained the registers directly related to horizontal interruptions as described above. Therefore, let's use these registers to handle page switching and hardware vertical scrolling on the screen.

In SCREEN 5 and SCREEN 6, write the page number to bits 0 to 5 of control register 2. Like the BASIC "SET PAGE" command, you can switch the display page (the screen displayed) (see Figure 4.14). bit 7 should always be 0 and bit 4 for every others should be written 1. Similarly, in SCREEN 7 and SCREEN 8, specify the page in bit 5 and always write 0 in bit 6.

Figure 4.14: VDP Register for Controlling Page Switching

VDP Control Register - 2

```
| b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0      |
|  0 | A16 | A15 |  1  |  1  |  1  |  1  |  1  |
|                                          |
```

- Always write 1
- Display page
- Always write 0

VDP Control Register - 23

```
| b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0      |
| DO7 | DO6 | DO5 | DO4 | DO3 | DO2 | DO1 | DO0 |
|                                          |
```

- Display offset

Fig. 4.15: Mechanism of hardware vertical scrolling

When Control Register 23 is 0:

```
 VRAM Address           Display Position
      0                       0
   2A00H                    212
   4000H                    Display
   6A00H                     0
   7FFHH                    255
 Non-display
```

When Control Register 23 is 128:

```
 VRAM Address           Display Position
      0                       128
   2A00H                    212
   4000H                    Non-display
   7FFHH                    128
   (Text in top square) Display
```

By writing to it, page swapping becomes possible. Additionally, hardware vertical scrolling can be performed by manipulating Control Register 23. Specifically, as shown in Fig. 4.15, the screen display position changes based on the values set in the register, and a display called vertical scrolling is achieved. However, if this hardware vertical scrolling method is used, there is a possibility of splitting running numbers, necessitating corrections such as adding or subtracting the number of scroll steps from the running numbers. The sample program illustrates this correction starting from "ON_VSYNC".

4.7.8 How to Assemble and the Operation of the BASIC Part

The sample program to invoke line scanning is composed of parts created in Assembly and parts created in BASIC. We will first explain the assembly method and operation principles, followed by an introduction to the BASIC part.

The assembler program (source list) itself is listed in version 4.9 below. First, input the ”ONSCAN.Z80” file into the MSX-DOS screen editor and save it. Next, use a program called ”M80.COM” and ”L80.COM” found in ”MSX-DOS TOOLS” and follow the steps to assemble and link it. Once you obtain the machine file ”ONSCAN.BIN,” the process is complete. Note that the part ”DEL ONSCAN.BIN” is to delete old files when re-assembling, hence it is unnecessary the first time.

M80 =ONSCAN.Z80/R/Z L80 ONSCAN,ONSCAN/N/E DEL ONSCAN.BIN REN ONSCAN.COM ONSCAN.BIN

Next, let’s explain the operating principles (refer to section 4.8) of the BASIC program that controls the machine file. First, lines 190 to 200 are for initializing page 0 of the video RAM. If you need to scroll the screen vertically, you should initialize the entire page 0. “CLS” command alone cannot initialize only the portions displayed on the screen. “COPY” command replicates the page 0 content between (0,0)-(255,127) and (0,128)-(255,255) to initialize the entire page 0. Additionally, lines 210, 240, 250 display respective test patterns.

The BASIC program calls the assembly language program using the “USR function”. Lines 260 specify the accumulated line number to initiate and display page 0 on the left half, and page 1 on the right half of the screen.

USR(256+Accumulated Line Number)

Accordingly, changing the accumulated line number will change where the line scan initiates. Furthermore, you can scroll the top half of the displayed page 0 content vertically by line 290.

USR(512+Accumulated Line Number)

which indicates the value in this format. Additionally, although not used this time,

USR(768 + line number)

If specified, the part of Page 1 displayed on the lower part of the screen can be scrolled vertically. To stop line boundary scrolling,

USR(0)

should be executed. Note that in this program, the parameters for the USR function are constrained to integers to simplify handling in the assembler part. Hence,

USR(868.0)

or

USR(100+A!)

parameters with real numbers as above cannot be used.

List 4.8 (ONSCAN.BAS)

```
100 '
110 ' onscan1.bas : test interrupt on scan
120 ' by nao-i on 24. Sep. 1989
130 '
140 CLEAR 300,&HAFFF
150 DEFINT A-Z
160 OPEN "GRP:" FOR OUTPUT AS #1
170 BLOAD "onscan.bin"
180 SCREEN 5
190 SET PAGE 0,0: COLOR 15, 6,1: CLS
200 COPY (0,0)-(255,127) TO (0,128)
210 CIRCLE (128,106),80: PAINT (128,106)
```

```
220 SET PAGE 1,1: COLOR 15, 1,1: CLS
230 DEFUSR0=&H8000
240 PSET (8,192),0,PRESET
250 PRINT #1,"This area will not scroll."
260 JK=USR(256+100)
270 FOR J=1 TO 5
280     FOR I=0 TO 255
290         JK=USR(512+I)
300     NEXT I
310 NEXT J
320 JK=USR(0): COLOR 15,4,4: SCREEN 0
330 END
```

4.7.9 The Operating Principle of the Assembler Section

Now, let's use Listing 4.9 to explain the operating principle of the main program that actually handles the interrupt splitting.

First, the three lines from ”ASEG” are to create an object by using M80.COM or L80.COM as BASIC subroutines. M80.COM and L80.COM generate files from programs written in assembler. The extension of files created this way is ”.COM”. In other words, they become machine files that can be executed on MSX-DOS. Therefore, it is necessary to use the ”ASEG” command if you want to create files that can be handled in BASIC.

Also, a header for machine programs that can be loaded using BASIC's BLOAD command exists, which includes the following data:

FEH

- Load Start Address
- End Address
- Execution Start Address

These details are specified in the four lines following ASEG.

Next, let's explain the core of the program following ”AD_LOAD:”. First, it verifies the contents of the HL register pair and the parameters for the number of bytes of the USR-related. Additionally, the values of the input parameters are stored in HL+2 and HL+3 registers. Once confirmed, it ends by setting the vertical scroll of page 1 on the screen.

When an actual interrupt splitting occurs, parts of FD9AH (HOOKDT) are called. The five lines below are written as follows:

RST 30H DB Slot Number DW Address RET

By writing these five lines, you can call and execute the specified program when an interrupt splitting occurs. The place where these parts are executed under certain conditions is called a ”hook”. Here, we are preparing to have the interrupt called by ”ON_H.KEY:” command.

FD9FH is a timer interrupt call (ON_H.TIMI:), which is hierarchical. Here, the call section is invoked when the interrupt occurs by VDP status.

The value of 0 is stored in the A register, so you should not overwrite the AF register. Also, if you overwrite the stack, save the original value in a temporary buffer like the sample program, and after the interrupt processing is completed, restore the buffer.

When an interrupt occurs, a program that operates the VDP is called, and appropriate subroutines such as ”ON_VSYNC.” and ”ON_SCAN” are executed. By changing these, it could be used for the purposes of executing interrupt processing at the same timing as the horizontal retrace interrupt (HR interrupt).

The subroutine ”VDPSTA” reads the status register of the VDP. The subroutine ”WRTVDP” writes a specified value to the control register of the VDP, storing the value in the respective storage location (refer to Table 4.9 for details). Moreover, the ROM included with the MSX2 and later versions has a BIOS function that performs the same functions as these subroutines, so usually, you don't need to create subroutines yourself if you use the BIOS. But, since the BIOS is in the ROM, calling the BIOS takes several milliseconds, which could be an issue. Therefore, in the interrupt processing as this time, the BIOS was not used due to poor timing.

This isn't particularly related to BASIC programming subroutines, but in DOS programming, the interrupt processing program needs to be placed in a segment larger than 4000H. Below this, specific slot structures on MSX might cause issues.

List 4.9 (ONSCAN.Z80)

```
; onscan.z80 : test program for interrupt on scan
;            by nao-i on 26. Sep. 1989
;
; called as USR function from BASIC
;  USR(&H00xx)   restore registers and hooks
;  USR(&H01xx)   set interrupt line
;  USR(&H02xx)   set display offset line of page 0
;  USR(&H03xx)   set display offset line of page 1
;
        .Z80
START   EQU   0B000H   ; address to load and execute
USR_SUB EQU   0
USR_WRTVDP EQU 0
;
        IF    USE_WRTVDP
WRTVDP  EQU   0047H
        ENDIF
EXTROM  EQU   015FH
VDPSTA  EQU   0131H
SETPAG  EQU   013DH
;
RAMAD2  EQU   0F343H   ; slot of RAM in page 2
RAMAD3  EQU   0F344H   ; slot of RAM in page 3
RG0SAV  EQU   0F3DFH
DPPAGE  EQU   0FAF5H
ACPAGE  EQU   0FAF6H
H.KEYI  EQU   0FD9AH
H.TIMI  EQU   0FD9FH
RGBSAV  EQU   0FFE7H
;
        ASEG
        ORG   100H     ; to make .COM file
        .PHASE START-7
;
        DB    0FEH     ; header to BLOAD
        DW    AD_LOAD  ; address to load
        DW    AD_NEXT-1 ; address of end of file
        DW    DO_NOTHING ; address to execute
;
AD_LOAD:
        PUSH  AF
        PUSH  HL
        PUSH  DE
        PUSH  BC
        CP    2
        JR    NZ,RESET_SCAN  ; parameter is not integer
```

```
        INC   HL
        INC   HL
        LD    E,(HL)
        INC   HL
        LD    D,(HL)   ; DE = parameter of USR()
```

124 Chapter 4 V9958 VDP

```
        LD    A,D
        OR    A
        JR    Z,RESET_SCAN
        DEC   A
        JR    Z,SET_SCAN
        DEC   A
        JR    Z,SET_D0
        DEC   A
        JR    Z,SET_D1
        JR    RESET_SCAN
;
SET_SCAN:
        LD    A,E
        LD    (ILSAV),A
        CALL  SET_ILREG    ; set interrupt line
        LD    A,(HOOKED)
        OR    A
        JR    NZ,RET_BASIC ; hook is already set
;
        LD    HL,H.KEYI
        LD    DE,HOOKSA
        LD    BC,10
        LDIR               ; save hooks
        LD    HL,HOOKDT
        LD    DE,H.KEYI
        LD    BC,10
        DI
        LDIR               ; set hooks
        LD    A,(RAMAD2)
        LD    (H.KEYI+1),A
        LD    (H.TIMI+1),A
        LD    A,(RG0SAV)
        OR    00010000B
        LD    B,A
        LD    C,0
        CALL  WRTVDP       ; interrupt on
        LD    A,1
        LD    (HOOKED),A
        JR    RET_BASIC
;
RESET_SCAN:
        CALL  RESET_SCAN_SUB
        JR    RET_BASIC
;
```

SET_D0: ; set display offset of page 0

```
        LD    A,E
        LD    (D0VAL),A
        JR    RET_BASIC
```

SET_D1: ; set display offset of page 1

```
        LD    A,E
        LD    (D1VAL),A
;
RET_BASIC:
        EI
```

```
        POP   BC
```

”4.7 Studying the Run-Line Overlay (RLO) Design”

The rest of the content appears to be assembly code with comments in English.

130 Chapter 4 V9958 VDP

```
        DW      DR_8.KEY1
        RET
        RST     30H
        DB      0
        DB      4
        DW      DR_8.TIM1
        RET
;
;       .data area
;
```

ROUNT: DB 0 ; non-zero if hooks have been set SAVE0: DB 10 ; save area for hooks DSPAL: DB 3 ; display offset of page 3 DSPA2: DB 0 ; display offset of page 2 INTSTAT: DB 0 ; interrupt line

```
;
;       .VDPSTA : read a VDP status register
;       Entry  a   : VDP status register number
;       Return a   : value of the status register
;       Modify AF, BC
;       None   : compatible with ROM-BIOS
;                 32 oban return
;
;.VDPSTA
        IF      USE_SUB
        LD      IX,VDPSTA
        JP      EXTROM
        ELSE
        DI
        AND     00001111B
        LD      B,A
        LD      A,8
        LD      C,A
        CALL    BC.0C)
        OUT     C
        INC     C
        LD      A,(IX)
        PUSH    AF
        LD      A,8
        OUT     (C),A
        POP     AF
        EI
        RET
        ENDIF
;
;       .WRTVDP : write a byte into VDP register
;       Entry  b   : datum to write
;              c   : VDP register number
;       Return none
;       Modify AF, BC
;       None   : compatible with ROM-BIOS
;                 32 oban return
;
;.WRTVDP
;.WRTVDP
        STR     USE_WRTVDP
WRTVDP:
```

```
        PUSH    HL
        PUSH    DE
```



```
        LD      C,B             ; =datum
        LD      A,C             ; =register #
        SL      BL,MODSAY
        CP      8
        JR      C,B
        JR      NC,SAVEBSK
        LD      BC.SAVEBSK
        LD      BL,BOROSP-8
        SS      14              ; save data
        JR      BC.HORATE
;
SAVEBSK:
        SUB     8
        LD      B,A             ; DC=register #
        LD      AND    BC,(8)    ; BL=BSTSKY
        ADC     BL,DC           ; save data
        JR      BC.HORATE
;
HORATE:
        LD      A,C             ; =register #
        LD      DC,(7)
        INC     C
        DI
        LD      A,B
        OUT     (C),A
        AND     00111110B
        DR      10000000B
        OUT     (C),A
        POP     DE
        POP     HL
        RET
        ENDIF
;                               ; IFE USE_WRTVDP
;
;       please modify following subroutines as you need
;
DR_VSYNC:
        LD      A,A
        LD      (DPPAGE),A
        LD      DC,DPCSR        ; write DPR into reg#0
        CALL    WRTVDP          ; set page 3
        LD      A,(DSPAL)
        LD      C,23
        LD      DC,17
        CALL    WRTVDP          ; set display offset
        LD      A,A
        LD      (SET_VLINE)     ; set reg#2
DR_8LSR:
        LD      A,A
        LD      (DPPAGE),A
        LD      DC,DPCSR        ; write DPR into reg#0
        CALL    WRTVDP          ; set page 2
        LD      A,(DSPA2)
        LD      C,23
        LD      DC,17
        JP      WRTVDP          ; set display offset
;
```

AS_NEXT_EQU: ; end of program + 1

```
        DEFB    0
        DEFSAGE
        END
```


4.7.10 Machine Language Routine with Scanline Interruption

Finally, to avoid the hassle of entering the source list of the program for scanline interruption and for those who are not adept at using assemblers, a BASIC program that automatically creates and runs machine-language software is provided. By entering and running the following program, a file named "ONSCAN.BIN" will be automatically created.

List 4.10 (MKONSCAN.BAS)

```
10 CLEAR 100, &HCFFF:DIM D(15)
20 PRINT"Making onscan.bin":AD=&HB000:C=0:L=0
30 FOR I=0TO15:READ A$:IF A$="*" GOTO100
40 A=VAL("&H"+A$):C=(C+A) AND 255:D(I)=D(I)+A AND 255
45 POKE AD, A:AD=AD+1:NEXT
50 READ A$:A=VAL("&H"+A$):L=L+1
55 IF C<>A THEN PRINT "Error in line ";990+10*L:END
60 GOTO 30
100 ?
1100 PRINT"Saving"
120 BSAVE"onscan.bin",&HB000,&HB14D
130 PRINT"Done.":END
1000 DATA F5, E5, D5, C5, 7E, 20, 00, 53, 23, 23, 5E, 23, 56, 7A, B7, 28, 5D
1010 DATA A3, 3E, 28, 03, 5D, 28, 49, 3D, 28, 42, 18, 3F, 7B, 32, D9, 00, BD
1020 DATA 04, F5, CD, F4, B0, F7, 20, 07, ED, 44, 7B, 41, 21, 9A, FD, 11, CD, B0, 11, 33
1030 DATA 0A, 00, ED, B0, 21, C2, B0, 11, 9A, FD, 01, 0A, 00, F3, 26, B7, B0
1040 DATA 0A, 43, F3, 32, 9B, FD, 22, A0, FD, 3A, DF, F3, F6, 10, 47, 0E, 20
1050 DATA 09, CD, F4, B0, 18, 0F, C7, D0, 18, 0A, B4
1060 DATA A3, 34, 7B, 32, B9, B0, FB, C1, D1, E1, F1, C9, 61
1070 DATA B3, 2B, DF, F3, 62, 18, 07, DE, 00, F4, B0, 01, 17, CD, B0, 64
1080 DATA F4, B0, AF, 32, F5, FA, 01, 02, 1F, CD, F4, B0, 3A, CC, B0, BF, 54
1090 DATA C8, 21, CD, B0, 1A, 02, 7A, B1, 35, CD,
1100 DATA C9, 3A, D9, B0, 21, D7, B0, 66, 47, 0E, 13, C3, F4, B0, 3E, 01, 2F
1110 DATA CD, BA, B0, E0, A1, 81, 3A, B2, 18, 13, F5, CD, 12, B1, F1, F8, A5
1120 DATA 18, 10, F7, 00, 0E, 05, 01, 7F, 00, A8, F5, 01, 0F, 00, E7
1130 DATA F0, 00, 00, 00, D1, 01, 3A, E0, 20, ED, 16, F2, 07,
1135 DATA 4F, CD, F4, B0, D4, B0, 66, 47, 0E, 0E,
1155 DATA 21, 9F, C9, 3E, 35, 67, 69, 21, DF, 73, FE, 00, 78, D5, 01, CC, 00, F5
1165 DATA 21, 7D, 47, 09, 72, 79, 0B, EB, 47, 00, 5D
1185 DATA 01, 02, 3F, CD, ID, 42, F5, FA,
```


Chapter 5: MSX-MUSIC

This chapter is a re-edited collection of articles from the magazine MSX Magazine from July 1990 to October 1990 entitled "MSX2+ Technical Exploration".

5.1 What is FM Sound?

While the specification under the name MSX-MUSIC has been established, many may know it as a sound source that enhances the effect of games. But what is its mechanism? Let's delve into it on these pages.

5.1.1 History of electronic musical instruments leading to FM sound sources

Before explaining FM sound sources, let's turn back to the history of electronic musical instruments.

In 1968, Dr. Bob Moog created the 'Moog Synthesizer' which combined a sound-generating device that could be controlled by an electric signal and a filter that could shape the waveform. Unlike modern devices that use semiconductors, the earliest synths in 1968 and the early 70s used Moog's technology and various other modules but relied heavily on analog circuits and sound sources.

However, the synth had limits: susceptible to temperature changes, expensive, and noise entry. Recognizing these shortcomings, synthesizer manufacturers switched to digital circuits for electronic sound. One such method was called 'Programmable Sound Generator' or PSG for short. A PSG is a chip with several independent sound sources, a digital-to-analog converter (D/A converter), converting digital signals into analog audio signals. This system was less expensive and widely used in MSX2+ to generate sound.

Another method aside from PSG used to replicate sound synthesizing is 'Sampling Sound Source'. This system captures the audio of an acoustic instrument with a microphone, stores it in memory, then plays it back. This was called a 'Sampler Synthesizer'. Although its audio reproduction was high quality, the required large memory was expensive.

Lastly, our topic of interest merges the merits of analog and digital sound sources: the FM sound source. FM, or 'Frequency Modulation', is the same method used in FM radio broadcasts.

5.1 What is an FM sound source?

Table 5.1: Comparing the performance of electronic musical instruments

	Analog Synthesizer	PSG	Sampling Synthesizer	FM Sound Source
Hardware	Complex	LSI	LSI + Large Memory	LSI
Stability	Weak to temperature	Stable	Stable	Stable
Tone	Varied	Limited	Varied	Varied
Data Size	Large	Small	Large	Small
Price	High	Low	High	Average

In other words, as illustrated in Figure 5.1, by changing the frequency of a digital oscillator, it produces a more complex sound compared to PSG. This single digital oscillator is called an "operator," and as shown in Figure 5.1, combining two oscillators creates FM sound, otherwise known as a "2-operator FM sound source." Incidentally, "MSX-MUSIC" is officially called "OPLL YM2413," which is a 9-operator 2-operator FM sound source, reducing the LSI needed.

[Figure 5.1: Exploring the structure of four types of electronic musical instruments]

Analog Synthesizer Analog Sound Source → Analog Filter

PSG Noise Source | Square Wave Source | | | |

| (D/A Converter)

Sampling Synthesizer Microphone → A/D Converter → Memory → D/A Converter

FM Sound Source Digital Oscillator | Change | Digital Oscillator → (D/A Converter)

Among these, currently related to MSX machines are PSG and FM sound sources. The former has been built into MSX1 since its early models, producing sounds internal to the device. As shown in Figure 5.1, it uses three sounds and one noise to produce sounds. The latter is called MSX-MUSIC. Sound sources are also included in the FM-PAC.

MSX-MUSIC Chapter 5

Formatted Figure 5.2: Analyzing Basic Sounds

- Pure Tone

```
- Voltage
- Time
```

- Clarinet

```
- Short Wave
- Voltage
- Time
```

- Sawtooth Wave

```
- Voltage
- Time
```

- Snare Drum

```
- Noise
- Time
```

- Chime

```
- Decaying Noise
- Voltage
- Time
```

5.1.2 Let's Analyze Musical Instrument Sounds

To ”see” sound, you should look at the voltage of the sound signal on the screen of an ”Oscilloscope.” Furthermore, by using a device called a ”Spectrum Analyzer (sometimes abbreviated as SpeAna),” which analyzes the sound in terms of its frequency components, you can understand the characteristics of sound. The left part of Figure 5.2 shows the waveform characteristics of different sounds as seen on an oscilloscope, while the right part shows the spectrum as measured by a spectrum analyzer.

Generally, the most basic sound is called a ”pure tone,” represented as a sine wave shape. This tone contains only a single frequency component of the sound. Other examples include ”square waves,” which contain multiple components of the basic frequency, such as 440Hz, 1320Hz, and 2200Hz.

5.1 What is FM sound generation?

Including the tones of musical instruments, the shape of the sound waves of actual musical instruments is close to the short rectangular waves of a clarinet. Next is something similar to a sawtooth wave. This includes sound waves with harmonics that are similar to the characteristics of stringed instruments. The analog synthesizer creates sounds that mimic real musical instruments by processing these sawtooth waves. Now, the sounds of percussion instruments, especially snare drums, differ significantly as their waves are closer to noise (irregular signals). When viewed on a spectrum analyzer, the partial tones in a broad frequency range can be seen.

The sounds of the timpani are similar to the timbres between string instruments and percussion instruments, containing tones close to both the base frequency and the harmonics of the base frequency. This is called ”pitch noise” as it combines tones processed from wind and sound waves. Moreover, this kind of sound is the basis of the range of sounds in wind instruments and the pitches/noises of xylophones played by amateurs.

Instead of creating these sounds correctly by the trained hands of musicians, FM sound generation artificially creates a similar wave with an exact frequency by including the base frequency and its harmonic components. This sophisticated technique is done by

synthesizing some specially designed components with the instrument. In FM sound generation, there is a specified program that combines some musical instrument sounds together to create an end result. Most importantly, the initial elements generating sound include directly affecting the base frequency and the loudness from it. This change in loudness is divided into different categories known as ”envelopes”.

For example, if you strike a guitar or a percussion instrument, there’s a strong initial sound followed by a damping of the sound. Pressing the key of a piano initially produces a big sound that gradually decreases, holding the key continues this same sound until it’s released. Releasing the key causes the sound to decay. Xylophones exhibit the same properties where the sound grows smaller over time. The diagram (Figure 5.3) explains the setup of how the speed of sound and tone quality decrease as discussed earlier.

In contrast, the attack, decay, sustain, and release (ADSR) are categorized to produce timbre in analog synthesizers and FM sound generation. This combination generates an envelope, which in devices, is called ”ADSR”.

In older rock groups lead by keyboard idols, the effects were made by using a synthesizer to rotate the sound of fan-like wind against the accordion loudness when played live. To recreate these effects, synthesizers were used to produce the attack as if suddenly slicing through, and the decay ensuring it did not remain long, producing a variety of sounds across different devices. The specific combination of these unique components, including the combination of envelopes of FM sound generation, as defined earlier, can also be easily done by oneself by emitting effective sound timbres through a sense of experimentation.

Section 5 - MSX-MUSIC

5.3: Instrument and Synthesizer Envelopes

Figure 5.3: Instrument and Synthesizer Envelopes

- Guitar
- Piano
- Wind Instruments (General)
- Synthesizer

Envelope (ADSR): A - Attack D - Decay S - Sustain R - Release

5.1.3 The Pitch is not Restricted to Equal Temperament

Let's suddenly start talking about classical music here. First, let's organize the relationship between pitch and frequency. It can be summarized as follows in Table 5.2.

Table 5.2: Relationship Between Pitch and Frequency

Note	Frequency
A	440.0Hz
A#	466.2Hz
B	493.9Hz
C	523.3Hz
C#	554.4Hz
D	587.3Hz
D#	622.3Hz
E	659.3Hz

Note	Frequency
F	698.5Hz
F#	740.0Hz
G	784.0Hz
G#	830.6Hz
A	880.0Hz

Among these, the frequency of the note A is 440Hz. The frequency of the note one octave above A is 880Hz. Basically, the frequency of any note that is one octave higher is double. The ratio of frequencies for a one-semitone increment (e.g., from C to C#) is approximately 1.0595 times. Therefore, for twelve semitones (one octave), the frequency ratio is 2 times ($1.0595^{12} \approx 2$).

In this way, the pitch frequency can be mathematically described using ratios. This system of equal temperament was popularized to accommodate the tuning of modern pianos and other instruments. In classical music, it became a standard practice for composing. Before the development of this system, pianos and other instruments used to be tuned based on just intonation, which was more complex but provided better harmonics. However, in practice, equal temperament is more widely used because it allows for more consistent tuning across various keys, even though just intonation is said to provide better harmony.

As you can see, perfect equal temperament is not the only system. Other temperaments take into account the harmonic complexity of chords, with different focusing points. For instance, in some music before the equal temperament was standardized, chords in some keys sounded better than others. Such a difference can be found even in comparisons like between C and C# or B and C. Although equal temperament allows simpler tuning across various keys, different scales can provide distinct characteristics.

Beyond equal temperament, pre-equal temperament music and harmonics weren't as simple as in modern music. Students can learn more about the history of tuning and harmony by studying historical progressions, such as from Pythagorean tuning to just intonation and mean-tone temperament.

Therefore, even though pitches are more frequently discussed in equal temperament today, it's important to understand its limitations and the beauty of other temperaments in order to fully appreciate the historical development and nuances in music theory.

Page 135

5.1 What are FM sounds.

Table 5.3: List of tunings that can be set with MSX-Music

Number	Settable Tuning	Number	Settable Tuning
0	Pythagorean	11	Just intonation cis Major(b minor)
1	Meantone	12	Just intonation d Major(h minor)
2	Werckmeister	13	Just intonation es Major(c minor)
3	Werckmeister (Correction 1)	14	Just intonation e Major(cis minor)
4	Werckmeister (Correction 2)	15	Just intonation f Major(d minor)
5	Kirnberger	16	Just intonation fis Major(dis minor)
6	Kirnberger (Correction)	17	Just intonation g Major(e minor)
7	Vallotti & Young	18	Just intonation gis Major(f minor)
8	Rameau	19	Just intonation a Major(fis minor)
9	Equal temperament (initial setting)	20	Just intonation b Major(g minor)
10	Just intonation a Major(a minor)		

It is also possible. Instruments like violins that can continuously change frequencies adapt to any tuning. However, to change the tuning of a piano, it is necessary to retune all the strings, requiring practical effort to change the tuning. The FM source mounted on the MSX can easily change the tuning. By utilizing the characteristics in Table 5.3, you can create the sound of an instrument that cannot be realized simply by toggling the registers of the FM sound source. It may be possible to accurately perform classical music and traditional instruments. The problem with the FM source frequency number list accurately reflecting tuning can be solved, but PSG's frequency number cannot catch up. So, using PSG for tuning might pose a threat as PSG is used on basis of electronic sounds and sound effects. To solve this, enhancing the precision of tuning algorithms specific to FM sources and adding parameter settings necessary for musical performance can aid composers and sound designers in game music creation.

5.1.4 Analyzing MSX-MUSIC.

MSX-MUSIC comes with 63 prepared tone colors. 15 out of these tones are embedded in the FM sound LSI; the remaining 48 are recorded in ROM. Data for tones recorded in ROM can be called using the command:

CALL VOICE COPY.

This command displays the data from List 5.1. Note that this program specifies the number of tone colors for FM sources according to frequency parameters.

If an error occurs, in the case of the program in List 5.1,

Voice No. * has no data.

A message like that will be displayed.

List 5.1 (READFM.BAS)

```
100 ' read VOICE data of MSX-MUSIC
110 ' by nao-i on 20. Apr. 1990
120 '
130 CALL MUSIC : DEFINT A-Z
140 DIM VI(15), VD(31), VO(3)
150 PRINT "Voice Number (0,..., 63; 64 for all; 65 for end) ";
160 INPUT ME
170 IF 0 <= ME AND ME <= 63 THEN VN=ME : GOSUB 200 : GOTO 150
180 IF ME = 64 THEN FOR VN=0 TO 63 : GOSUB 200 : NEXT VN : GOTO 150
190 END
200 ON ERROR GOTO 460
210 CALL VOICE COPY(VN, VI)
220 ON ERROR GOTO 0
230 FOR I=0 TO 15
240   VD(I*2) = VI(I) AND 255
250   VD(I*2+1) = (VI(I) / 256) AND 255
260 NEXT I
270 NA$ = ""
280 NA$ = ""
290 FOR I=0 TO 8
300   IF VD(I) THEN NA$ = NA$ + CHR$(VD(I))
310 NEXT I
320 PRINT : PRINT "Vocal No."; VN;" : " NA$
330 PRINT " Transpose = "; VI(4)
340 PRINT " Feedback% = "; (VD(10) AND 14) / 2
350 FOR I=0 TO 3 : VO(I) = VD(I+16) : NEXT I
360 PRINT "Operator 0" : GOSUB 490
370 FOR I=0 TO 3 : VO(I) = VD(I+24) : NEXT I
380 PRINT "Operator 1" : GOSUB 490
390 CALL BGM(0)
400 CALL VOICE(CVN, 0, VN, 0, VN)
```

```
410 PLAY #2, "CEG<G>CR", "VEGEF<B>ER", "V4GBADGR"
420 CALL VOICE(0, 0, 0, 0, 0)
430 CALL BGM(1)
440 ON ERROR GOTO 0
450 RETURN
460 ' *** error
470 PRINT "Voice No. "; VN; "has no datum."
480 RESUME 440
490 ' *** print data of an operator
500 PRINT " AM = "; (VO(0) / 128) AND 1;
510 PRINT " PM = "; (VO(0) / 64) AND 1;
520 PRINT " EG = "; (VO(0) / 32) AND 1;
530 PRINT " KSR = "; (VO(0) / 16) AND 1;
```



```
540 PRINT " MULT=";VO(0) AND 15;
550 PRINT " Lks=";(VO(1) ¥ 64) AND 3;
560 PRINT " Lf=";VO(1) AND 63
570 PRINT " ADSR=";(VO(2) ¥ 16) AND 15;"";"";" VO(2) AND 15;"";
580 PRINT (VO(3) ¥ 16) AND 15;"";" VO(3) AND 15
590 RETURN
600 ON ERROR GOTO 0
```

An operator that operates in a program falls into two categories. Operator 1 is basically a ”Carrier Operator” that creates a basic form of sound wave, and Operator 2 is a ”Modulator Operator” that modulates it. Explaining each setting’s meaning in detail would need an entire book, so let’s leave it to the reference. Try to understand this by actually listening to the sound. Also, you can change the ”PRINT” section in list 5.1 to ”LPRINT”, which might be a useful reference material for designing your own sound.

5.1.5 Challenging Rhythm Sounds Using FM Sound Source

Not limited to MSX-MUSIC, FM sound sources struggle with rhythm sounds. Actual percussion instruments and analog synthesizers may produce unregulated noise, but FM tone generators are too rule-bound, often creating ”fake” or ”mechanical” sounds.

Thus, MSX-MUSIC is equipped with a function for generating ”rhythm sounds” alongside 63 types of musical instruments. Among these 63 instrument tones are also drum tones, but the rhythm sounds here are distinctly different from the combined instrument sounds.

As written in manuals, MSX-MUSIC has built-in 6 operators that can be used from channel 1 to channel 9. These enable the production of voice-based sounds among the sound sources.

The method to create rhythm sounds is to arrange it so that channels 1 to 6 can produce tones like usual, and channels 7 to 9, comprising 6 operators, can also produce rhythm sounds. Hence, a similar expression is used to imply ”musical instruments + drum sounds” in the context of MSX-MUSIC features.

To utilize these functions in BASIC, you need to change the parameter of the ”CALL MUSIC” command. For instance:

CALL MUSIC(0,0,1,1,1,1,1,1,1) to output 9 musical instruments.

CALL MUSIC(1,0,1,1,1,1,1,1,1) to output 1 drum and 8 musical instruments.

Page 138: Chapter 5 MSX-MUSIC

Various types of instruments + 1 percussion sound are selected.

The program example printed below is to listen to the difference in rhythm sounds generated by the MSX-MUSIC sound data and to create your own sounds. First, the sound of instrument number 31, ”Bass Drum,” is played four times using 2 operators, and then, a rhythm sound is played using 6 operators. Try to play the rhythm sounds in the list into your machine and listen to them with your own ears to see how it compares to a real drum.

List 5.2 (BASSDRUM.BAS)

```
10 CALL MUSIC (1,0,1,1,1,3)
20 CALL BGM(0)
30 CALL VOICE (031)
40 PLAY "v15OCC", "","","","", "RRRRB!4B!4B!4B!4"
50 CALL VOICE (00)
```

Also, with MSX, you can play FM sound source and PSG simultaneously. You can produce musical sounds using FM sound source, and percussion and sound effects using PSG. However, the volume of the FM sound source and the volume of the PSG can be different depending on the type of MSX hardware. Thus, you will need a program to adjust the sounds accordingly to match your machine.

5.2 Controlling FM Sound Source

The FM sound source has become an indispensable existence for game BGM and sound effects. In this page, let's try to control the FM sound source with machine language programs.

5.2.1 Playing Sound with a Machine Language Program

As introduced on the previous page, it was possible to easily manipulate the FM sound source using extended BASIC. However, to use it as the game’s sound effect or BGM, it is necessary to manipulate the FM sound source by calling the machine language program “FM-BIOS”.

Here, I would like to introduce an example program and library for MSX-C to control the FM sound source. Before proceeding, to compile and run these machine language files, you will need the “MSX-DOS TOOLS” (or DOS2 TOOLS) and “MSX-C ver.1.1 (or ver.1.2)”.

So, in List 5.3, the test program made with MSX-C is “TESTFM.C”. While distributing test data and playing the bass drum and snare drum, it will play the ‘Doremi-fasorashido’ scale four times. The way to create this data is also written in the information published on the MSX network msx.spec board.

List 5.3 (TESTFM.C)

/*

```
    testfm.c
    by nao-i on 29. May. 1990
    (C) Isikawa 1990
    free to use and copy, but no guarantee or support
```

*/ #include <stdio.h> #include ”fmlib.h” #pragma nonrec

#define TESTLENGTH 20 #define TESTTIMES 4

static char fmdata[] = { /* test data */

```
    14, 0,                    /*  0: offset to rhythm */
    33, 0,                    /*  2: offset to ch 1 */
    0, 0, 0, 0, 0, 0, 0, 0, 0, /*  4: offset to ch 2..6 */
    FM_RVOL + 31, 8,             /* 14: rhythm VOL = 8 */
    0x30, TESTLENGH,              /* 16: B drum */
    0x28, TESTLENGH,              /* 18: S drum */
```

};

5.2 Controlling FM Sound Source

Next, let's briefly explain the program. First, prepare an array called ”fmwork” in auto memory with a size of ”FMWORK” bytes. Then, pass the address of this array to call the library ”fmopen”. This array needs to be above 8000H in memory, so let's declare it as auto, not static.

If the above process succeeds in setting up the FM sound source, “fmwork” will be below 7FFFH, and “fmopen” will be returned.

One thing to be careful about is that before using the FM sound source, you must call ”fmclose” to terminate the FM sound source program. If the program ends before ”fmclose” is called, it can cause problems, so this library has to be terminated with either the CTRL + C key or the CTRL + STOP key.

If you are creating a program that uses the FM sound source yourself, it is also possible that processing related to the disk may be entered by pressing the Ctrl + C key. Always call “fmclose” in any situation and make sure it finishes. Next, let's actually call for the start of FM sound source playback. Immediately, the FM-BIOS starts playing. Since this FM-BIOS is a timer-controlled activity, it continues to play while the program progresses. In this program, we tried to display ”Playing.” on the screen while playing.

”fmtest” is used to check during playback and returns 0 if playback has ended. ”fmstop” stops playback and initializes the FM-BIOS.

5.2.2 Explaining Library Overview

List 5.4 lists the header file “FMLIB.H,” which defines the functions and constants related to the FM sound source library.

List 5.4 (FMLIB.H)

```
/*
 * fmlib.h : header file for fmlib
 *       by nao-t on 31. May. 1990
 *       by nao-t on 24. Feb. 1991 FM_01 changed from 0 to 1
 *                (C) Isikawa 1990
 *       free to use and copy, but no guarantee or support
 */
```

extern char fmopen(); /* please call this first / extern VOID fmclose(); / please call this last */

extern VOID fmwrite(); /* write to OPLL register / extern VOID fmtori(); / write to OPLL register 0...7 / extern VOID fmstart(); / background music / extern VOID fmstop(); / stop background music / extern char fmread(); / read data from ROM / extern char fmtest(); / now playing ? / #define FMWORK (0x00a+32) / size of work area */

#define FM_VOL 0x0060 /* volume 60H...6FH / #define FM_INST 0x0070 / instrument 70H...7FH / #define FM_SUSOFF 0x0080 / sustain off / #define FM_SUSON 0x0081 / sustain on / #define FM_EXPINST 0x0082 / expanded instrument / #define FM_USRINST 0x0083 / user-defined instrument / #define FM_LEGOFF 0x0084 / legato off / #define FM_LEGON 0x0085 / legato on / #define FM_Q 0x0086 / Q / #define FM_END 0x00ff / end of data / #define FM_RVOL 0x00a0 / volume of rhythm */

/* pitch / #define FM_C 0 / C / #define FM_CS 1 / C# / #define FM_D 2 / D / #define FM_DS 3 / D# / #define FM_E 4 / E / #define FM_F 5 / F / #define FM_FS 6 / F# / #define FM_G 7 / G / #define FM_GS 8 / G# / #define FM_A 9 / A / #define FM_AS 10 / A# / #define FM_B 11 / B */

/* octave / #define FM_O1 1 / FM_O1+FM_C means C of octave 0 */ #define FM_O2 13 #define FM_O3 25 #define FM_O4 37 #define FM_O5 49 #define FM_O6 61 #define FM_O7 73 #define FM_O8 85

Next, the long List 5.5 below is the FM sound source library. From the beginning of the list, in order, there are the definitions of addresses such as BIOS, the definitions of the macros that call FM-BIOS, the definitions of the work areas used by the library, and then the main body of the library program.

5.2 Controlling the FM Sound Source

List 5.5 (FMLIB.Z80)

```
; fmlib.z80 : library for MSX-C
; by nao-i on 29. May. 1990
; (C) ASCII 1988 for 'search', (C) Isikawa 1990
; free to use and copy, but no guarantee or support

; .Z80
;
; address of BIOS and system work area
rdslt   equ   000ch
calslt  equ   001ch
enaslt  equ   0024h
breakv  equ   0f325h  ; ^C break vector
ramad0  equ   0f341h  ; slot # of RAM
ramad1  equ   0f342h
ramad2  equ   0f343h
ramad3  equ   0f344h
exptbl  equ   0fcc1h
h.timi  equ   0fd9fh  ; timer interrupt hook

; address of FM-BIOS jump table
.idstr  equ   4018h+4
```

-wrtopl equ 4110h -iniopl equ 4113h -mstart equ 4116h -mstop equ 4119h -rddata equ 411ch -opldrv equ 411fh -tstbgm equ 4122h

; MACROs to call FM-BIOS CALLFM MACRO ADDRESS

```
  ld  ix, address
  ld  iy, (biosslot-1)
  call  calslt
  ENDM
```

JUMPFM MACRO ADDRESS

```
  ld  ix, address
  ld  iy, (biosslot-1)
  jp  calslt
  ENDM

; dseg
```

biosslot: ds 1 ; slot of FM-BIOS breaks: ds 2 ; saving ^C vector p.ontime: ds 2 ; address of interrupt handler p.oldhook: ds 2 ; address of saved hook

```
  cseg
;
ontime:
  push   af
  ld     ix,_opldrv
  ld     iy,0          ; will be modified
ontime.slot equ $-1
  call   calslt
  pop    af
oldhook:
  nop
  nop
  nop
  nop
  nop
  ret
sizeof_ontime equ $-ontime
IF      sizeof_ontime GT 31
    .PRINTX "ontime routine too big"
ENDIF
;
hooktbl:
  rst    30h
  db     0             ; will be modified
  dw     0             ; will be modified
toret:
  ret
vvv:
  dw     toret         ; to ignore ^C
;
;  char  fmopen(address)
;  char  *address;    /* address of work area */
;
;  0 : successful
;  1 : no FM-BIOS
;  2 : bad address of work area
;
fmopen@::
  ld     a,2
  bit    7,h
  ret    z             ; address of work area < 8000H
  ld     (p.ontime),hl
  push   hl
  ex     de,hl
  ld     hl,ontime
  ld     bc,sizeof_ontime
  ldir                 ; transfer ontime routine
  pop    hl
  push   hl
  ld     de,oldhook-ontime
  add    hl,de
  ld     (p.oldhook),hl
  pop    hl
  ld     de,32
```



```
  add    hl,de
  res    0,l            ; make sure address is even
  push   hl
  call   search
  ld     a,(biosslot)
  ld     hl,(p.ontime)
  ld     de,ontime.slot-ontime
  add    hl,de
```

```
  ld     (hl),a        ; modify LD IY,??00
  or     a
  ld     a,i
  pop    hl            ; address of work area
  ret    z             ; no FM-BIOS
  CALLFM _inispl
  ld     h,40h
  ld     a,(ramad1)
  call   enaslt        ; does not restore slot1
;
  ld     hl,h.timi
  ld     de,(p.oldhook)
  ld     bc,5
  ldir                 ; save h.timi
  ld     hl,hooktbl
  ld     de,h.timi
  ld     bc,5
  di
  ldir                 ; set h.timi
  ld     a,(ramad2)
  ld     (h.timi+1),a
  ld     hl,(p.ontime)
  ld     (h.timi+2),hl
;
  ld     hl,(breakv)
  ld     (breaks),hl   ; save break vector
  ld     hl,vvv
  ld     (breakv),hl   ; set break vector
  ei
  xor    a
  ret
;
; VOID  fmclose()
;
fmclose@::
  di
  ld     hl,(breaks)
  ld     (breakv),hl   ; restore break vector
  ld     hl,(p.oldhook)
  ld     de,h.timi
  ld     bc,5
  ldir                 ; restore h.timi
  ei
  ret
;
```



```
;   VOID fmwrite(RegNum, Datum)
;       char    RegNum;      /* OPLL register number        */
;       char    Datum;       /* datum to write              */

fmwrite0::
        JUMPFM  _wrtopl
;
;   void    fmotir(aData)
;       char    aData[8];

fmotir0::
        ld      b,8
        xor     a
        di
fmotir_loop:
        ld      e,(hl)
        inc     hl
```

```
        CALLFM  _wrtopl
        inc     a
        djnz    fmotir_loop
        ret

;
;    void    fmstart(pDatum, Times)
;       char    *pData;      /* pointer to Music data   */
;       char    Times;       /* times to play           */

fmstart0::
        ld      a,e
        inc     a
        ret     z               ; make sure that Times != 255
        dec     a
        bit     7,h
        ret     z               ; make sure that pData >= 8000H
        JUMPFM  _mstart
;
;   VOID fmstop()
;
fmstop0::
        JUMPFM  _mstop
;
;   char    **fmread(ptr, num)
;       char    *ptr;
;       char    num;

fmread0::
        ld      a,e
        JUMPFM  _rddata
;
;    char    fmtest()
;
fmtest0::
        JUMPFM  _tstbgm

;
; Search FM-BIOS
```



```
search:
        ld      b,4
search_id:
        push    bc              ;save counter
        ld      a,4
        sub     b               ;make primary slot number
        ld      c,a             ;save it
        ld      hl,exptbl       ;point expand table
        ld      e,a
        ld      d,0
        add     hl,de
        ld      a,(hl)          ;get the slot is expanded or not
        add     a,a             ;expanded ?
        jr      nc,no_expanded  ;no..
        ld      b,4             ;number of expanded slots
search_exp:
        push    bc              ;save it
        ld      a,24h           ;[a]=00100100b
        sub     b               ;make secondary slot # A=001000ss
        rlca                    ;[a]=01000ss0b
        rlca                    ;[a]=1000ss0b
        or      c               ;make slot address A=1000sspp
        call    chkids          ;check id string
```

```
        pop     bc              ;restore counter
        jr      z,search_found  ;exit this loop if found
        djnz    search_exp
not_found:
        xor     a
        ld      (biosslot),a
        pop     bc
        djnz    search_id
        ret
no_expanded:
        ld      a,c             ;get slot address
        call    chkids          ;check id string
        jr      nz,not_found    ;exit this loop is found
search_found:
        pop     bc
        ret
;
id_string:
        db      'OPLL'
id_string_len equ $-id_string
;
; Check ID string
;       Entry : [A]=slot address to check
;       Return: Zero flag is set if ID is found
;       Modify: [AF],[DE],[HL]
;
chkids:
        ld      (biosslot),a
        push    bc              ;save environment
        ld      hl,idstrg
        ld      de,id_string
```

Interpreting the program points, ”fmopen@” transfers the timer-embedded processing program from ”ontime” to a specific location, searches for the slot where the FM-BIOS is located, initializes it, and sets the timer-embedded vector. The transferred data it references must be placed above the 8000H address, so when transferring the program, the part written as ”ld iy, 0” is rewritten to ”ld iy, FM-BIOS slot number * 256”.

The program itself, which searches for the slot where the FM-BIOS is located, references the specifications from ASCII NET MSX, examining all slots. By examining whether the string ”OPLL” exists at address 401C, it finds the FM-BIOS.

Now, the 192-byte work area indicated by ”fmopen@” is used for embedded processing programs, temporary storage of the contents of ”th.timer,” and the work area of the FM-BIOS (160 bytes). At this time, the work area for FM-BIOS is necessary, including potential BIOS data, so the starting address is carefully set to align with the remaining work area.

5.2 FM Sound Source Control

Though it was not written in the specification, when you initialize the FM-BIOS with ”CALLFM_ainiopl”, there are cases where Page 1 is switched to another slot and does not return. In such cases, you need to switch Page 1 back to its original slot.

As mentioned earlier, it is troublesome if a program ends with a timer-side interrupt hook written as it is. If you write hook number F235H of the MSX-DOS work area, you can return by pressing the CTRL + C key. Alternatively, if you add a program that returns the disk to the boot state, you will be able to operate it. And ”fmcIose” is one that returns the timer interrupt hook and the CTRL + C key process to its original.

For the remaining part of the FM library, you can operate it just by setting the necessary

value in the register and calling the FM-BIOS. Try various things by yourself.

5.2.3 Compiling with MSX-C

If you enter the list in an editor such as MED or KID of MSX-DOS, save it under the filename "TESTFM.C". List 5.4 is "FMLIB.H". And List 5.5 is "FMLIB.Z80". Next, compile it with the following step to generate "TESTFM.COM". To run the program, just enter "TESTFM" from the MSX-DOS command line.

List 5.6 (FMLIB.BAT) m80 =fmlib.z80/r/m/z cf testfm cg testfm m80 =testfm.mac/r/m/z 180 testfm,fmlib,ck,clib/s,crun/s,cend,testfm/n/y/e:xmain

5.3 The Data Structure of the FM Sound Source

This section explains the data structure of the FM sound source and how to actually specify sound source data. Use it together with the machine-language program for driving the FM sound source that was described earlier.

5.3.1 Let's Create Data for the FM Sound Source

On the previous pages, we presented an example program that calls the FM-BIOS from machine language to produce sound. Here, we explain how to create the FM sound source data needed to perform music using that program.

To summarize the FM-BIOS data structure, it amounts to one large array. Each data position within the array is counted by the number of bytes from the head of the array, and this count is called the "offset." Therefore, the offset of the head of the array is 0. Also, the head of the array is called the "0th byte," the next one the "1st byte," and so on.

Table 5.4 shows the data structure for 6 instrument sounds + 1 percussion sound, together with an actual data example. As you can see from it, the data body itself is arranged toward the end of the array, and the leading bytes of the array contain the offsets at which each piece of data is placed.

Table 5.4: Data structure for 6 instrument sounds + 1 percussion sound |Offset|Contents| |---|---| |0|Offset of the percussion sound data (Note 1)| |2|Offset of instrument sound 1 data (Note 2)| |4|Offset of instrument sound 2 data (Note 2)| |6|Offset of instrument sound 3 data (Note 2)| |8|Offset of instrument sound 4 data (Note 2)| |10|Offset of instrument sound 5 data (Note 2)| |12|Offset of instrument sound 6 data (Note 2)| |(Note 3)|Instrument sound 1 data| |(Note 3)|Instrument sound 2 data| |(Note 3)|Instrument sound 3 data|

(Note 1) First write the 2 bytes 0EH, 00H here. (Note 2) For each instrument sound, write the offset (in bytes, from the head of the data) to the start position of that instrument sound data, in the order low byte then high byte. For instrument sounds that are not used, write 0. (Note 3) The instrument sound data is written at the location specified by the offset.

Explaining the data structure in order: first, the 0th and 1st bytes (offsets 0 and 1) hold the offset at which the percussion sound data is placed, and offset 2 through 12 (0EH, 00H) is written there. So that the Z80 CPU can understand this value, it is expressed in 2 bytes in the order low byte, high byte, which become 0EH and 00H respectively.

In the following 2nd through 13th bytes (offsets 2 to 13) are written the offsets of the data for instrument sound channels 1 through 6. If at this time offset 0 is spec-

5.3 FM Sound Source Data Structure

Offset	Content
0	00H 00H
2	21H 00H
4	00H 00H
6	00H 00H
8	00H 00H
10	00H 00H
12	00H 00H
14~	Drum Sound Data
33~	Instrument Data

This is an example of the data structure for 6 instrument sounds + 1 drum sound. For example, from the 14th to the 32nd byte, there is drum sound data; from the 33rd byte onwards, there is instrument sound data. This suggests that if the 0th byte contains 00H, the second channel will not be used.

For example, in the data structure example of the Table 5.5, at the second and third byte (offset 2 and 3), it is noted as 21H 00H. This means that the sound source data for instrument channel 1 is located starting from offset 33. Also, from the 14th byte to the 33rd byte (offset 14 to 33), if it contains 00H, the second instrument channel will not be used.

The previously introduced program to control FM sound source might embed the length of data in a language program, using offset to specify. However, it could be easier to create the sound source data as shown below. Referencing:

```
FMDATA:
  DW 14
  DW CH1-FMDATA
  DW CH2-FMDATA
  DW CH3-FMDATA
  DW CH4-FMDATA
  DW CH5-FMDATA
  DW CH6-FMDATA
  DB Drum Sound Data...

CH1:
  DB Channel 1 Data...

CH2:
  DB Channel 2 Data...
```

Similar entries for channels CH3 to CH6

With this approach, an assembler like MSX-DOS's assembler (M80) can automatically calculate offsets at the time of assembling the source list.

However, this program cannot be directly used as is. In a BASIC context where the PLAY statement is used, using a music macro language would be more appropriate.

(MML) is necessary to convert to FM-BIOS sound source data.

Now, let's consider the data structure if you perform 9 tone sounds without percussion sound as shown in Table 5.6. Basically, it will have the same 6 tone sounds + 1 percussion sound data structure, so, until the 17th byte (offset + 017) the channel 1-9 tone sound data assigns specified offset to the next, if you do not specify 00H it will not be used.

Also, in channel 1 tone data, you start from the 18th byte (offset + 018), therefore you need to make the data structure so that if there are fewer channels in the tone data, from the

10th byte (18 in hexadecimal).

Table 5.6: 9 Sound Instrument Data Structure

Offset	Content
0	Tone Data 1 Offset (Note 1)
2	Tone Data 2 Offset (Note 2)
16	Tone Data 9 Offset (Note 2)
18	Tone Data 1
(Note 3) Tone Data 9	

(Note 1) Has to be written either 12H or 00H at the beginning. (Note 2) Refers top byte tone data beginning position, offset next bytes. (Note 3) When specified by the offset, the tone data will be gathered at the location.

Incidentally, as an aside, FM-BIOS decided at the starting point of the sound data to be 0EH and 12H decides whether 6 tone + 1 percussion instrument = 9 tone instruments. Hence if tone data in percussion is not 14 bytes (offset + 14), it is needed starting from channel 1, if tone sound does not exist, it will start from byte 18 (offset + 18).

5.3.2 Specifying Percussion Data

Regarding the details of the figure reposted in Figure 5.4, actually a separate detail data of 5 types of percussion is expressed by alphabets "BSTCH" and sequentially as sound data.

Like this data is assumed in 2 bytes, specifying each percussion sound. 101BSTCH 0000VVVV

5.3 FM Sound Data Construction

Figure 5.4: Percussion Sound Data

```
b7  b6  b5  b4  b3  b2  b1  b0
-----------------------------
1  0  1  B  S  T  C  H
0  0  0  0  V  V  V  V
0  0  1  B  S  T  C  H
-----------------------------
Length (0~255)

10110000  Bass Drum
00000000  Volume 0
10100101  Snare Drum
00000001  Volume 1
10110000  Bass Drum
00010100  Volume 20
10110000  Snare Drum
00010100  Volume 20
10110000  Hi-Hat
00010100  Volume 20
11111111  Data End
```

B -> Bass Drum S -> Snare Drum T -> Tom C -> Cymbal H -> Hi-Hat

To select a percussion instrument, set each of the 5 bits `BSTCH` (each to either 1 or 0). The volume has a maximum value from 0 to 15. The volume setting specifies the attenuation level, with the maximum being 0 and the minimum being 15, so be careful. The data for

specifying the volume of the bass drum to 0, the snare drum, and the cymbal to 1 would be specified as:

```
10110000
00000000
10110010
00001010
```

First, designate the type of percussion, and then next, specify the length for each percussion instrument. However, the length itself is always fixed, so it indicates the interval between the emitted sound and the next sound.

```
0001BSTCH
```

This specifies the type of percussion, and the next byte indicates length (up to 255). If the length is over 255, first write 255, then the actual length from the next step.

Page 154

Specify a value minus 255. If this value is over 255, repeat the same operation. For example,

00110000 11111111 00000000

represents bass drum, sound length 255.

00110010 11111111 11111111 11101011

represents bass drum and cymbal, sound length 1000.

Although the FM sound specification documentation did not describe the unit of ”sound length,” test data showed that the sound unit was the same period as the timer division, namely, 1 second of 60 minutes.

5.3.3 Let's Specify Instrument Data

As referenced in Table 5.7, this is the detailed data for instrument sounds. Among this data are values that make sense with a single byte, byte and a half, or two bytes combinations continuing onward.

The order of instrument data is specified as: sound volume, tone color, sustain, legato, Q decay.

Each sound peg is denoted with the same data as instrument pegs, expressed from maximum volume to decay. In other words, 15 represents the pegs from maximum volume to decay.

Sustain is supposed to hold the decay for the instrument sound. As explained earlier, the instrument's envelope is determined by the value ”ADSR”. Repeating this, A is attack (speed of reach), D is decay, S is sustain (level of maintenance), R is release (decay to decay).

The color of the OPLL bass sound is fixed by ”ADSR.” However, it was not meant to be a sustain. When the sustain is off, a loud drop can continue for both channels 1 and 2. Specific to both channels, tampering is possible to add a project-style peg only. In this case, one sound is connected from pitch Y to the next sound of pitch X. If a channel-specific decay is needed, twisting a slight peg might be necessary.

5.3 FM Sound Data Structure

Table 5.7: Instrument Sound Data

Value	Meaning
00H	Rest, followed by 1 byte for length
01H	C in octave 1, followed by 1 byte for length
5FH	A# in octave 7, followed by 1 byte for length
60H	Volume 0 (minimum volume)
6FH	Volume 15 (maximum volume)
70H	Sound 0 (ROM internal tone or user-specified tone)
71H	Sound 1 (Violin)
7FH	Sound 15 (Electric Bass)
80H	Sustain Off
81H	Sustain On
82H	1 byte follow (00H~3FH) for ROM internal tone number
83H	2 bytes follow (lower byte, upper byte) for sound data address
84H	Legato Off
85H	Legato On
86H	Followed by 1 byte (01H~08H) as Q
FFH	Data End

The value that can be specified for Q ranges from 1 to 8 and represents the ratio of the specified note's length to its actual sound length. For example, if Q=6 and the length of the note is 80, then $80 \times 6 \div 8 = 60$ being the length of the sound, so $80 \times (8 - 6) \div 8 = 20$ being the rest length.

Above, the combination of value specified for legato, sustain, and Q, determines the connection of the note, in other words, it decides the phrasing of the melody.

Table 5.8: Example of Instrument Sound Data

68H, 73H	Sound 8
73H	Sound 3 (Guitar)
81H	Sustain On
84H	Legato Off
86H, 06H	Q=6
25H, 14H	Octave 4, C, sound 20
27H, 14H	Octave 4, D, rest 20
29H, FFH, FFH, FFH, EBH	Octave 4, E, sound 1000
FFH	Data End

Next, specify the pitch and length for each note. The 1-byte value from 00H to 5FH represents the pitch, and the next 1-byte represents the length of the note. For example:

25H, 14H

The 2 bytes of data represent the pitch of C in octave 4 and a length of 20. To represent a length of more than 255, it is the same as for percussion instruments. For example:

29H, FFH, FFH, FFH, EBH

The 5 bytes of data represent the E in octave 4 and a length of 1000.

5.3.4 Things That OPLL Drivers Can't Do

Among the functions provided by FM-BIOS, automatically playing back data given at intervals called by timer interrupts is called ”OPLL driver.” Here, using this driver automatically in BASIC with

CALL PITCH CALL TEMPER CALL TRANSPOSE

was explained as the method to achieve this capability. However, as I was about to release FM-BIOS, I received an inquiry, saying, ”Please try doing it yourself.” In other words, the

user had to write to the OPLL registers themselves; therefore, it was determined that it would not be possible with the OPLL driver.

In conclusion, when using an OPLL driver, the standard 12-tone equal temperament at A 440 Hz can be achieved directly by inputting, but it is not feasible to set detailed performance instructions or write to the sound source registers in MIDI in fine detail, as FM-BIOS drivers can. Additionally, if you keep in mind the background music for games, the effect may not appear satisfactory.

Therefore, if you're aiming for sound effects using FM sound sources, you should use a driver with extended functions or convert to sound driver data elements like MML. Detailed FM sound-related information will be featured later in the article, so those confident in their programming skills, give it a try.

5.3.5 Let's Add Sound Data

As mentioned earlier, FM sound source LSI (OPLL) has 15 types of control tones. FM-BIOS ROM includes 48 types of sound data.

5.3 FM Tone Data Structure
157

Figure 5.5: Tone Data

b7	b6	b5	b4	b3	b2	b1	b0
0	AM	VIB	EGT	KSR	Multiple	Op. 0	
1	AM	VIB	EGT	KSR	Multiple	Op. 1	
2	KSL (Op. 0)	Total LEVEL MODELEATER					
3	KSL (Op. 1)	Empty	DC	DM	Feedback		
4	Attack (Op. 0)	Decay (Op. 0)					
5	Attack (Op. 1)	Decay (Op. 1)					
6	Sustain (Op. 0)	Release (Op. 0)					
7	Sustain (Op. 1)	Release (Op. 1)					

Op. 0 represents the Modulator Operator, and Op. 1 represents the Carrier Operator. These 8 bytes are written into OPLL registers 0 to 7. Details can be found in ”MSX2+ Powerful Utilization Techniques (ASCII Publications)”.

This kind of data structure illustrated in Figure 5.5 makes it possible to add even more tones.

These 8-byte tone data are written into registers 0 to 7 of the FM sound source LSI (OPLL) just as they are. They serve as extension commands of BASIC:

CALL VOICE CALL VOICE COPY

Note that the 32-byte data used has a different format.

Inside the OPLL, each channel's tone is specified with values from 0 to 15. When a tone is specified, the tone is set by registers 0 to 7 of the OPLL, displaying original tone data. Thus, user-defined tone data or tone data stored in FM-BIOS or ROM can use 16 types simultaneously.

This was due to the advent of channel number limitations rather than tone color limitations. Therefore, a user-defined tone can be allocated to channels 1 and 2, making it possible to assign another set of user-defined tones to channels 2 to 4.

If explaining the contents of tone data, we would end up writing another book, so we’ll introduce a more concise reference this time.

”MSX2+ Powerful Utilization Techniques” ASCII Edition, ASCII Publications

However, this book contains some errors. We should correct each page's content here.

Also, a resource that you should access once through your personal computer is the network's MSX ”msx.spec” board. This board contains MSX machine language programs.

Page 158

To use MUSIC, the ”FM-BIOS” specification is published. In addition, various information regarding MSX is included.

5.3.6 Explaining Sample Data

Finally, let's summarize and introduce an example of specifying actual data. List 5.7 shows part of a program list that was previously put together. The first part is to specify the data offset for the channels. The drum data starts from byte 1 and the music channel data starts from byte 33. Channels 2 to 5 are not used. The middle part of the list is the data for the music. First, volume 8 is set for all instruments. The value of ”FM_RVOL” is 0xA0, and adding this makes it 0xBF, meaning all instruments' volume specification has been unified. This is followed by specifying the bass drum note 20, followed by the snare drum note 28, in order. The value for ”FM_END” is 0xFF, indicating the end of the data. The lower part of the list is the data for music channel 1. First, the sound 8, OPLL internal range F number, key-on, and the length 7 are specified. Then, the chord ”FM_FANFARE” is played in length 20, and ”FM_END” indicates the end. Since it is inconvenient to specify the actual frequency directly like 95, the value of octave 4 is divided into 12 scales and indicated. For example,

FM_04 + FM_D

indicates D in octave 4. The value of ”FM_04” is 37, and the value of ”FM_D” is 2; adding these results in 39, which represents D in octave 4. Also, to create test data with an assembler (M80), it would be useful to define numbers as follows:

```
FM_C EQU 1
FM_CS EQU 2
```

... FM_VOL EQU 60H ...

And for volume,

DB FM_VOL + 8

In the same way, using the same scale,

5.3 FM Sound Source Data Structure

DB FM_04 + FM_C

Specify as follows.

List 5.7 (Sample Data for Musical Instruments)

#define TESTLENGTH 20 #define TESTTIMES 4

static char fmdata[] = /* Here, offsets for each channel are specified. */ {

```
    14, 0,
    33, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0,

  /* This is the percussion instrument data. */
  /* After specifying the common parameters at the beginning, */
  /* specify each percussion instrument sound in sequence. */
```

```
    FM_PRVOL + 31, 8,
    0x30, TESTLENGTH,
    0x28, TESTLENGTH,
    0x28, TESTLENGTH,
    0x28, TESTLENGTH,
    0x28, TESTLENGTH,
    0x28, TESTLENGTH,
    0x28, TESTLENGTH,
    FM_END,

  /* This is data for musical instrument channel 1. */
  /* In this list, channels 2 to 5 are not used. */
    FM_VOL + 8,
    FM_INST + 3,
    FM_SUSON,
    FM_LEGOFF,
    FM_Q, 6,
    FM_04 + FM_C, TESTLENGTH,
    FM_04 + FM_D, TESTLENGTH,
    FM_04 + FM_E, TESTLENGTH,
    FM_04 + FM_F, TESTLENGTH,
    FM_04 + FM_G, TESTLENGTH,
    FM_04 + FM_A, TESTLENGTH,
    FM_04 + FM_B, TESTLENGTH,
    FM_05 + FM_C, TESTLENGTH,
    FM_END,

};
```

Page 160

5.4 Various Aspects of FM Sound Source

I thought the explanation about FM sound sources was all over by now, but it seems something was omitted. Please bear with me for a little longer.

5.4.1 Corrections to the Content of Powerful Utilization Method

I introduced the ”MSX2+ Powerful Utilization Method” as a reference for the MSX2+ machine (ASCII publication, price 1240 yen including tax), but several errors have been found. In contrast to the actual situation, it seems many people are troubled with confusion. Therefore, I will correct the errors within my understanding on this page. If you find any errors other than the ones mentioned here, please inform the M7 editorial department.

Firstly, a correction from the second day. By looking at the ”Table 4.4 List of Voice Libraries” on page 147, you will find this color number ”10 Guitar” in the column of ”OPLL VOICE” summary. Please add a guitar note to this.

Next, there were several errors in the program listed on page 136 (READFM.BAS). Verify the correct abbreviated notation as it displays inaccurate numbers. Play the actual sound while validating it.

Next, the explanation part related to ”VOICE COPY” on page 148. The expression part in the text says, ”The number you can specify as a parameter 1 is between 0~63, but in the OPLL VOICE column, the number for each color is not assigned incorrectly. The correct notation is really an invalid number.”

Similarly, if you specify a non-existent voice number in the OPLL VOICE, ”Illegal function call” will be the error message. Correct it to, ”If you specify nonexistent color data...”

Also, in the registers on pages 151 to 158, there were several errors in the list of OPLL

registers. Please see the corrected version in Chart 5.6. It may be more convenient as it corrects the content in the Powerful Utilization guide you have in hand.

Regarding the F-Number required for the target frequency on page 155, there was an error. Correct it as follows. The first notation in the chart,

F-Number = (440 x 2^18 + 50000) ÷ 2^4 -1 = 288

5.4 Various things related to the FM sound source 165

the formula is, correctly,

F-Number = (440 × 2^{18} ÷ 50000) ÷ 2^(4 - 1) = 288

which is how it works out.

Finally—and this is not limited to ”MSX2+ Powerful Utilization Methods”—there seems to be a generally mistaken explanation of how to specify musical-tone data for the FM sound source, so I will correct it here.

For specifying the pitch of each note, it is generally said that the values from 00H to 5FH correspond, respectively, to C of octave 1 through B of octave 7, but this is wrong.

Figure 5.6: List of OPLL registers

br	b6	b5	b4	b3	b2	b1	b0
00H	AM(M)	VIB(M)	EGT(M)	KSR(M)	-	-	Multiple(M)
01H	AM(C)	VIB(C)	EGT(C)	KSR(C)	-	-	Multiple(C)
02H	-	-	-	Total Level Modelater			
03H	KSL(C)	-	DC	DM	-	Feed Back	
04H	Attack(M)	Decay(M)					
05H	Attack(C)	Decay(C)					
06H	Sustain(M)	Release(M)					
07H	Sustain(C)	Release(C)					
08H	-	R	BD	SD	TOM	T-CT	HH
0FH	test register						
10H	F-number						
18H	-						
20H	-						
28H	-	Sus.	Key	Block	F-number		
30H	Inst.	Vol.					
38H	-						

In the case of rhythm mode

br	b6	b5	b4	b3	b2	b1	b0
36H	-	Bass Drum					
37H	Hi Hat	Snare Drum					
38H	Tom Tom	Top Cymbal					

In the table, the parts marked (M) indicate operator 0, which works as the modulator, and those marked (C) indicate operator 1, which works as the carrier. For details, refer to ”MSX2+ Powerful Utilization Methods” or to Yamaha's technical documentation.

Page 162 Chapter 5 MSX-MUSIC

... Actually 00H is a rest symbol and continues from 01H to 5FH, corresponding to octave 1 C to octave 7 A#.

5.4.2 List of Sound Data for MSX-MUSIC

In response to the requests of many programmers, we will list the dump list of the sound data stored in the ROM of MSX-MUSIC.

The one listed on the left side of Table 5.9 is the 32-byte sound data obtained by the BASIC "CALL VOICE COPY" statement, extracted from the 8 bytes of data written to OPLL registers 0 to 7. Sound data is said to be "voice transfer" data that also contains data for register 9, but because it is not data directly written to the OPLL registers, it is omitted from this document.

Sound data has 61 types, and although these may look almost the same, the differences in the values of voice transfer are what cause the differences in sound. The sound color code written on the tables in the "using data of OPLL" list has sound data assigned from the OPLL, so there is sound data in the ROM.

Next, the various ROM-internal sound data for the FM-BIOS is listed. Until now, it was still unclear to everyone whether this was really internal sound data, so we will determine this matter. Therefore, the right side of Table 5.9 lists the 8-byte sound data for each sound, obtained using the "RDDATA" function of FM-BIOS.

Comparing the left and right sides of Table 5.9, you can see that the tone numbers and names are roughly the same between extended BASIC and FM-BIOS, but the sound data slightly differs. Therefore, if music is composed using BASIC MML and the data is converted for FM-BIOS, there may be slight problems with the sound.

Regarding most of the sound data for FM-BIOS, there may be those who find it strange that the data written to register 3 is 20H, but since the 5th bit of register 3 is "mute," even if the value written to register 3 is 20H, the actual performance sound color will not change.

5.4 FM Music Tidbits

Table 5.9: List of Timbre Data

No.	Timbre Name	Expanded BASIC Timbre Data	FM-BIOS Timbre Data
0	Piano 1	using data of OPLL(3)	31 11 0E 20 D9 B2 11 F4
1	Piano 2	30 10 0F 04 D9 B2 10 F4 30 10 0F 20 D9 B2 11 F3	
2	Violin	using data of OPLL(1)	61 61 12 20 B4 56 14 17
3	Flute 1	using data of OPLL(4)	61 21 30 20 8E 34 13 26
4	Clarinet	using data of OPLL(5)	A2 30 20 A0 88 54 14 06
5	Oboe	using data of OPLL(6)	31 34 20 20 F2 56 20 06
6	Trumpet	using data of OPLL(7)	34 71 16 20 51 52 26 24
7	Pipe Organ 1	34 30 37 06 50 30 76 06 34 30 37 06 50 30 76 06	
8	Xylophone	17 52 18 05 89 B6 66 24 17 52 18 05 89 B6 66 24	
9	Organ	using data of OPLL(8)	63 40 20 08 FC F8 28 29
10	Guitar	using data of OPLL(2)	02 41 15 20 A3 A3 75 05
11	Santool 1	19 53 0C 06 C7 F5 11 03 19 53 0C 06 C7 F5 11 03	
12	Electric Piano 1	using data of OPLL(15)	23 43 09 20 DD BF 4A 05
13	Clavicode 1	03 09 11 06 D2 B4 F5 F6 03 09 11 06 D2 B4 F5 F5	
14	Harpsicode 1	using data of OPLL(11)	00 06 20 4A E3 24 F4 F4
15	Harpsicode 2	01 01 11 06 C0 B4 01 F7 01 01 11 20 C0 B4 D1 F6	
16	Vibraphone	using data of OPLL(12)	F9 F1 44 21 20 54 1A F6
17	Koto 1	13 11 0C 06 FC 2A 33 84 13 11 0C 20 FC 2A 33 83	
18	Taiko	01 01 0E 07 64 E4 44 01 01 0E 20 64 E4 44 24	
19	Engine 1	F0 F4 18 B7 97 10 F4 08 F0 F4 18 20 97 10 F4 08	
20	UFO	7F 10 F1 05 00 5F 01 02 7F 10 F1 20 00 5F 15 06	
21	Synthesizer Bell	13 11 11 07 FA F2 21 F5 13 11 11 20 FA F2 21 F4	
22	Chime	A6 42 10 05 FB B9 11 02 A6 42 10 20 FB B9 11 02	
23	Synthesizer Bass	using data of OPLL(13)	40 31 89 20 C7 F9 14 04
24	Synthesizer	using data of OPLL(10)	42 44 08 20 94 B0 33 F6
25	Synthesizer Percussion	01 03 0B 07 BA D9 25 06 01 03 0B 20 BA D9 25 06	
26	Synthesizer Rhythm	40 00 00 07 FA D9 37 04 40 00 00 20 FA D9 37 04	
27	Harm Drum	02 03	

No.	Tone name	Tone data for Extended BASIC	Tone data for FM-BIOS
32	Piano 3	11 11 08 04 FA B2 20 F5	11 11 08 20 FA B2 20 F4
33	Electric Piano 2	using data of OPLL(14)	11 11 11 20 CB B2 01 FA
34	Santool 2	19 53 15 07 E7 8F 25 21 03	19 53 15 20 E7 8F 25 01
35	Brass	30 70 19 07 42 62 26 24	30 70 19 20 42 62 26 24
36	Flute 2	62 71 25 07 E6 66 26 26	62 71 25 20 E6 66 26 26
37	Clavicode 2	21 03 08 05 D9 04 02 F6	21 03 08 20 D9 04 02 F5
38	Clavicode 3	01 03 0B B4 AD 02 F6	01 03 0B 20 AD 02 F6
39	Koto 2	43 53 06 05 B9 8E 05 06	43 53 06 20 B9 8E 04 04
40	Pipe Organ 2	50 47 A3 07 E7 24 20 2F	50 47 A3 20 E7 24 20 2F
41	PohdsPLA	73 33 0A 06 F9 F5 14 13	73 33 5A 20 F9 F5 13 F5
42	RohdsPRA	33 33 6A 05 F9 F5 33 06	73 33 6A 20 F9 F5 33 06
43	Orch L	61 21 15 07 76 54 23 05	61 21 15 20 76 54 23 05
44	Orch R	61 21 15 07 76 54 23 05	61 21 15 20 76 54 23 05
45	Synthesizer Violin	61 A1 0A 05 F6 54 12 07	61 A1 0A 20 F6 54 12 07
46	Synthesizer Organ	61 78 0D 05 FE 24 12 03	61 78 0D 20 FE 24 12 03
47	Synthesizer Brass	31 71 15 07 B6 F9 03 26	31 71 15 20 B6 F9 03 26
48	Tube	using data of OPLL(9)	61 71 0D 20 75 F2 18 03
49	Shamisen	03 0C 14 06 AF 7C 13 15	03 0C 14 20 AF 7E 13 15
50	Magical	13 32 81 03 05 B0 33 10	13 32 80 20 05 A3 10 05
51	Huwawa	F1 31 17 05 A4 87 30 05	F1 31 17 20 A0 8F 30 05
52	Wander Flat	F0 74 17 47 5A 43 06 FD	F0 74 17 20 5A 43 06 FC
53	Hardrock	20 71 0D 06 C1 D5 56 06	20 71 0D 20 C1 D5 56 06
54	Machine	30 32 06 06 40 04 04 74	30 32 06 20 40 04 04 74
55	Machine V	03 03 06 03 40 04 04 74	03 03 06 20 40 04 04 74
56	Comic	01 08 07 78 F8 7F FA	01 08 0D 20 F8 7F FF
57	SE-Comic	C8 C0 06 B7 76 F7 11 F9	C8 C0 08 20 78 F7 11 F9
58	SE-Laser	49 40 0B 07 B4 F9 00 05	49 40 0B 20 B4 F9 00 05
59	SE-Noise	CD 42 06 A2 F0 03 20	CD 42 06 20 F0 03 20
60	SE-Star 1	51 42 13 07 13 10 42 01	

APPENDIX

R800 Instruction Table January 24, 1991 ASCII Corporation Systems Division, MSX Magazine Editorial Department

For those burning with the desire to program at the machine language level, definitely take on the challenge of developing programs on the R800. By utilizing the instruction table detailed below, which includes operation details for each mnemonic and machine code, please use it to your fullest advantage. What kind of programs can you create using the speed of the R800?

A.1 How to Use the Instruction Table

This table organizes the R800 instructions by type. The ”mnemonics” column shows the name of each command, and the ”operation” column summarizes its function.

A notation in the form of ”operation/mnemonics” within the table indicates that the content of the right side replaces the content on the left side. The term ”← [h1]” signifies that the address of the content displayed in the .h1 register memory is represented by an 8-bit value indicated by the operation. Note that input commands of [in] and [c] denote the corresponding I/O port numbers.

The ”flags” row shows the behavior of each flag, and the ”op codes” specify the machine code for each instruction, recorded consecutively in 2-byte sequence. The right-hand columns ”B” and ”C” each represent the length (number of bytes) and the number of clock cycles required to execute the command, respectively.

In addition, concerning the abbreviations listed on the instruction table, they are summarized in the following examples. Note that some mnemonics are different from those listed for the Z80.

This is due to the unique function of the R800. Although arithmetic commands added to the R800, which were not officially supported during the Z80 era, differ, the mnemonic operations registered are essentially identical. Please refer to the Z80 instruction tables while developing your programs.

A.1 How to Use the Instruction Table

a.(7) The most significant bit of register a a.4..1 Bits 4-7 of register a

```
                  Division of operations
.de.hl+  The upper 16 bits are in de, the lower 16 bits are in hl, 32-bit integer
```

.[ix+d] The address pointed to by adding the signed value d to the ix register CF Carry flag Z Zero flag S Sign flag P/V Parity overflow flag N Subtract flag H Half carry flag

- ```
  Flag doesn't change
  ```

```
0             Flag becomes 0 depending on execution result
1             Flag becomes 1 depending on execution result
```

- ```
  Uncertain
  ```

M Used when the overflow flag is reset P Used when the parity flag is set

- ```
  Value of the signed flag
  ```

r, r' 8-bit register ... a, b, c, d, e, h, l R 8-bit register ... a, b, c, d, e, h, l, ixh, ixl v, v' 8-bit register ... a, b, c, d, e, h, l, iyh, iyl p 8-bit register ... ixh, ixl q 8-bit register ... iyh, iyl ss 16-bit register ... bc, de, hl, sp tt 16-bit register ... bc, de, ix, sp uu 16-bit register ... bc, de, iy, sp pp 16-bit register ... bc, de, hl, af qq 16-bit register ... bc, de, hl, af short Signed 8-bit address difference in terms of command address (+127 to -128) brk Jump destination address 00h, 08h, 10h, 18h, 20h, 28h, 30h, 38h k 16-bit immediate value or absolute address nn 8-bit immediate value b Indicates which bit number in an 8-bit number N Number to be added B Number to be subtracted NOT Logical NOT V Logical OR ^ Logical XOR & Logical AND -> Shift to the right side tmp Temporary register B Number of bytes for the command C Minimum number of clocks required for execution of the command

If the clock count for a branch instruction or jump instruction is written separately, the upper part is the condition where it is established, and the lower part is the condition where it is not established.

Also, if the clock count for an output command is written separately, the upper part is the transfer end and the lower part is the transfer not end.

The clock count in the table is "conversion of SYSCLK to 4 divisions" of the XTAL oscillation frequency wave number. Additionally, the value when executed with no wait and the value when executed with DRAM are listed; it is automatically inserted into the wait by the page brake or refresh.

A.2 8-bit Shift Commands

| Mnemonic | Command Action | flags | Opcode | | | | SZHPNC | 76543210 | Hex | B | C |

| ld r, r' | r ← r' | ••●••• | 01 r r' | 1 1 | | ld r, n | r ← n | ••●••• | 00 r n | 2 2 | | ld r, [hl] | r ← [hl] | ••●●•• | 01 r 110 | 2 2 | | ld r, [ix+d] | r ← [ix+d] | ••●●•• | DD 01 r 110 | d d | 3 5 | | ld r, [iy+d] | r ← [iy+d] | ••●●•• | FD 01 r 110 | d d | 3 5 | | ld [hl], r | [hl] ← r | ••●••• | 01 110 r | 1 2 | | ld [ix+d], r | [ix+d] ← r | ••●●•• | DD 01 110 r | d d | 3 5 | | ld [iy+d], r | [iy+d] ← r | ••●●•• | FD 01
  ```

110 r | d d | 3 5 | | ld u, u' | u ← u' | ••●••• | 01 u u' | 2 2 | | ld v, v' | v ← v' | ••●••• | 01 v v' | 2 2 | | ld u, n | u ← n | ••●••• | DD 01 110 | d n 00 u 110 | 3 3 | | ld v, n | v ← n | ••●••• | FD 01 110 | d n 00 v 110 | 3 3 | | ld [hl], n | [hl] ← n | ••●••• | 00 110 n | 2 3 6 | | ld [ix+d], n | [ix+d] ← n | ••●●•• | DD 00 110 n | d n 36 | 4 5 | | ld [iy+d], n | [iy+d] ← n | ••●●•• | FD 00 110 n | d n 36 | 4 5 |

A.3 16-bit Transfer Instructions (169)

Mnemonic	Command Operation	Flags	Opcode
ld a, i	a ← i	↑ ↓ 1 0 FF 0	ED 2 2
ld a, r	a ← r	↑ ↓ 1 0 FF 0	5 7
ld i, a	i ← a	.....	ED 2 2
ld r, a	r ← a	.....	4 7
ld a, bc	a ← (bc)	.....	0A 1 2
ld a, de	a ← (de)	.....	1A 1 2
ld a, (nn)	a ← (nn)	.....	3A 3 4

| ld (bc), a | (bc) ← a | ..... | 0 2 1 2 | | ld (de), a | (de) ← a | ..... | 1 2 1 2 | | ld (nn), a | (nn) ← a | ..... | 3 2 3 4 | | r,b c d e h l | (a) | | v,b c d e ixh ixl | (a) | | v,b c d e iyh iyl | (a) |

A.3 16-bit Transfer Instructions

Mnemonic	Command Operation	Flags	Opcode
ld ss,nn	ss ← nn	.....	3 3
ld ix,nn	ix ← nn	.....	DD 4444
ld iy,nn	iy ← nn	.....	FD 4444

| ld sp,hl | sp ← hl | ..... | F9 1 1 | | ld sp,ix | sp ← ix | ..... | DD 2 2 | | ld sp,iy | sp ← iy | ..... | FD 2 2 |

This table shows various 16-bit transfer commands, their operations, flag effects, and opcodes.

APPENDIX R800 Instruction Table

Mnemonic | Operation | flags | Opcode | Hex | BC | | S Z H P/V N C | 76 543 210| |ld ss,[nn] |ssH←[nn+1] | | 11 101 101 | ED | 4 | 6 | |ssL←[nn] | | 01 ss 1011 | | | || |ld hl,[nn] | h←[nn+1] | | 00 101 010 | 2A | 3 | 5 | | l←[nn] | | nnH | | | || |ld ix,[nn] | ixh←[nn+1] | | 11 101 101 | DD | 4 | 6 | | ixl←[nn] | | 00 101 010 | 2A | | || |ld iy,[nn] | iyh←[nn+1] | | 11 111 101 | FD | 4 | 6 | | iyl←[nn] | | 00 101 010 | 2A | | || |ld [nn],ss | nn+1]←ssH | | 11 101 101 | ED | 4 | 6 | | [nn]←ssL | | 01 ss 0011 | | | || |ld [nn],hl | nn+1]←h | | 00 100 010 | 22 | 3 | 5 | | nn]←l | | nnH | | | || |ld [nn],ix | nn+1]←ixh | | 11 101 101 | DD | 4 | 6 | | nn]←ixl| | 00 100 010 | 22 | | || |ld [nn],iy | nn+1]←iyh | | 11 111 101 | FD | 4 | 6 | | nn]←iyl | | 00 100 010 | 22 | | || | ss bc de hl sp | |00 01 10 11 |

A.4 Exchange Instructions

Mnemonic	Instruction Operation	flags	Opcode
xch .de,.hl	de↔hl	••••••	11011011
xch .af,.af'	af↔af'	↑ ↑ ↑ ↑ ↑ ↑	00001000
xch .[sp],.hl	[sp]↔hl;h↔[sp+1]	••••••	11100001
xch .[sp],.ix	ixl↔[sp];	••••••	111001101
		ixh↔[sp+1]	••••••
xch .[sp],.iy	iyl↔[sp];	••••••	111000011
		iyh↔[sp+1]	••••••
xchx	bc↔bc';de↔de';hl↔hl'	••••••	11101001

A.5 Stack Operation Instructions

Mnemonic	Instruction Operation	flags	Opcode
push qq	sp-2←qq;[sp-1]←qqh; sp←sp-2	•••••• ••••••	11000101
push .ix	sp-2←ix;l;sp-1←ixh; sp←sp-2	•••••• ••••••	110111101
push .iy	sp-2←iy;l;sp-1←iyh; sp←sp-2	•••••• ••••••	110111101
pop qq	qq←[sp];qqh←[sp+1] sp←sp+2	•••••• ••••••	111000001
pop .ix	ixl←[sp];ixh←[sp+1] sp←sp+2	•••••• ••••••	110111101
pop .iy	iyl←[sp];iyh←[sp+1] sp←sp + 2	•••••• ••••••	111100011

00 01 10 11 qq,bc,de,hl,af

When pop .af, all flags will change.

A.6 Block Transfer Instructions

Mnemonic	Operation	flags
move [.de→(hl);.de→de+1]	→ [.hl→++;de→hl;↓bc--;]	*1.
move [.hl++;.de→hl;↓bc--;]	→ [.hl→de;.hl→hl(+1);bc→bc(−1)]	○●○
move [.de→(hl);.de→de+1]	→ [.hl→--.];de.->hl;bc->bc(-1)	
move[.hl→.de]	→[.hl--;.de--.;hl-->.-1	*de++
move[mov[.repeate]:.de→(hl)]	→.[.hl→.2(-1);.de→(.de.->)(de.(-:de)1<seq→add(	×:×)]
movem[.hl->.->];→hl++;de--.→de[-][-];bc---→bc(0)]	...ignore invalid instructions → .bc≠0(*0) × ■ 0110111000 ●[	BC▽0

*1.bc~1=0, otherwise 1

A.7 Block Search Instructions

Mnemonic	Operation	flags	C
cmp [a,(hl++);hl→hl (+1)]	→ [(a.;std+:;seq ←);(units sel=:)]	■ 2↑ HL=-0.[Σ0]	
cmp [a,(hl--); hl→hl (hl--)]	→(a(-1);seq→units;((unit=cmd)	(cmd→add)])(std+)	H
hl →:.cmpm	repeat:;•→cmp hl+de	seq_→cmd◆(00×)com←at units=-1 ED. 24	A
.a[.→(cmp).(hl); BC-BM_Seq;, =empty.-]	else.(.=b++ [:(01) ↑ repeat]	*4&& BC-bc= (S-HL=)01	

*2. a=(HL=0=HL=) .: HL&cmdσ

A.8 Arithmetic Instructions

Mnemonic	Operation	flags	Opcode
mulub a, r	.hl→axr	0 0 0 0	11 1011 101
muluw hl,ss	.de:hl→hl←+ss	x0 0 1 1	[HL»

```
mulub (this applies).:--and  hod.,d, e→should not exist to prevent. (correct:CMD:)|
 ↪  [HL=||@:nn<units doesn't work]) for 11   |
mulbw (this applies)→:ss applies should be only used with other same→HL=(&c→s ,apply
 ↪  [HL←SS.always work)
```

A.9 Add Commands (Arithmetic Commands)

Mnemonic	Command Operation	Flags	Opcode	HEX	B	C
add a, r	a ← a + r	[↓ ↕ ↑ V0]	10000 r	1 1		
add a, p	a ← a + p	[↓ ↕ ↑ V0]	11000101	DD 2	2	

Mnemonic	Command Operation	Flags	Opcode	HEX	B	C
			10000 p			
add a, q	a ← a + q	[↓ ↕ ↑ V0]	11111001	FD 2	2	
			10000 q			
add a, [hl]	a ← a + [hl]	[↓ ↕ ↑ V0]	10000110	86 1	1	
add a, [ix + d]	a ← a + [ix + d]	[↓ ↕ ↑ V0]	11001101 DD	D 3	3	
			10000110	86		
add a, [iy + d]	a ← a + [iy + d]	[↓ ↕ ↑ V0]	11111001 FD	3 5	5	
			10000110	86		
add a, n	a ← a + n	[↓ ↕ ↑ V0]	11001110	C6 2	2	
			n			
addc a, r	a ← a + r + C	[↓ ↕ ↑ V0]	10001 r	1 1		
addc a, p	a ← a + p + C	[↓ ↕ ↑ V0]	11010101	DD 2	2	
			10001 p			
addc a, q	a ← a + q + C	[↓ ↕ ↑ V0]	11111011	FD 2	2	
			10001 q			
addc a, [hl]	a ← a + [hl] + C	[↓ ↕ ↑ V0]	10001110	8E 1	2	
addc a, [ix + d]	a ← a + [ix + d] + C	[↓ ↕ ↑ V0]	11001101 DD	3 5	5	
			10001110	8E		
addc a, [iy + d]	a ← a + [iy + d] + C	[↓ ↕ ↑ V0]	11111001 FD	3 5	5	
			10001110	8E		
addc a, n	a ← a + n + C	[↓ ↕ ↑ V0]	11001110	CE 2	2	
			n			
addc hl, ss	hl ← hl + ss + C	[↓ ↕ ↑ 0]	0 11 ss 10101	ED 2	2	
add hl, ss	hl ← hl + ss	•••0 ↑	001 ss 1001	1 1		
add ix, pp	ix ← ix + pp	•••0 ↑	11001101 DD	2 2	2	
			00 pp 1001			
add iy, rr	iy ← iy + rr	•••0 ↑	11111001 FD	2 2	2	
			00 rr 1001			

Key:

- ”↓ ↓ ↕ ↑ V0” represents the flag settings.

Note: ”a”, ”r”, ”p”, ”q”, ”hl”, ”ix”, ”d”, ”n”, ”C”, ”ss”, ”pp”, ”rr” etc. in the commands and flags refer to registers and values relevant to the context of assembly language programming.

Page 174 APPENDIX R800 Instruction Table

Mnemonic	Operation	Flags	Opcode	Hex	B	C
incr	r ← r+1	↑ ↑ ↓ V0 • •	00 r 100	00	1	1
incp	p ← p+1	• • • • • •	11 001 101 100 101	DD 22	2	
incq	q ← q+1	• • • • • •	11 110 001 101 103	FD 22	2	
inc [hl]	[hl] ← [hl]+1	• • • • 0	00 000 010	2		
inc [ix+d]	[ix+d] ← [ix+d]+1	• • • • 0	11 001 101 01			
incss	ss ← ss+1	• • • • • •	00 ss 0011	1		
inc.ix	ix ← ix+1	• • • • • •	10 011 001	DD 22	3	
inc.iy	iy ← iy+1	• • • • • •	11 111 011 101 101	FD 22	23	

00 01 10 11 ss: bc, de, hl, sp pp: bc, de, ix, sp rr: bc, de, iy, sp

000 001 010 011 100 101 110 111 p: ixh, ixl q: iyh, iyl

A.10 Subtraction Instructions

A.10 Subtraction Instructions

Mnemonic	Operation	Flags S Z H P/V N C	Machine Code (Hex)	Length (Bytes)	Cycles
sub a,r	a ← a - r	↑ ↑ ↓ V ↑ ↑	10000 r	1	1
sub a,p	a ← a - p	↑ ↑ ↓ V ↑ ↑	11001101 10010 p	DD 2	2
sub a,q	a ← a - q	↑ ↑ ↓ V ↑ ↑	11111 101 10010 q	FD 2	2
sub a,[hl]	a ← a - [hl]	↑ ↑ ↓ V ↑ ↑	10010110	96 1	2
sub a,[ix+d]	a ← a - [ix+d]	↑ ↑ ↓ V ↑ ↑	11011101 10010110 d	DD 3	5
sub a,[iy+d]	a ← a - [iy+d]	↑ ↑ ↓ V ↑ ↑	11111101 10010110 d	FD 3	5
sub a,n	a ← a - n	↑ ↑ ↓ V ↑ ↑	11001110 n	D6 2	2
subc a,r	a ← a - r - c	↑ ↑ ↓ V ↑ c	10011 r	1	1
subc a,p	a ← a - p - c	↑ ↑ ↓ V ↑ c	11001101 10011 p	DD 2	2
subc a,q	a ← a - q - c	↑ ↑ ↓ V ↑ c	11111101 10011 q	FD 2	2
subc a,[hl]	a ← a - [hl] - c	↑ ↑ ↓ V ↑ c	10011110	9E 1	2
subc a,[ix+d]	a ← a - [ix+d] - c	↑ ↑ ↓ V ↑ c	11011101 10011110 d	DD 3	5
subc a,[iy+d]	a ← a - [iy+d] - c	↑ ↑ ↓ V ↑ c	11111101 10011110 d	FD 3	5
subc a,n	a ← a - n - c	↑ ↑ ↓ V ↑ c	11011110 n	DE 2	2
subc hl,ss	hl ← hl - ss - c	↑ ↑ ↓ V ↑ ↑	11111110 01ss0010	ED 2	2

Please note that the symbols S, Z, H, P/V, N, and C represent different flags in the CPU status register, '↑' indicates the flag is affected, '↓' indicates it is reset, and 'V' indicates it might be affected based on the operation's outcome. ”r”, ”p”, ”q”, ”n”, ”[hl]”, ”[ix+d]”, ”[iy+d]”, and ”ss” represent different registers or operands in the CPU.

Page 176

Mnemonic | Command Operation | flags | Op-Code | Hex | BC SZH P/V N C | 76543210 decr | r←r-1 | ↑ ↑ 1 v 1 0 | 00 r 101 | 1 1 decp | p←p-1 | ↑ ↑ 1 v 1 0 | 11011 101 | DD 2 2 decq | q←q-1 | ↑ ↑ 1 v 1 0 | 00 p 101 | FD 2 2 dec. [hl] | .hl←.hl-1 | ↑ ↑ 1 v 1 0 | 00 q 101 | 35 1 4 dec. [ix+d] | .ix+d←(.ix+d)-1 | ↑ ↑ 1 v 1 0 | 111011 101 | DD 3 7 dec. [iy+d] | .iy+d←(.iy+d)-1 | ↑ ↑ 1 v 1 0 | 111011 101 | FD 3 7

```
          |                    |                    | 00 110 101       |    *      35
                               |                                  |                    |
```

decss | ss←ss-1 | …….. | 00 ss 110 | dec.ix | ix←ix-1 | ……. | 101011 101 | DD 2 2B dec.iy | iy←iy-1 | …….. | 111011 101 | FD 2 2B

A.11 Comparison Command Mnemonic | Command Operation | flags | Op-Code | Hex | BC SZH P/V N C | 76543210 cmp.a,r | a←r | ↑ ↑ 1 v 1 0 r | 10111 r | 1 1 cmp.a,p | a←p | ↑ ↑ 1 v 1 0 r | 10111 101 | DD 2 2 cmp.a,q | a←q | ↑ ↑ 1 v 1 0 r | 11111 101 | FD 2 2 cmp.a,. [hl] | a←. [hl] | ↑ ↑ 1 v 1 0 r | 11111 110 | BE 1 2 cmp.a,. [ix+d] | a←. [ix+d] | ↑ ↑ 1 v 1 0 r | 111011 101 | DD 3 5 | 101111 110 | BE cmp.a,. [iy+d] | a←. [iy+d] | ↑ ↑ 1 v 1 0 r | 111011 101 | FD 3 5 | 101111 110 | BE cmp.a,n | a←n | ↑ ↑ 1 v 1 0 | 111111 110 | FE 2 2 | n

A.12 Logical Operation Instructions 177

A.12 Logical Operation Instructions

```
Mnemonic    | Operation     | Flags                | Opcode
            |               | S Z H P N C          | 76543210 | Hex | B | C
and a, r    | a ← a∧r       | ↑ ↑ 1 P 0 0          | 10100rrr |     | 1 | 1
and a, p    | a ← a∧p       | ↑ ↑ 1 P 0 0          | 11011101 | DD  | 2 | 2
            |               |                      | 10100ppp |     |   |
and a, q    | a ← a∧q       | ↑ ↑ 1 P 0 0          | 11111101 | FD  | 2 | 2
            |               |                      | 10100qqq |     |   |
and a, [hl] | a ← a∧[hl]    | ↑ ↑ 1 P 0 0          | 10100110 | A6  | 1 | 2
and a, [ix+d] | a ← a∧[ix+d] | ↑ ↑ 1 P 0 0         | 11011101 | DD  | 3 | 5
            |               |                      | 10100110 | A6  |   |
            | d             |                      |          |     |   |
and a, [iy+d] | a ← a∧[iy+d] | ↑ ↑ 1 P 0 0         | 11111101 | FD  | 3 | 5
            |               |                      | 10100110 | A6  |   |
            | d             |                      |          |     |   |
```

```
and a, n    | a ← a∧n       | ↑ ↑ 1 P 0 0           | 11100110 | E6  | 2 | 2
            | n             |                       |          |     |   |

or a, r     | a ← avr       | ↑ ↑ 0 P 0 0           | 10110rrr |     | 1 | 1
or a, p     | a ← avp       | ↑ ↑ 0 P 0 0           | 11011101 | DD  | 2 | 2
            |               |                       | 10110ppp |     |   |
or a, q     | a ← avq       | ↑ ↑ 0 P 0 0           | 11111101 | FD  | 2 | 2
            |               |                       | 10110qqq |     |   |
or a, [hl]  | a ← av[hl]    | ↑ ↑ 0 P 0 0           | 10110110 | B6  | 1 | 2
or a, [ix+d] | a ← av[ix+d] | ↑ ↑ 0 P 0 0          | 11011101 | DD  | 3 | 5
            |               |                       | 10110110 | B6  |   |
            | d             |                       |          |     |   |
or a, [iy+d] | a ← av[iy+d] | ↑ ↑ 0 P 0 0          | 11111101 | FD  | 3 | 5
            |               |                       | 10110110 | B6  |   |
            | d             |                       |          |     |   |
or a, n     | a ← avn       | ↑ ↑ 0 P 0 0           | 11110110 | F6  | 2 | 2
            | n             |                       |          |     |   |

xor a, r    | a ← a□r       | ↑ ↑ 0 P 0 0           | 10101rrr |     | 1 | 1
xor a, p    | a ← a□p       | ↑ ↑ 0 P 0 0           | 11011101 | DD  | 2 | 2
            |               |                       | 10101ppp |     |   |
xor a, q    | a ← a□q       | ↑ ↑ 0 P 0 0           | 11111101 | FD  | 2 | 2
            |               |                       | 10101qqq |     |   |
xor a, [hl] | a ← a□[hl]    | ↑ ↑ 0 P 0 0           | 10101110 | AE  | 1 | 2
xor a, [ix+d] | a ← a□[ix+d] | ↑ ↑ 0 P 0 0         | 11011101 | DD  | 3 | 5
            |               |                       | 10101110 | AE  |   |
            | d             |                       |          |     |   |
xor a, [iy+d] | a ← a□[iy+d] | ↑ ↑ 0 P 0 0         | 11111101 | FD  | 3 | 5
            |               |                       | 10101110 | AE  |   |
            | d             |                       |          |     |   |
xor a, n    | a ← a□n       | ↑ ↑ 0 P 0 0           | 11101110 | EE  | 2 | 2
            | n             |                       |          |     |   |
```


Section Title: A.13 Bit Operation Instructions

Table Headings: | Mnemonic | Command Action | flags S Z H (B) N C | Operation Code 76 543 210 | Hex | BC | |-------------------------|----------------------|-----------------------|-----|----|| bit b,r | z ← NOT r^(b) | ? 1 ? 0 0 | 01 001 011 | CB | 22 | | bit b,[hl] | z ← NOT [hl]^(b) | ? 1 ? 0 0 | 11 001 011 | CB | 25 | | bit b,[ix+d] | z ← NOT [ix+d]^(b) | ? 1 ? 0 0 | 11 001 011 | DD | 4 5| | | | | 11 001 011 | CB | | bit b,[iy+d] | z ← NOT [iy+d]^(b) | ? 1 ? 0 0 | 11 111 101 | FD | 4 5| | | | | 11 001 011 | CB | | set b,r | r^(b)←1 | · · · · · · · · | | CB | 22 | | set b,[hl] | [hl]^(b)←1 | · · · · · · · · | | CB | 25 | | set b,[ix+d] | [ix+d]^(b)←1 | · · · · · · · · | | DD | 4 7| | | | | | CB | | | set b,[iy+d] | [iy+d]^(b)←1 | · · · · · · · · | | FD | 4 7| | | | | | CB | | | clr b,r | r^(b)←0 | · · · · · · · · | | CB | 22 | | clr b,[hl] | [hl]^(b)←0 | · · · · · · · · | | CB | 25 | | clr b,[ix+d] | [ix+d]^(b)←0 | · · · · · · · · | | DD | 4 7| | | | | | CB | | | clr b,[iy+d] | [iy+d]^(b)←0 | · · · · · · · · | | FD | 4 7| | | | | | CB | |

A.14 Rotate Instructions 179

A.14 Rotate Instructions

```
Mnemonic        | Operation                                  | Flags          | Opcode
                |                                            | S Z H P N C    | 76543210 |
                ↪  Hex | B | C
rola            | C ← a(7); a := a×2; a(0) := C              | – – 0 – 0 ↑    | 00000111 |
↪  07  | 1 | 1
rora            | C ← a(0); a := a/2; a(7) := C              | – – 0 – 0 ↑    | 00001111 |
↪  0F  | 1 | 1
rolca           | tmp = C; C ← a(7); a := a×2; a(0) := tmp   | – – 0 – 0 ↑    | 00010111 |
↪  17  | 1 | 1
rorca           | tmp = C; C ← a(0); a := a/2; a(7) := tmp   | – – 0 – 0 ↑    | 00011111 |
↪  1F  | 1 | 1
```

```
rol r           | C ← r(7); r := r×2; r(0) ← C              | ↑ ↑ 0 P 0 ↑    | 11001011 |
↪  CB  | 2 | 2
                |                                          |              | 00000rrr |    |   |
rol [hl]        | C ← [hl](7); [hl] := [hl]×2; [hl](0) ← C  | ↑ ↑ 0 P 0 ↑    | 11001011 |
↪  CB  | 2 | 5
                |                                             |               | 00000110 | 06
                ↪  |   |
rol [ix+d]      | C ← [ix+d](7); [ix+d] := [ix+d]×2;        | ↑ ↑ 0 P 0 ↑    | 11011101 |
↪  DD  | 4 | 7
                | [ix+d](0) ← C                              |               | 11001011 | CB
                ↪  |   |
                |                                          |              | d       |    |   |
                |                                             |               | 00000110 | 06
                ↪  |   |
rol [iy+d]      | C ← [iy+d](7); [iy+d] := [iy+d]×2;        | ↑ ↑ 0 P 0 ↑    | 11111101 |
↪  FD  | 4 | 7
                | [iy+d](0) ← C                              |               | 11001011 | CB
                ↪  |   |
                |                                          |              | d       |    |   |
                |                                             |               | 00000110 | 06
                ↪  |   |
ror r           | C ← r(0); r := r/2; r(7) ← C              | ↑ ↑ 0 P 0 ↑    | 11001011 |
↪  CB  | 2 | 2
                |                                          |              | 00001rrr |    |   |
ror [hl]        | C ← [hl](0); [hl] := [hl]/2; [hl](7) ← C  | ↑ ↑ 0 P 0 ↑    | 11001011 |
↪  CB  | 2 | 5
                |                                             |               | 00001110 | 0E
                ↪  |   |
ror [ix+d]      | C ← [ix+d](0); [ix+d] := [ix+d]/2;        | ↑ ↑ 0 P 0 ↑    | 11011101 |
↪  DD  | 4 | 7
                | [ix+d](7) ← C                              |               | 11001011 | CB
                ↪  |   |
                |                                          |              | d       |    |   |
                |                                             |               | 00001110 | 0E
                ↪  |   |
ror [iy+d]      | C ← [iy+d](0); [iy+d] := [iy+d]/2;        | ↑ ↑ 0 P 0 ↑    | 11111101 |
↪  FD  | 4 | 7
                | [iy+d](7) ← C                              |               | 11001011 | CB
                ↪  |   |
                |                                          |              | d       |    |   |
                |                                             |               | 00001110 | 0E
                ↪  |   |
```

Page: 180

Mnemonic	Command Action	Flags	Opcode
rolc r r <- r × 2; r(0) <- tmp	tmp <- C; C <- r(7)	1 0 P 0 1 110001011 11 000 1110	CB 2 2 0 010 r
rolc (hl) A <- A × 2; (A,0,0,3) <- (hl)(4, r) .hl <- (hl)/2; (hl)(r) <- tmp	tmp <- C; C <- (hl)(7) 1 0 P 0 1	1 0 P 0 1 111011101 ED 2 5 67	CB 2 5 6F 5

A.15 Shift Commands

Mnemonic	Command Action	Flags	Opcode	Hex	B C
shl r	C←r(7)	1 1 0 P 0 1	11000111	CB	2
shla	r←r*2			00 1000	- 26
shl .hl	C←(.hl) (7)	1 1 0 P 0 1	11000111	CB	2
shla	(.hl) ← (.hl)*2			00 1000	- 26
shl .ix+d	C←(.ix+d) (7)	1 1 0 P 0 1	11000101	DD	4

Mnemonic	Command Action	Flags	Opcode	Hex	B C
shla ~	(.ix+d) ←(.ix+d)*2			11 0011 011	CB ~

| shl .iy+d| C ← (.iy+d) (7)| 1 1 0 P 0 1 | 11110101 | DD | 4 | 7 | shla | (.iy+d) ← (.iy+d)*2| | | 11 0011 011 | CB | 00 1000 | 26

| shr r | C←r(o) | 1 1 0 P 0 1 | 11000101 | CB | 2 | 2 | | | r ← r/2 | | | ~ | | ~

| shr .hl | C← (.hl) (0) | 1 1 0 P 0 1 | 11000111 | CB | 2 | 5 | | | (.hl) ← (.hl)/2| | | 3E | | 2 | shr .ix+d| C← (.ix+d) (0) | 1 1 0 P 0 1 | 11000101 | DD | 4 | 7 | | (.ix+d) ← (.ix+d)/2| | | 11 0011 011 | CB | -| 3E | shr .iy+d| C← (.iy+d) (0) | 1 1 0 P 0 1 | 11110101 | FD | 4 | 7 | | (.iy+d) ← (.iy+d)/2| | | 11 0011 011 | CB | - | 3E

| shrar | tmp←r(7); C← r(0) | 1 1 0 P 0 1 | 11000101 | CB | 2 | 2 | | r ← r/2; r(7)←tmp| | | | ~ | | shra .hl | tmp←(.hl) (7); C← (.hl) (0)| 1 1 0 P 0 1 | 11000111 | CB | 2| 5 | | (.hl) ← (.hl)/2; (.hl) (7)← tmp| | | 2E | shra .ix+d|tmp←(.ix+d) (7);C←(.ix+d) (0)| 1 1 0 P 0 1 | 11000101 | DD | 4 | 7 | |(.ix+d)←(.ix+d)/2; (.ix+d) (7)←tmp| | 11 0011 011 | CB | | 2E

| shra .iy+d|tmp←(.iy+d) (7);C←(.iy+d) (0)| 1 1 0 P 0 1 | 11110101 | FD | 4 | 7 | (.iy+d) ←(.iy+d)/2; (.iy+d) (7)← tmp | 11 0011 011 |CB| 2E

shl command and shla command are exactly the same, so the operand is the same -

(Note: The symbols and operations represented here are based on my understanding of the contents and can vary based on context. The 'C' seems to represent the carry flag, r(x) and r(0) represent bit shifts, etc.)

A.16 Branch Instructions

| Mnemonic | Instruction Operation | Flags | Operation Code | | | | S Z H V N C | (76 543 210) Hex | B | C | |----------|----------------------|---------------|-----------------|---|---| | br nn | .pc←nn | • • • • • • • • | 11000011 | C3 | 3 | 3 | | bnz nn | if z=0 | | • • • • • • • • | 11000010 | C2 | 3 | 3 | | | .pc←nn | | | | | | bz nn | if z=1 | • • • • • • • • | 11000100 | CA | 3 | 3 | | | .pc←nn | | | | | | | bnc nn | if c=0 | • • • • • • • • | 11001000 | D2 | 3 | 3 | | | .pc←nn | | | | | | | bc nn | if c=1 | • • • • • • • • | 11001100 | DA | 3 | 3 | | | .pc←nn | | | | | | | bponn | if pv=0 | • • • • • • • • | 11010000 | E2 | 3 | 3 | | | .pc←nn | | | | | | | | bpe nn | if pv=1 | • • • • • • • • | 11010100 | EA | 3 | 3 | | | .pc←nn | | | | | | | bp nn | if s=0 | • • • • • • • • | 11110000 | F2 | 3 | 3 | | | .pc←nn | | | | | | | bm nn | if s=1 | • • • • • • • • | 11110100 | FA | 3 | 3 | | | .pc←nn | | | | | | | br .hl | .pc←.hl | • • • • • • • • | 11101001 | E9 | 1 | 1 | | br .ix | .pc←.ix | | 11101101 | DD | 2 | 2 | | | | • • • • • • • • | 11101101 | E9 | | | | br .iy | .pc←.iy | | 11101101 |DD | 2 | 2 | | | | • • • • • • • • | 11101001 | E9 | | |

A.17 Call Commands

Mnemonic	Command Action	Flags	Instruction Code
short	.pc←.pc+e	·····	00011000 18 2 3
br e	←e-2	·····	
short	if Z=0	.pc←.pc+e	·····
bnz e	←e-2	·····	
short	if Z=1	.pc←.pc+e	·····
bz e	←e-2	·····	
short	if C=0	.pc←.pc+e	·····
bnc e	←e-2	·····	
short	if C=1	.pc←.pc+e	·····
bc	←e-2	·····	
dbnz e	.b←.b-1;if .b≠0	·····	0010100 10 2 2

Mnemonic	Command Action	Flags	Instruction Code
.pc←.pc+e	←e-2		

A.17 Call Commands

Mnemonic	Command Action	Flags	Instruction Code
call nn	[.sp-2]←.pc ;[.sp-1]←.pch	·····	11001101 CD 3 5
.sp←.sp-2 ;.pc←nn	·····		
call nz,nn	if Z=0	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call z,nn	if Z=1	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call nc,nn	if C=0	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call c,nn	if C=1	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call po,nn	if PV=0	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call pe,nn	if PV=1	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call p,nn	if S=0	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			
call m,nn	if S=1	[.sp-2]←.pc ;[.sp-1]←.pch	·····
.sp←.sp-2 ;.pc←nn			

Page 184

APPENDIX R800 Instruction List

Mnemonic	Command Actions	flags	Opcode	B
ret	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11001 001	C9
ret nz	if z=0	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11101 0
ret z	if z=1	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11101 0
ret nc	if c=0	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11101 1
ret c	if c=1	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11101 0
ret po	if pv=0	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11100 0
ret pe	if pv=1	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11100 1
ret p	if s=0	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11110 0
ret m	if s=1	pc ← (sp); pch ← (sp+1); sp ← sp+2	SZHNC	11110 1
reti	interrupt return		SZHNC	110101
retn	Non Maskable Interrupt return	SZHNC	1110 101	ED
brk k	sp-2 ← pc; sp-1 ← pch; sp ← sp-2; pc ← k; pch ← 0	SZHNC	11 k 111	

Note: Some symbols may not display precisely due to font limitations.

A.18 Input/Output Commands

Mnemonic	Command Action	Flags	Opcode
in a, [n]	a ← [n]	• • • • •	11 010 011
in a, [c]	a ← [c]	• n • •	11 101 101
in r, [c]	r ← [c]	↑ ↓ P O •	01 r 000
in [c]	[c]	↑ ↓ P O •	01 r 000
out [n], a	[n] ← a	• • • • •	11 010 011
out [c], r	[c] ← r	• • • • •	11 101 101

Note: *1. If b = 1, the value is 1; otherwise, it is 0. *2. The contents of [c] and the flags change according to the port indicated by the register, but the contents are not stored anywhere.

This text appears to be part of a technical manual, likely for assembly language or machine code instructions for some kind of processor or microcontroller.

A.19 CPU Control Instructions

Mnemonic	Operation Description	Flags	Opcode
adj .a	adjust to decimal		10001111 27
not .a	a ← NOT .a		10001111 2F
neg .a	a ← NOT .a + 1		11101101 ED 44
notc	c ← NOT c		01111111 3F
setc	c ← 1		00101111 37
nop	NO operation		00000000 00
halt	HALT		01111110 76
di	IFF ← 0		11110011 F3
ei	IFF ← 1		11111011 FB
im 0	interrupt mode 0		11101101 ED 46
im 1	interrupt mode 1		11101101 ED 56
im 2	interrupt mode 2		11101101 ED 5E

Note:

- S: Sign flag
- Z: Zero flag
- H: Half carry flag
- P/V: Parity/Overflow flag
- N: Add/Subtract flag
- C: Carry flag
- B: Borrow flag

Index

A

I/O Ports ..... 57 Access Time ..... 22 Address ..... 48 Address Bus ..... 48 Interlace ..... 111 Wait Function ..... 95 ADSR ..... 133 SECAM ..... 111 SRAM ..... 50 NTSC ..... 110 FM-BIOS ..... 139 MSX-Engine ..... 17 MSX-JE ..... 72 MSX-MUSIC ..... 131 Types of MSX ..... 62 Envelope ..... 133 OPLL YM2413 ..... 131 OPLL Driver ..... 156 Operators ..... 131 Sound Noise 133

Ka

MSX made for export overseas ..... 62 Expansion BIOS ..... 62 Kanji ROM expansion 21

Kanji Graphic Mode.....77 Kanji Text Mode.....77
Kanji Driver.....74 Perfect Average.....134
Capacity.....99 Carrier Operator.....137
K (Kilo).....48 Square Wave.....132
Command Register.....92 Control Register.....92

[Sa] Sub ROM.....52 Sampling Synthesizer.....
CPU.....48 Hue Phase.....99
JIS Code.....74 System Timer.....20,
27 System Work Area.....61 Shift JIS Code.....74
Hexadecimal.....48 Main Memory.....50
Vertical Line Insertion.....114 Horizontal Resolution.....11
Status Register.....92 Superimpose.....113
Slot Expansion.....55 Correct Display.....132
Full-size Characters.....74 Software Stack.....78

[Ta] Timer Insertion.....114 Single Kanji Conversion.....
TAND.....106 D/A Converter.....20

Index 189

DRAM page access.....24 DRAM mode.....
TC9769.....17 Disk presence detection.....62
Diskware.....61 Data bus.....48
Sync signal.....110

[Na] Binary.....48 Residual wave.....133
Non-interlace.....111

[Ha] BIOS.....57 Byte.....48
PAL.....111 Half-width character.....74
Channel.....48 PSG.....130
Bit.....48 Bit image print.....79
Video RAM (VRAM).....50 Video RAM capacity.....
Video digitize.....113 VDP command.....95
VDP mode.....82 Hook.....121
Programmable Sound Generator.....130 Basic Input Output System.....57 Page.....51 Pause key control.....20

[Ma] Microsoft kanji code.....74 Multi-scan monitor.....
Undefined command.....98

Main ROM ..... 52 Main Memory.....
50 Memory Mapper..... 22 MORALE.....
113 Modulator Operator..... 137

Ya USR Function..... 119

Ra RAM..... 50 Rhythm Tone.....
137 Reset Status..... 22 Clause Conversion.....
72 ROM..... 50

References

[1] ASCII-Microsoft FE Headquarters, Japan Musical Instruments Manufacturing Co., Ltd. ”V9938 MSX-VIDEO Technical Data Book”, ASCII, 1985 (Out of Print)

[2] ASCII Corporation, ”V9958 Specification Document”, Not for Sale, 1988

[3] Supervising ASCII-Microsoft FE, ”MSX2 Technical Handbook”, ASCII, 1986

[4] Noboru Sugiyama, ”MSX2+ Powerful Utilization Method”, ASCII, 1989

[5] ASCII Corporation, ”MSX-Datapack”, ASCII, 1991

Author's History

Naota Ishikawa

Graduated from Yokohama National University and joined ASCII Corporation, where he was involved in the development of MSX. Graduated from the Department of Information Science, Faculty of Science, University of Tokyo, as well as from the Graduate School of Industrial Science and Technology at the same university. Currently, he is a part-time lecturer at the Faculty of Education, Tokyo Gakugei University. He specializes in information processing with microcomputers such as MSX, electronic circuits, digital processing, and online information processing. Email: naotas@slab.sfc.keio.ac.jp

MSX turbo R Technical Handbook

- First Edition: July 31, 1991
- Third Edition: June 15, 1992
- Price: 2,500 yen (excluding tax, 2,427 yen)
- Author: Naota Ishikawa
- Editor: Susumu Fukudome
- Publisher: ASCII Corporation
- Publisher Location:

```
- 107-24 3-11-1 South Aoyama Building Suite E102, Minato-ku, Tokyo
- **Phone:** Tokyo (03) 3486-7114, Osaka (03) 3486-1977 (Daiwa Line)
```


- Production: Tokyo Printing University Co., Ltd.
- Printed: Tokyo Printing University Co., Ltd.
- Editorial Department: MSX Magazine Editorial Department
- ISBN: 4-7561-0621-8
- Printed in Japan

(Blank page in the original.)

(Blank page in the original.)

(Blank page in the original.)

Table of Contents

- MSX turbo R System Configuration
- R800 Programming Techniques
- List of BASIC Commands: Addition, Deletion, Modification
- List of Extended BIOS Commands: Addition, Deletion, Modification
- PCM Recording, Playback Functions
- MSX2+ and MSX turbo R Slot Structure
- Overview of Kanji BASIC
- V9958VDP Specifications
- Detailed Natural Screen Display Mode
- Implementation of Scanline Interruption
- Control of MSX-MUSIC

- Color Data List of BASIC and BIOS
- R800CPU Instruction Code Table

Price: 2,500 yen (Base price 2,427 yen)

ISBN4-7561-0621-8 C3055 P2500E